

武汉大学计算机学院

本科生实验报告

os 内核实验

专 业 名 称 : 计算机科学与技术

课 程 名 称 : 操作系统实践 A

指 导 教 师 : 蔡朝晖 教授

学 生 学 号 : 2023302111199

学 生 姓 名 : 黄文婷

二〇二五 年 十二 月

目录

1	实验一：RISC-V 引导与裸机启动	4
1.1.	1. 实验目的	4
1.2.	2. 系统设计与代码组织	4
1.3.	3. 关键数据结构与符号	4
1.4.	4. 实现步骤	4
1.5.	5. 调试与测试流程	5
1.6.	6. 问题与解决方案	5
1.7.	8. 运行结果	6
2	实验二：内存与页表管理	9
2.1.	测试验证部分	27
3	实验三：中断处理实现	30
3.1.	一、实验目的	30
3.2.	二、实验原理	31
3.3.	三、实验内容	33
3.4.	四、中断处理流程	39
3.5.	五、关键数据结构	41
3.6.	六、实验测试	41
4	实验四：特权级转换与首个用户态进程创建	46
4.1.	一、实验目的	46
4.2.	二、实验原理	46
4.3.	三、实验内容	49
4.4.	四、关键技术点	59
4.5.	五、遇到的问题及解决方案	61
4.6.	六、实验测试	64
5	实验五：用户虚拟内存管理与系统调用实现	66
5.1.	一、实验目的	66
5.2.	二、实验原理	66
5.3.	三、实验内容	70
5.4.	四、实验结果与测试	82

5.5.	五、实验心得	83
5.6.	六、实验总结	85
6	实验六：进程管理 实验报告	85
6.1.	一、实验目标	85
6.2.	二、实验内容	86
6.3.	三、遇到的问题及解决方案	87
6.4.	四、实验结果	97
6.5.	五、实验总结	97
7	实验七：文件系统实验报告	98
7.1.	实验目标	98
7.2.	实验概述	98
7.3.	磁盘布局与文件系统设计	99
7.4.	遇到的问题及解决方案	102
7.5.	sysfile.c 和 sysproc.c 的实现总结	110
7.6.	核心功能模块详解	115
7.7.	测试	123

1 实验一：RISC-V 引导与裸机启动

1.1.1. 实验目的

- 理解 RISC-V 的启动流程（从 `entry.S` -> `start.c` -> `main.c`）以及多核引导的基本思路。
- 实现最小串口输出（基于 UART），能在 QEMU 上看到指定输出。
- 掌握调试方法（GDB + QEMU），能定位启动或输出问题。

1.2.2. 系统设计与代码组织

仓库主结构（与本次实验相关）：

- `kernel/boot/`：启动入口与 C 启动代码（`entry.S`, `start.c`, `main.c`）
- `kernel/dev/`：设备驱动（`uart.c`）
- `kernel/lib/`：打印与同步库（`print.c`, `spinlock.c`）
- `kernel/proc/`：与处理器/多核相关代码
- `kernel/kernel.ld`：链接脚本，定义入口与段布局

设计说明要点：

- 入口点 `_entry` 被放置在 `0x80000000`（由链接脚本指定），QEMU 会从这里开始执行。
- `entry.S` 负责最早期的处理：设置栈、将控制权交给 C 代码（`start.c`）。
- `start.c` 完成从 Machine mode 到 Supervisor/User mode 的必要设置，并最终跳转到 `main.c`。
- `uart.c` 提供最小的串口字符发送函数；`print.c` 封装 `printf`，并使用自旋锁保护输出以避免并发混乱。

1.3.3. 关键数据结构与符号

- `CPU_stack`：为每个 hart 保留一段内核栈空间，通常在 `start.c` 中定义为 `uint8 CPU_stack[NCPU * STACKSIZE]`。
- 链接脚本中常见符号：`_entry`, `etext`, `edata`, `end`，用于代码/数据段边界和 BSS 清零。

1.4.4. 实现步骤

1. 阅读并理解 `kernel/entry.S` 中的启动序列：设置初始栈地址、调用 `start`。

2. 在 `start.c` 中完成多核/模式切换逻辑（必要时），并跳转到 `main`。
3. 在 `lib/print.c` 中实现 `print_init` 与 `printf`。
4. 实现 `spinlock.c` 的简单自旋锁（基于原子交换或 `lr/sc` 指令），并在 `printf` 中使用锁保护串口操作。
5. 使用 Makefile 生成两个产物：
 - `kernel-qemu.elf`（带调试符号的 ELF，用于 gdb 加载）
 - `kernel-qemu`（裸二进制镜像，用于 QEMU 启动，使用 `objcopy -O binary` 从 ELF 生成）

1.5.5. 调试与测试流程

1. 编译并生成镜像：

```
1 make clean
2 make build --directory=kernel
3
```

sh

2. 在一个终端启动 QEMU，并暂停等待 gdb 连接（端口示例 26000）：

```
1 qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m
  128M -smp 2 -nographic -serial mon:stdio -S -gdb tcp::26000
2
```

sh

3. 在另一终端启动 gdb 并加载 ELF（带符号文件）：

```
1 gdb-multiarch kernel-qemu.elf
2 (gdb) target remote :26000
3 (gdb) b _entry
4 (gdb) c
5
```

sh

4. 到达入口或 `main` 后，使用 `n / s` 单步执行，或启用 TUI（`layout asm` 或 `gdb -tui`）查看源码与汇编。
5. 检查 UART 输出函数（`uart_putc` 与 `uart_init`）是否被调用：设置断点 `b uart_putc`、`b uart_init`。
6. 典型调试指令：’ `(gdb) x/10i $pc`

1.6.6. 问题与解决方案

- 没有任何串口输出。

- 排查：确认 `_entry` 是否被暴露并正确链接到 `0x80000000`；检查 `uart` 基地址是否与 QEMU 的 `virt` 设备一致（通常为 `0x10000000`）；在汇编最早阶段直接写一个字符到 UART 来验证硬件连通。
- 解决：在 `entry.S` 中添加 `.globl _entry` 并重新编译；确认 `kernel.ld` 将 `_entry` 放到 `0x80000000`。
- gdb 无法定位函数（Cannot find bounds of current function / 无调试符号）
 - 排查：确认 ELF 未被 strip，且 gdb 加载的是 ELF（非裸二进制）。
 - 解决：链接产生 `kernel-qemu.elf`（保留符号），用 `objcopy` 生成裸二进制；在 gdb 中加载 `kernel-qemu.elf`。
- 执行 `make qemu` 后观察到以下现象：
 - 链接警告：


```
riscv64-linux-gnu-ld: 警告：无法找到项目符号 _entry; 缺省为 0000000080000000。
```

 原因是 `entry.S` 中 `_entry` 未被 `.globl` 导出。
 - 在早期 `Makefile` 中，`ls` 的通配符写法不当导致 `ls` 报错（`ls: 无法访问 './*/*/*.o': 没有那个文件或目录`），已修正为只匹配 `./*/*.o`。
 - gdb 报 `file format not recognized` 是因为使用了裸二进制 `kernel-qemu` 作 gdb 加载对象。已改为生成 `kernel-qemu.elf` 并用其进行调试。

1.7.8. 运行结果

- 添加输出前

```
hwt@WP:~/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code$ make qemu
make build --directory=kernel
make[1]: 进入目录"/home/hwt/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code/kernel"
make build --directory=boot/
make[2]: 进入目录"/home/hwt/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code/kernel/boot"
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -I ../include -c entry.S
make[2]: 离开目录"/home/hwt/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code/kernel/boot"
make build --directory=dev/
make[2]: 进入目录"/home/hwt/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code/kernel/dev"
make[2]: "build"已是最新。
make[2]: 离开目录"/home/hwt/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code/kernel/dev"
make build --directory=lib/
make[2]: 进入目录"/home/hwt/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code/kernel/lib"
make[2]: "build"已是最新。
make[2]: 离开目录"/home/hwt/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code/kernel/lib"
make build --directory=proc/
make[2]: 进入目录"/home/hwt/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code/kernel/proc"
make[2]: "build"已是最新。
make[2]: 离开目录"/home/hwt/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code/kernel/proc"
make[1]: 离开目录"/home/hwt/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code/kernel"
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 2 -nographic
QEMU: Terminated
hwt@WP:~/sources/lab/OSLab/WHUOS/whu-oslab-lab1/code$ git checkout -b Lab-1
切换到新分支 'Lab-1'
```

- 添加输出后

```

hwt@WP:~/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code$ make qemu
make build --directory=kernel
make[1]: 进入目录"/home/hwt/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code/kernel"
make build --directory=boot/
make[2]: 进入目录"/home/hwt/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code/kernel/boot"
make[2]: "build"已是最新。
make[2]: 离开目录"/home/hwt/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code/kernel/boot"
make build --directory=dev/
make[2]: 进入目录"/home/hwt/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code/kernel/dev"
make[2]: "build"已是最新。
make[2]: 离开目录"/home/hwt/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code/kernel/dev"
make build --directory=lib/
make[2]: 进入目录"/home/hwt/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code/kernel/lib"
make[2]: "build"已是最新。
make[2]: 离开目录"/home/hwt/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code/kernel/lib"
make build --directory=proc/
make[2]: 进入目录"/home/hwt/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code/kernel/proc"
make[2]: "build"已是最新。
make[2]: 离开目录"/home/hwt/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code/kernel/proc"
make[1]: 离开目录"/home/hwt/sources/lab/OSLab/WHU0S/whu-oslab-lab1/code/kernel"
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 2 -nographic -serial mon:stdio
cpu 0 is booting!
cpu 1 is booting!
□

```

- 加法使用锁前

```

cpu 0 is booting!
cpu 1 is booting!
cpu 0 report: sum = 1013947
cpu 1 report: sum = 1035034
□

```

- 加法使用锁后

- 问题回答: 要修复 **sum** 的不一致性, 保证每次更新都在临界区内或使用原子操作; 为最佳性能, 采用本地累加 + 一次合并或原子加, 而不要在持锁期间做 **printf** 等长操作
- 结果图

```

cpu 0 is booting!
cpu 1 is booting!
cpu 0 report: sum = 1000000
cpu 1 report: sum = 2000000
□

```

- 对应代码

```

1 #include "riscv.h"
2 #include "lib/print.h"
3 #include "lib/lock.h"
4
5 volatile static int started = 0;
6
7 volatile static int sum = 0;
8
9 static spinlock_t sum_lock;
10
11 int main()
12 {
13     int cpuid = r_tp();
14     spinlock_init(&sum_lock, "sum");
15     if(cpuid == 0) {
16         print_init();
17         printf("cpu %d is booting!\n", cpuid);
18         __sync_synchronize();
19         /* 启动其它 CPU */
20         started = 1;
21         spinlock_acquire(&sum_lock);
22         for(int i = 0; i < 1000000; i++) {
23             sum++;
24         }
25         spinlock_release(&sum_lock);
26         printf("cpu %d report: sum = %d\n", cpuid, sum);
27     } else {
28         while(started == 0);
29         __sync_synchronize();
30         printf("cpu %d is booting!\n", cpuid);
31         spinlock_acquire(&sum_lock);
32         for(int i = 0; i < 1000000; i++) {
33             sum++;
34         }
35         spinlock_release(&sum_lock);
36         printf("cpu %d report: sum = %d\n", cpuid, sum);
37     }
38     while (1);
39 }

```

- print 去掉锁

- 结果图

```

al mon:stdio
ccpu pu 01 is booting!

QEMU: Terminated
kubepb: /sources/lab/OSlab/WHIS/whu-oslab-lab1/code$ make clean ff make qemu

qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 2 -nographic -seri
al mon:stdio
ccpu pu 01 is booting!

□

```

- 对应修改: 去掉 start 变量和 print 中使用了 spin_lock 的相关语句

2 实验二：内存与页表管理

2.0.1. 关键数据结构

2.0.1.1. page_node_t

- 描述: 物理页节点，用于链表连接。
- 结构:

```
1 typedef struct page_node {
2     struct page_node* next;
3 } page_node_t;
4
```

C

2.0.1.2. alloc_region_t

- 描述: 可分配区域结构，管理若干物理页。
- 结构:

```
1 typedef struct alloc_region {
2     uint64 begin; // 起始物理地址（包含）
3     uint64 end;   // 终止物理地址（不包含）
4     spinlock_t lk; // 自旋锁，保护下面的链表和计数器
5     uint32 allocable; // 可分配页面计数
6     page_node_t list_head; // 哨兵链表头节点（list_head.next 指向第
    一个可用页）
7 } alloc_region_t;
8
```

C

2.0.1.3. pte_t 和 pgtbl_t

- 描述: 页表项和页表类型。
- 定义:

```
1 typedef uint64 pte_t;
2 typedef uint64* pgtbl_t;
3
```

C

2.0.2. 与 xv6 对比分析

2.0.2.1. pmem.c vs kalloc.c

2.0.2.1.1. 基本功能

- **kalloc.c**: 单一的全局物理页池（`kmem.freelist`），初始化时把 `end` 到 `PHYSTOP` 的所有页面放入同一个自由链表，分配/释放都从这个池操作。

- **pmem.c:** 把物理页池划分为两个独立的区域: `kern_region` (内核专用) 和 `user_region` (用户专用), 分区基于 `ALLOC_BEGIN / ALLOC_END` 和 `KERNEL_PAGES`。分配/释放时通过 `in_kernel` 参数选择区域。

2.0.2.1.2.接口差异

- **kalloc.c:** 提供 `kalloc(void)` 和 `kfree(void *pa)`。分配器不区分调用者是内核还是用户, 调用者自己负责语义上的用途区分。
- **pmem.c:** 提供 `pmem_alloc(bool in_kernel)` 和 `pmem_free(uint64 page, bool in_kernel)`, 调用者必须显式说明是内核分配还是用户分配, 从而使用不同的物理页池。

2.0.2.1.3.并发与锁

- **kalloc.c:** 一个全局自旋锁 `kmem.lock` 保护单一 `freelist`; 高并发下该锁可能成为瓶颈。
- **pmem.c:** 每个区域维护自己的自旋锁 `spinlock_t lk ( kern_region.lk 和 user_region.lk )`, 在理论上减少了跨内核/用户竞争 (内核和用户请求通常不在同一锁上竞争), 但同一区域内仍可能发生竞争。

2.0.2.1.4.安全性与鲁棒性

- **kalloc.c:** 没有区分内核与用户资源, 恶意用户不断分配页会耗尽全局物理页, 影响内核资源, 存在被 DOS 的风险。
- **pmem.c:** 通过 `KERNEL_PAGES` 保留内核专用页, 能有效防止用户耗尽内核必须的物理页, 提升内核抗攻击性。但若用户区耗尽, 用户程序仍会失败/阻塞, 这符合设计目标。

2.0.2.1.5.配置与适应性

- **kalloc.c:** 依赖链接器符号 `end` 和常量 `PHYSTOP`, 对链接脚本要求较低且直接。
- **pmem.c:** 依赖 `ALLOC_BEGIN / ALLOC_END` linker 符号, 且默认 `KERNEL_PAGES` 为 6 (可由 `-DKERNEL_PAGES=N` 覆盖)。`pmem_init` 会确保区域按页对齐并且总页数至少 $\geq$ `KERNEL_PAGES`。

2.0.2.1.6.调试/保护措施

- 两者共同点: 都用 `memset` 在 `free/alloc` 时填充字节以帮助检测悬空引用; 都使用 `panic` 在遇到非法释放或不符合预期的范围时中止。

- **pmem.c 额外:** 增加了对页面是否属于指定区域的边界检查（避免错误地把用户页释放到内核池或反向），从而增加了安全性和调试能力。

2.0.2.1.7.时间复杂度

- **两者:** 分配/释放都是 $O(1)$ （在持锁的情况下从链表头插入/弹出），所以性能基线相近。

2.0.3. 设计决策理由

2.0.3.0.1.内存分配器设计决策

- **单链表结构:** 简单高效，分配和释放都是 $O(1)$ 时间复杂度，适合页级分配。
- **零额外内存开销:** 将元数据存放在空闲页面本身，避免额外的位图或数组，节省空间。
- **内核与用户区域分离:** 防止用户进程耗尽内核资源，提升系统安全性。
- **自旋锁保护:** 在多核环境下保证并发安全。
- **填充页面检测悬空引用:** 在分配和释放时用不同值填充页面，便于调试内存错误。

2.0.3.0.2.页表管理设计决策

- **SV39 分页模式:** RISC-V 标准，支持 39 位虚拟地址空间。
- **权限位设置:** 区分读、写、执行、用户等权限，确保内存安全。
- **恒等映射内核区域:** 简化内核访问物理内存的逻辑。
- **设备 MMIO 映射:** 将设备地址映射到虚拟地址空间，便于内核访问。

2.0.4. 实验过程部分

本实验旨在实现一个基于 RISC-V 架构的操作系统内核中的物理内存分配器和虚拟内存管理模块。实验过程分为以下几个阶段：

1. 需求分析与设计

- 分析 xv6 操作系统中的内存管理机制，理解物理内存分配器（`kalloc.c`）和虚拟内存管理（`vm.c`）的实现原理。
- 设计改进方案，包括将物理内存分配器分为内核和用户区域，以提高安全性。

2. 代码实现

- 基于 xv6 的代码，重新实现物理内存分配器（`pmem.c`）和虚拟内存管理器（`vmem.c`）。
- 确保代码符合项目的接口和宏定义。

3. 集成与调试

- 将新实现的模块集成到项目中，解决编译和链接错误，确保模块能正确工作。

4. 测试与验证

- 编写测试用例，验证内存分配和虚拟内存映射的功能正确性。

2.0.5. 实现步骤记录

1. 分析 xv6 源码

- 阅读 xv6 的 `kalloc.c` 和 `vm.c` 文件，理解物理内存分配和虚拟内存管理的实现。
- 分析数据结构和关键函数的逻辑。

2. 设计 `pmem.c`

- 定义 `page_node_t` 和 `alloc_region_t` 结构体。
- 实现 `pmem_init()` 函数，初始化内核和用户区域。
- 实现 `pmem_alloc()` 和 `pmem_free()` 函数，支持从指定区域分配和释放内存。
- 添加自旋锁保护并发访问。

3. 设计 `vmem.c`

- 定义 `pte_t` 和 `pgtbl_t` 类型。
- 实现 `vm_getpte()` 函数，遍历页表并返回 PTE 指针。
- 实现 `vm_mappages()` 和 `vm_unmappages()` 函数，进行虚拟地址到物理地址的映射和解除映射。
- 实现 `kvm_init()` 函数，创建内核页表并映射设备和内核区域。
- 实现 `kvm_inithart()` 函数，激活页表。

4. 解决宏定义冲突

- 将重复的宏定义从 `riscv.h` 移到 `vmem.h` 中。
- 更新 `vmem.c` 使用正确的宏名称。

5. 修复链接符号错误

- 将 `_etext` 改为 `etext`，以匹配链接脚本中的定义。

6. 集成到项目

- 将 `pmem.c` 和 `vmem.c` 添加到项目的源文件列表。
- 更新 `Makefile` 确保编译包含新文件。

7. 调试与测试

- 运行 `make` 命令，检查编译错误。

- 修复遇到的语法和链接错误。
- 编写简单的测试代码验证功能。

2.0.6. 问题与解决方案

2.0.6.0.1.1. 宏定义重复问题

- 问题: riscv.h 和 vmem.h 中有重复的宏定义, 如 PTE_V、PA2PTE 等, 导致编译冲突。
- 解决方案: 将页表相关的宏定义移到 vmem.h 中, 并在 riscv.h 中添加注释说明。更新 vmem.c 使用 vmem.h 中的宏。

2.0.6.0.2.2. 链接符号未定义错误

- 问题: 编译时出现 undefined reference to `_etext`, 因为链接脚本中使用的是 `etext`。
- 解决方案: 将 vmem.c 中的 `extern char _etext[]` 改为 `extern char etext[]`, 并更新所有引用。

2.0.6.0.3.3. 隐式函数声明警告

- 问题: 编译时出现 implicit declaration of function ‘proc_mapstacks’, 因为该函数未声明。
- 解决方案: 暂时移除对 proc_mapstacks 的调用, 并在注释中说明稍后实现。

2.0.6.0.4.4. 编译器警告处理

- 问题: 编译时出现 -Werror=implicit-function-declaration, 导致编译失败。
- 解决方案: 移除未实现的函数调用, 或添加函数声明。

2.0.6.0.5.5. 自旋锁使用导致的 panic 问题

- 问题描述: 在测试物理内存分配器时, 使用自旋锁保护共享变量会导致系统 panic, 报错信息为 “panic: double acquire detected”。具体表现为:
 - 在不使用锁时, 内存分配和释放功能正常
 - 一旦在循环中使用 `spinlock_acquire()` 和 `spinlock_release()` 保护临界区, 系统就会触发 panic
 - 单次获取和释放锁可以正常工作, 但在循环中频繁获取释放锁时就会出现问题
- 问题排查过程:
 1. 初步分析: 怀疑是重复获取锁导致的, 但检查代码逻辑发现并没有嵌套获取同一个锁。

2. 调试信息: 在 `spinlock_acquire` 中添加调试输出, 发现 panic 时的状态显示:

- `mycpuid() = 0`
- `lk->locked = 0` 或 `1` (不一致)
- `lk->cpuid = 0`
- 但 `spinlock_holding(lk)` 却返回 `true`

3. 深入分析: 发现在调用 `spinlock_holding()` 检查时和打印调试信息时, 锁的状态不一致, 说明存在竞态条件。

4. 根本原因定位:

- 问题出在 `spinlock_init()` 函数中, 将 `lk->cpuid` 初始化为 `0`
- 同时 `spinlock_release()` 函数在释放锁后也将 `lk->cpuid` 重置为 `0`
- 而 CPU 0 的 ID 正好也是 `0`, 导致以下问题:
 - 在锁未被持有时, `lk->cpuid == 0`
 - 当 CPU 0 尝试获取锁时, `spinlock_holding()` 检查 `(lk->locked && lk->cpuid == mycpuid())`
 - 在特定的时序下 (例如释放锁后、或初始化状态), `lk->cpuid` 为 `0` 与 CPU 0 的 ID 相同
 - 导致 `spinlock_holding()` 误判为 CPU 0 已经持有该锁, 从而触发 “double acquire” panic

• 解决方案:

1. 修改初始化值: 将 `spinlock_init()` 中的 `lk->cpuid` 初始化值从 `0` 改为 `-1` (表示无效的 CPU ID)
2. 修改释放后的值: 将 `spinlock_release()` 中释放锁后的 `lk->cpuid` 重置值也改为 `-1`
3. 添加 CPU 初始化: 在 `main()` 函数开始时, 添加 `cpu_init()` 函数显式初始化所有 CPU 的结构体 (`cpu_t` 中的 `noff` 和 `origin` 字段)
4. 优化检查逻辑: 简化 `spinlock_holding()` 的判断逻辑为 `(lk->locked && (lk->cpuid == mycpuid()))`

• 代码修改:

在 `kernel/lib/spinlock.c` 中:

```
1 // 自旋锁初始化
```

C

```

2 void spinlock_init(spinlock_t *lk, char *name)
3 {
4     lk->name = name;
5     lk->locked = 0;
6     lk->cpuid = -1; // 初始化为无效的 CPU ID, 避免与 CPU 0 冲突
7 }
8
9 // 释放自旋锁
10 void spinlock_release(spinlock_t *lk)
11 {
12     if(!spinlock_holding(lk))
13         panic("release");
14
15     lk->cpuid = -1; // 重置为无效的 CPU ID
16
17     __sync_synchronize();
18     __sync_lock_release(&lk->locked);
19     pop_off();
20 }
21
22 // 检查是否持有锁
23 bool spinlock_holding(spinlock_t *lk)
24 {
25     int r;
26     r = (lk->locked && (lk->cpuid == mycpuid()));
27     return r;
28 }
29

```

在 `kernel/proc/proc.c` 中添加 CPU 初始化函数:

C

```

1 void cpu_init(void)
2 {
3     // 显式初始化所有 CPU 结构
4     for(int i = 0; i < NCPU; i++) {
5         cpus[i].noff = 0;
6         cpus[i].origin = 0;
7     }
8 }
9

```

在 `kernel/boot/main.c` 中调用初始化:

C

```

1 int main()
2 {
3     int cpuid = r_tp();
4     if(cpuid == 0) {
5         cpu_init(); // 初始化 CPU 结构
6         spinlock_init(&sum_lock, "sum");
7         print_init();
8     }
9 }

```

```

8      // ...
9      }
10     // ...
11 }
12

```

- 经验教训:

1. 避免使用 0 作为特殊值: 在设计数据结构时, 应该避免使用 0 或其他可能与有效值重叠的数字作为“无效“或“未初始化“的标记值, 建议使用 -1 或其他明确的无效值。
2. 多核并发的复杂性: 在多核环境下, 即使看似简单的初始化值也可能引发竞态条件和难以调试的问题。
3. 调试技巧: 当遇到状态不一致的问题时, 可能是竞态条件导致的。在调试输出时, 状态可能已经被其他操作改变。
4. 全面的初始化: 所有全局或静态数据结构都应该显式初始化, 不要依赖 C 语言的默认零初始化, 特别是在多核环境中。

2.0.6.0.6.6. 虚拟内存映射相关问题

2.0.6.0.6.1.6.1 vm_mappages 不允许重新映射导致的 panic

- 问题描述: 在测试虚拟内存映射功能时, 尝试修改已有映射的权限(如从只读改为只写)会触发 panic, 报错信息为“panic: vm_mappages: remap”。具体测试场景:

```

1 // test-1: 首次映射虚拟地址 0, 权限为只读
2 vm_mappages(test_pgtbl, 0, mem[0], PGSIZE, PTE_R);
3
4 // test-2: 尝试修改虚拟地址 0 的映射权限为只写
5 vm_mappages(test_pgtbl, 0, mem[0], PGSIZE, PTE_W); // ← 触发
   panic
6

```

- 问题根源: 在 vm_mappages 函数中有严格的重映射检查:

```

1 pte_t *pte = vm_getpte(pgtbl, a, true);
2 if (!pte)
3     panic("vm_mappages: out of memory");
4 if (*pte & PTE_V) // ← 检测到 PTE 已经有效
5     panic("vm_mappages: remap"); // ← 直接 panic, 不允许更新映射
6

```


这种设计虽然可以防止意外的重复映射，但过于严格，不支持合法的权限修改需求。

- 设计考量:

- ▶ 禁止 **remap** 的理由:

1. 防止内存泄漏：旧的物理页映射被覆盖后无法追踪和释放
2. 防止权限混乱：不经意地改变已有映射的权限可能导致安全问题
3. 检测编程错误：通常重复映射是逻辑错误的信号

- ▶ 允许 **remap** 的场景:

1. 修改页面权限（如从只读改为可写，用于写时复制）
2. 更新物理页映射（如页面换出/换入）
3. 调试和测试场景

- 解决方案: 允许更新已有映射，移除或修改过于严格的检查:

```
1 void vm_mappages(pgtbl_t pgtbl, uint64 va, uint64 pa, uint64
  len, int perm)
2 {
3     if (len == 0)
4         panic("vm_mappages: len == 0");
5
6     uint64 a = PG_ROUND_DOWN(va);
7     uint64 last = PG_ROUND_DOWN(va + len - 1);
8
9     for (;;) {
10         pte_t *pte = vm_getpte(pgtbl, a, true);
11         if (!pte)
12             panic("vm_mappages: out of memory");
13
14         // 移除严格的 remap 检查，允许更新已有映射
15         // if (*pte & PTE_V)
16         //     panic("vm_mappages: remap");
17
18         *pte = PA_TO_PTE(pa) | perm | PTE_V;
19         if (a == last)
20             break;
21         a += PGSIZE;
22         pa += PGSIZE;
23     }
24 }
25
```

C

2.0.6.0.6.2.6.2 vm_unmappages 物理页释放时的区域判断错误

- 问题描述: 在测试中释放映射的物理页时触发 panic, 报错信息为 “panic: pmem_free: page out of range”。具体场景:

```
1 // test-1: 从用户区分配物理页并建立映射
2 mem[1] = (uint64)pmem_alloc(false); // false 表示用户区
3 vm_mappages(test_pgtbl, PGSIZE * 10, mem[1], PGSIZE, PTE_R |
  PTE_W);
4
5 // test-2: 解除映射并释放物理页
6 vm_unmappages(test_pgtbl, PGSIZE * 10, PGSIZE, true); // ← 触
  发 panic
7
```

- 问题根源: 原始 vm_unmappages 函数硬编码将物理页释放到内核区:

```
1 void vm_unmappages(pgtbl_t pgtbl, uint64 va, uint64 len, bool
  freeit)
2 {
3     // ...
4     if (freeit) {
5         uint64 pa = PTE_TO_PA(*pte);
6         pmem_free(pa, true); // ← 硬编码 true, 总是释放到内核区
7     }
8     // ...
9 }
10
```

当物理页实际上是从用户区分配的 (pmem_alloc(false)), 但却尝试释放到内核区时, pmem_free 会检查地址范围并触发 panic。

- pmem_free 的区域检查逻辑:

```
1 void pmem_free(uint64 page, bool in_kernel)
2 {
3     alloc_region_t *r = in_kernel ? &kern_region :
  &user_region;
4
5     // 检查页面是否在指定区域范围内
6     if (page < r->begin || page >= r->end)
7         panic("pmem_free: page out of range");
8     // ...
9 }
10
```

- 解决方案: 自动检测物理页属于哪个区域, 然后释放到正确的区域:

C

```

1 void vm_unmappages(pgtbl_t pgtbl, uint64 va, uint64 len, bool
  freeit)
2 {
3     if ((va % PGSIZE) != 0)
4         panic("vm_unmappages: not aligned");
5
6     uint64 a;
7     for (a = va; a < va + len; a += PGSIZE) {
8         pte_t *pte = vm_getpte(pgtbl, a, false);
9         if (!pte)
10            panic("vm_unmappages: walk");
11        if (!(*pte & PTE_V))
12            panic("vm_unmappages: not mapped");
13        if (PTE_FLAGS(*pte) == PTE_V)
14            panic("vm_unmappages: not a leaf");
15
16        if (freeit) {
17            uint64 pa = PTE_TO_PA(*pte);
18
19            // 自动判断物理页属于哪个区域
20            uint64 kern_end = (uint64)ALLOC_BEGIN +
                KERNEL_PAGES * PGSIZE;
21            bool in_kernel = (pa >= (uint64)ALLOC_BEGIN && pa
                < kern_end);
22
23            // 释放到正确的区域
24            pmem_free(pa, in_kernel);
25        }
26        *pte = 0;
27    }
28 }
29

```

- 替代方案: 如果希望调用者明确指定释放区域, 可以添加参数:

C

```

1 void vm_unmappages_ex(pgtbl_t pgtbl, uint64 va, uint64 len,
2                        bool freeit, bool free_to_kernel)
3 {
4     // ...
5     if (freeit) {
6         uint64 pa = PTE_TO_PA(*pte);
7         pmem_free(pa, free_to_kernel);
8     }
9     // ...
10 }
11

```

- 经验教训:

1. 避免硬编码假设: 不要假设所有映射的物理页都来自同一个区域 (内核区或用户区)。
2. 自动化判断优于手动指定: 通过地址范围自动判断区域归属, 比要求调用者手动指定更不容易出错。
3. 接口设计的一致性: `pmem_alloc` 需要指定区域, `pmem_free` 也应该能正确对应, 避免分配和释放的区域不匹配。
4. 测试覆盖不同场景: 测试应该覆盖内核页和用户页的分配、映射、解除映射和释放, 确保所有路径都正确。

2.0.7. 源码理解总结

2.0.7.1. 物理内存管理

2.0.7.1.1.struct run 设计巧妙之处

- 简洁的单链表节点: 只包含一个指针, 用来把空闲的物理页面链接成单链表。
- 零额外内存开销: 空闲页的元数据直接存放在页面内 (通常页面最低地址), 节省了元数据空间开销。
- 对齐与直接利用: 页面大小通常大于或等于指针大小, 一个页面足够存储 `struct run`。把元数据与页面合体还能利用 CPU cache 更局部访问链表头。

2.0.7.1.2.kinit() 初始化过程分析

- 如何确定可分配的内存范围?
 - `kinit()` 通常通过链接器导出的符号 `end` (内核映像末尾)与编译时或配置文件里定义的 `PHYSTOP` (内核可管理物理内存上限) 来确定范围: 可分配范围是 `[roundup(end), PHYSTOP)`。`end` 指示内核代码和静态数据占用的物理内存末尾, 之后的空间就是可以用于分配的内存池。
- 空闲页链表是如何构建的?
 - `freerange(pa_start, pa_end)` 会把 `pa_start` 向上对齐到页边界, 然后从 `pa_start` 开始, 步长为 `PGSIZE`, 对每个页面调用 `kfree(p)`, 而 `kfree()` 会把该页面的首地址填充 (用于调试), 把页面 reinterpret 为 `struct run *r`, 并在持有 `kmem.lock` 的情况下把 `r->next = kmem.freelist; kmem.freelist = r;`。结果是把所有页面按顺序压入自由链表, 形成一个 LIFO 链表。
- 为什么要按页对齐?

- 分配器以页为单位管理内存(`kalloc` 返回整页), 因此所有页面地址必须和页大小对齐, 这样在做物理地址到页索引或页表映射时才能保证正确性。
- 将 `pa_start` 向上对齐可以避免覆盖内核已有的数据(`end` 可能不是页对齐), 确保不会将部分仍被内核使用的页加入 `freelist`。

2.0.7.1.3.kalloc() 和 kfree() 实现理解

- 分配算法的时间复杂度
 - `kalloc()`: 在持锁的情况下从链表头取出第一个节点, 操作是 $O(1)$ 。
 - `kfree()`: 把释放的页面插入链表头, 操作是 $O(1)$ 。
 - 因此分配和释放都具有常数时间复杂度 $O(1)$, 非常高效(在单线程或用自旋锁保护的多核场景中也能快速执行)。
- 如何防止 **double-free**?
 - 在 `xv6` 的简单实现中, `kfree()` 对传入地址做了基本检查(页对齐、地址范围、不能释放内核代码段之前的空间), 并会 `memset(pa, 1, PGSIZE)` 把页面填充为垃圾来帮助检测悬挂引用, 但它并没有用额外标志位来检测双重释放(**double-free**)。
 - 因此纯实现上并不能完全防止 **double-free**; 防护依赖于调用者的正确性。然而填充页面和在调试时对 `freed page` 做特殊模式可以在后续访问中较为快速地暴露问题。
 - 如果想主动检测 **double-free**, 需要额外元数据(比如一个位图或在页面头部写入 `magic` 值并在 `kfree` 前校验)。
- 这种设计的优缺点
 - 优点:
 - 极其简单, 代码短小易懂。
 - 时间复杂度低: 分配/释放 $O(1)$ 。
 - 零额外内存管理开销(元数据就位于空闲页面内部)。
 - 缺点:
 - 只支持固定大小的块(整页), 不能用于细粒度分配。
 - 不支持合并或块分裂, 因此碎片化控制能力有限(但在整页分配场景下碎片不像小粒度分配那样严重)。
 - 无内置安全检查或双重释放检测(需要额外机制来增强可靠性)。
 - 并发性能受限于全局锁(单个 `freelist lock` 会在高并发下成为瓶颈)。

2.0.7.1.4.设计思考

- 如何实现内存统计功能？
 - 维护全局计数器：在 `kmem` 中添加 `uint64 total_pages; uint64 free_pages;`，在 `kinit()` 初始化 `total_pages = (pa_end - pa_start) / PGSIZE`，在 `kalloc()` 减少 `free_pages`，在 `kfree()` 增加 `free_pages`。这些操作需在持锁状态下修改以保证并发安全。
 - 采样/阈值报警：如果 `free_pages` 低于阈值就触发警告或回收策略。
 - 每个 CPU 统计：为降低锁竞争，可以使用 per-CPU 的分配缓存或统计（例如每个 CPU 有局部缓存的自由链，和本地统计），再周期性合并到全局统计中。
- 如何检测内存泄漏？
 - 在内核引导/测试时记录总的分配次数与释放次数（或 track 当前分配计数），长期运行若 `allocated_pages - freed_pages` 保持增长并且不下降，可能为内存泄漏。
 - 引入引用计数或更高级的内存审计：给每次分配附加上下文（调用栈、分配时间），在释放时清楚记录。可以把这些信息写入一个可选的哈希表（物理页 -> 元数据），仅在调试或开发构建中启用以避免性能开销。
 - 定期运行内存一致性检查：在某些检查点遍历所有已知对象（例如进程页表中映射的物理页），验证哪些页没有被引用并和 free list 对比，找出被遗忘的页。
- 更高效的分配算法有哪些？
 - 分级空闲链表 (Segregated Free Lists)：对于不同大小类别维护不同的空闲链表（这里若支持小块分配非常适合）。
 - 位图 (Bitmap) + buddy 系统：buddy allocator 支持分裂和合并，适合可变大小的内存管理，内部实现也较为简单且支持 $O(\log n)$ 操作。
 - slab/SLUB 分配器：为经常分配/释放相同大小对象优化缓存，本质上预先分配大量相同大小的块并维护高速缓存，适合内核对象分配场景（inode、task_struct 等）。
 - per-CPU caches + lock-free/free-list caches：减少并发场景下的锁竞争，适合多核系统的高并发分配。

2.0.7.2. 虚拟内存与页表管理

2.0.7.2.1. 分页机制基础

- **satp 寄存器字段**
 - **MODE 字段**: 用于开启分页并选择页表级数。
 - **ASID**: 可用于降低上下文切换的开销。
 - **PPN 字段**: 以 4 KiB 页为单位存放根页表的物理页号。
- **分页启用过程** **M** 模式软件在第一次进入 **S** 模式前会将 **satp** 清零以关闭分页, 然后 **S** 模式软件在创建页表后将正确设置 **satp** 寄存器。**satp** 寄存器启用分页时, 处理器将从根部遍历页表, 将 **S** 模式和 **U** 模式的虚拟地址翻译为物理地址。
- **虚拟地址结构**
 - 第 39-30 位为一级页索引 **VPN0**
 - 第 30-21 位为二级页索引 **VPN1**
 - 第 21-12 位为三级页索引 **VPN2**
- **PTE 字段**
 - **V 位**: 表示该 PTE 的其余字段是否有效 (**V=1** 时有效)。若 **V=0**, 则遍历到此 PTE 的虚拟地址翻译过程将触发页故障。
 - **R、W、X 位**: 分别表示该页是否可读、可写、可执行。若 3 位均为 0, 则该 PTE 指向下一级页表, 否则为叶子节点。
 - **U 位**: 表示该页是否为用户页。若 **U=0**, 则 **U** 模式不能访问该页, 但 **S** 模式能。若 **U=1**, 则 **U** 模式能访问该页, 但 **S** 模式不能。
- **地址转换过程** 从 **satp** 找到根页表的物理地址, 然后按照 **L2, L1, L0** 的变化找到最终的物理地址。每个页表项 64 位 = 8 字节, 每个页表大小都为 $512 \times 8 = 4\text{KiB}$, 512 个页表项。最后的页表项保存了对应的 44 位物理页号。

2.0.7.2.2.xv6 页表管理代码分析

- **walk() 函数遍历逻辑**
 - 如何从虚拟地址提取各级索引?
 - **Sv39** 的虚拟地址划分为 **VPN[2], VPN[1], VPN[0]** 三级索引 (每级 9 位), 以及页内偏移 12 位。
 - 在 **xv6** 中常用的宏 (或等价位运算) 为:
 - 第 **i** 级索引 $\text{idx} = (\text{va} \gg (12 + 9*i)) \& 0x1FF$; **i** 从 2..0
 - 或者使用 **PX(va, i)** 之类的宏提取。

- ▶ 遇到无效页表项 (`PTE_V == 0`) 时如何处理?
 - 若 `walk(pagetable, va, alloc)` 中的 `alloc` 参数为 `false`: 遇到无效 PTE 就返回 `NULL` (表示找不到对应的下级页表或最终 PTE)。
 - 若 `alloc` 为 `true`: `walk` 会尝试为缺失的中间页表分配一个新的物理页 (通常通过 `kalloc/pmem_alloc`), 清零该页并把对应的父 PTE 设置为该页的物理地址并置 `PTE_V` (有效)。随后继续下钻。
- ▶ 为什么需要 `alloc` 参数?
 - `alloc` 决定 `walk` 是做“查找”还是做“创建并查找”。
 - 在建立映射 (例如 `mappings`) 时, 需要创建缺失的中间页表, 因此会传 `alloc = true`。
 - 在只做地址查询或访问转换 (例如查找某个映射以验证权限) 时, 不需要创建页表, 传 `alloc = false` 可以避免不必要的内存分配。
- `mappings()` 映射建立细节
 - ▶ 如何处理地址对齐?
 - `mappings()` 接受一个起始虚拟地址 `va`、长度 `size` 和物理基址 `pa` (或按页映射的基础地址)。它会先把 `va` 向下取整为页边界 (`PGROUNDDOWN`), 把 `va+size` 向上取整为页边界 (`PGROUNDUP`), 然后以 `PGSIZE` 为步长循环处理每一页映射。
 - 对齐保证: 每个映射操作都是整页的 PTE 设置, 避免覆盖非页对齐内存。
 - ▶ 权限位如何设置?
 - 每个 PTE 的低位用于权限 (`PTE_R/W/X/U` 等)。`mappings` 在写入最终叶子 PTE 时, 会把物理页号 (PA) 与 `perm` (传入的权限组合) 按位或, 再加上 `PTE_V` (有效位), 例如: `*pte = PA | perm | PTE_V`。
 - `perm` 根据映射用途选择: 内核只读代码页可能没有 `W`, 用户页需要 `U`, 设备内存通常设置为 `R|W` 而不设置 `X`。
 - ▶ 映射失败时的清理工作
 - 在映射循环过程中, `mappings` 可能在中途遇到错误 (例如为中间页表分配页失败, 或发现已有冲突的 PTE)。良好的实现应该回滚已经成功建立的那些映射 (解除已写入的 PTE 并释放在映射过程中为页表分配的临时页面), 以保持系统一致性并避免内存泄漏。

- xv6 的实现通常在中途失败时调用 `panic`（简单处理），或显式在失败路径上反向遍历已映射页并清除 PTEs，同时释放中间分配的页面（如果实现支持回滚）。

2.0.7.2.3.地址转换相关宏解释

- **PGROUNDUP(sz) 与 PGROUNDDOWN(a)**

- PGROUNDUP 将字节大小向上取整到 PGSIZE 的倍数，常用于计算要分配或映射的整页大小。
- PGROUNDDOWN 将地址向下取整到页边界，用于得到页起始地址。
- 例：PGROUNDUP(0x1003) = 0x2000（假设 PGSIZE = 0x1000）。

- **PTE_PA(pte) 的位运算**

- 宏 `#define PTE_PA(pte) (((pte) >> 10) << 12)` 的意图是从 64 位 PTE 中提取物理页号并还原为物理地址：页表项低 10 位保留为标志位和一些额外位（不同实现细节），所以右移 10 去掉低位标志，再左移 12 把页号转换为字节地址（乘以 4096）。
- 等价理解：PA = (pte & ~0x3FF) & ~0xFFF（去掉低 10 位和低 12 位标志），目的是得到高位的物理页帧地址。

2.0.7.2.4.实现挑战与对策

- 如何避免页表遍历中的“无限递归”？

- xv6 的 `walk()` 实际是按循环下钻（非真正递归函数调用），因此不存在函数递归的无限递归问题。但可能出现的危险是：在 `walk(..., alloc=true)` 时，分配新的页表页需要调用物理页分配器（`kalloc/pmem_alloc`），而分配器本身可能会在某些实现中访问页表（若有更高层次的缓存或统计），从而产生依赖回环。
- 对策：保证分配器在分配页时不依赖于页表（例如使用早期初始化的静态内存或独立的 allocator），或者在调用分配器时明确禁止调用能再次触发页表操作的路径；在实现上优先使用简单的物理页分配器（`kalloc`）来为页表页分配空间。

- 映射过程中的内存分配失败应该如何恢复？

- 在 `mappages()` 中，当为中间页表分配内存失败或在映射中途发生错误时，应回滚：
 1. 清除已经建立的叶子 PTE（把那些已写入的 PTE 设为 0 或适当值）；
 2. 释放在映射过程中申请的任何中间页表页；

3. 返回失败错误码（或在早期教学实现中 `panic`）。

- ▶ 实现时要记录哪些页/页表是新分配的，以便失败时能正确释放，而不会误释放原来已有的页表页。
- 如何确保页表的一致性？
 - ▶ 使用合适的锁：当多个 CPU 或线程可能并发修改同一进程/内核页表时，需要在修改页表(`mappages/unmap/walk`)时持有页表锁或进程锁(避免竞争和中间状态被其他 CPU 看到)。
 - ▶ 原子更新：写入最终叶子 PTE 时，尽量一次性把 `PA|flags` 写入，确保读者不会看到部分更新的标志。
 - ▶ TLB 同步：在更改页表（尤其是修改映射或权限）后调用 `sfence.vma`（或平台的等价 TLB flush）以确保已失效的 TLB 条目在处理器上被清除，避免地址转换不一致。
 - ▶ 顺序保证：在修改 PTE 之前先取消旧的映射（或先设置新的映射再 flush），按照硬件规范使用内存屏障，确保可见性。

2.0.7.2.5.参考 xv6 的内核初始化： `kvminit()` 和 `kvminithart()`

- `kvminit()` 的内核页表创建（需要映射哪些内存区域？）
 - ▶ 常见需要映射的区域包括：
 1. 内核文本（代码）段：以只读、可执行映射（去除写权限）。
 2. 内核数据/堆/全局变量区：读写映射（RW）。
 3. 内核使用的物理内存（如内核栈、页表本身、内核动态分配区），通常按恒等映射或偏移映射到高地址空间（`KERNBASE + PA`）。
 4. 设备 MMIO 区域（UART、PLIC、CLINT、virtio 等），这些地址通常映射为 RW、非可执行、并且 `U`（用户）位为 0（仅内核访问）。
 - ▶ 映射方式（为什么采用恒等映射/高地址恒等）
 - `xv6` 常用把物理地址恒等映射到内核虚拟地址空间（例如把物理地址 `0x0` 映射到 `KERNBASE + 0x0`），便于内核使用虚拟地址直接访问物理内存（简化实现）。
 - 恒等映射（或高位偏移映射）可以避免内核在访问某些内存时需要额外的转换逻辑，使早期初始化更简单。
- 设备内存的权限设置
 - ▶ 设备 MMIO 通常设置为 R/W, 无 X, 且 `U=0`（只有内核可访问）。这可以通过设置相应的 PTE 权限位实现。

- `kvminithart()` 的页表激活
 - `satp` 寄存器格式和设置:
 - 在 **RISC-V Sv39** 下, `satp` 包含 **MODE**(最高位若干位), **ASID**, 和 **PPN** (根页表物理页号)。内核通过宏 `MAKE_SATP(pagetable)` 或等价位运算把根页表物理地址填入 **PPN** 字段并设置 **MODE** 为 Sv39 的值, 然后写入 `satp`。
 - `sfence.vma` 的作用:
 - 在更改 `satp` (页表切换) 或修改页表后, 必须执行 `sfence.vma` 指令来刷新处理器的地址转换缓存(TLB), 确保新的页表设置生效且旧的条目被清除。
 - 激活页表前后的注意事项:
 - 确保新页表已正确初始化并包含必要的内核映射 (否则启用分页后内核可能因缺失映射而异常)。
 - 在多核环境中为每个 hart 分别设置 `satp` 并执行 `sfence.vma`, 或在切换后在目标 hart 上执行局部刷新。
 - 激活新页表前通常会把中断/异常处理向量设置好 (`stvec / mtvec` 等), 并确保 `trampoline` 或切换代码已准备好处理异常。

2.1. 测试验证部分

- 测试代码

```

1 //物理内存
2 #include "riscv.h"
3 #include "lib/print.h"
4 #include "mem/pmem.h"
5 #include "lib/str.h"
6
7 volatile static int started = 0;
8
9 volatile static int over_1 = 0, over_2 = 0;
10
11 static int* mem[1024];
12
13 int main()
14 {
15     int cpuid = r_tp();
16
17     if(cpuid == 0) {
18
19         print_init();
20         pmem_init();

```

C++

```

21
22     printf("cpu %d is booting!\n", cpuid);
23     __sync_synchronize();
24     started = 1;
25
26     for(int i = 0; i < 512; i++) {
27         mem[i] = pmem_alloc(true);
28         memset(mem[i], 1, PGSIZE);
29         printf("mem = %p, data = %d\n", mem[i], mem[i]
[0]);
30     }
31     printf("cpu %d alloc over\n", cpuid);
32     over_1 = 1;
33
34     while(over_1 == 0 || over_2 == 0);
35
36     for(int i = 0; i < 512; i++)
37         pmem_free((uint64)mem[i], true);
38     printf("cpu %d free over\n", cpuid);
39
40     } else {
41
42         while(started == 0);
43         __sync_synchronize();
44         printf("cpu %d is booting!\n", cpuid);
45
46         for(int i = 512; i < 1024; i++) {
47             mem[i] = pmem_alloc(true);
48             memset(mem[i], 1, PGSIZE);
49             printf("mem = %p, data = %d\n", mem[i], mem[i]
[0]);
50         }
51         printf("cpu %d alloc over\n", cpuid);
52         over_2 = 1;
53
54         while(over_1 == 0 || over_2 == 0);
55
56         for(int i = 512; i < 1024; i++)
57             pmem_free((uint64)mem[i], true);
58         printf("cpu %d free over\n", cpuid);
59     }
60     }
61     while (1);
62 }
63

```

C++

```

1 int main()
2 {
3     int cpuid = r_tp();
4
5     if(cpuid == 0) {

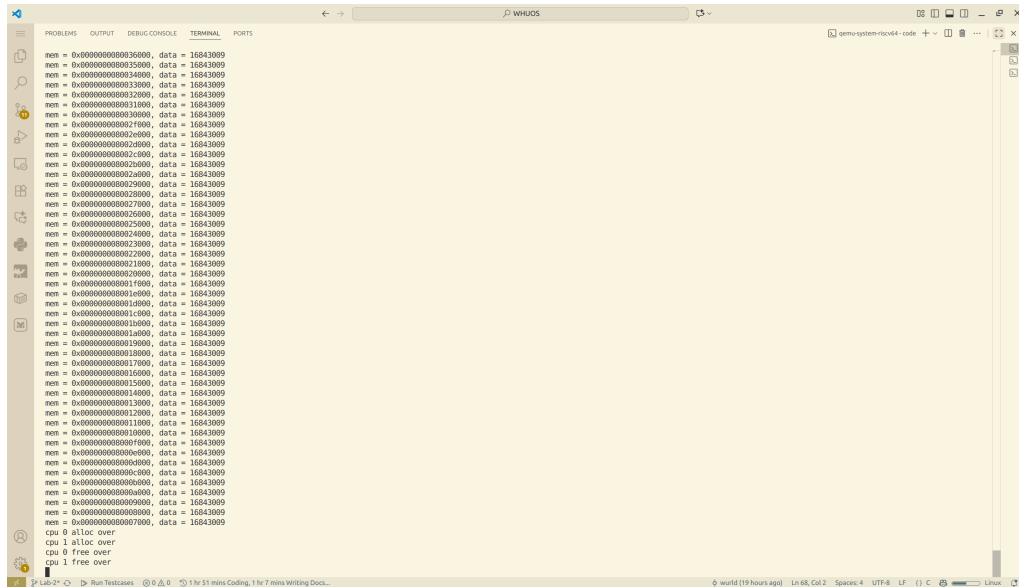
```

```

6
7     print_init();
8     pmem_init();
9     kvm_init();
10    kvm_inithart();
11
12    printf("cpu %d is booting!\n", cpuid);
13    __sync_synchronize();
14    // started = 1;
15
16    pgtbl_t test_pgtbl = pmem_alloc(true);
17    uint64 mem[5];
18    for(int i = 0; i < 5; i++)
19        mem[i] = (uint64)pmem_alloc(false);
20
21    printf("\ntest-1\n\n");
22    vm_mappages(test_pgtbl, 0, mem[0], PGSIZE, PTE_R);
23    vm_mappages(test_pgtbl, PGSIZE * 10, mem[1], PGSIZE /
24    2, PTE_R | PTE_W);
25    vm_mappages(test_pgtbl, PGSIZE * 512, mem[2], PGSIZE
26    - 1, PTE_R | PTE_X);
27    vm_mappages(test_pgtbl, PGSIZE * 512 * 512, mem[2],
28    PGSIZE, PTE_R | PTE_X);
29    vm_mappages(test_pgtbl, VA_MAX - PGSIZE, mem[4],
30    PGSIZE, PTE_W);
31    vm_print(test_pgtbl);
32
33    printf("\ntest-2\n\n");
34    vm_mappages(test_pgtbl, 0, mem[0], PGSIZE, PTE_W);
35    vm_unmappages(test_pgtbl, PGSIZE * 10, PGSIZE, true);
36    vm_unmappages(test_pgtbl, PGSIZE * 512, PGSIZE,
37    true);
38    vm_print(test_pgtbl);
39
40    } else {
41
42        while(started == 0);
43        __sync_synchronize();
44        printf("cpu %d is booting!\n", cpuid);
45    }
46    while (1);
47 }
48
49

```

- 测试结果



```

cpu 0 is booting!
mem[0] = 0x0000000087fff000
mem[1] = 0x0000000087ffe000
mem[2] = 0x0000000087ffd000
mem[3] = 0x0000000087ffc000
mem[4] = 0x0000000087ffb000

test-1

pte[0]=%016lx
pte[1]=%016lx
pte[255]=%016lx

test-2

pte[0]=%016lx
pte[1]=%016lx
pte[255]=%016lx

```

3 实验三：中断处理实现

3.1. 一、实验目的

本实验旨在实现 RISC-V 操作系统内核的中断处理机制，主要包括：

1. 理解 RISC-V 中断处理的基本原理和流程
2. 实现 M-mode 时钟中断的初始化和处理
3. 实现 S-mode 中断分发和处理机制
4. 实现外设中断（UART）的处理

5. 掌握中断上下文的保存和恢复
6. 理解多核环境下的中断处理

3.2. 二、实验原理

3.2.1.2.1 RISC-V 特权级架构

RISC-V 定义了三个特权级：

- **M-mode (Machine Mode):** 最高特权级，可访问所有硬件资源
- **S-mode (Supervisor Mode):** 操作系统内核运行级别
- **U-mode (User Mode):** 用户程序运行级别

3.2.2.2.2 中断处理机制

3.2.2.1.2.2.1 中断类型

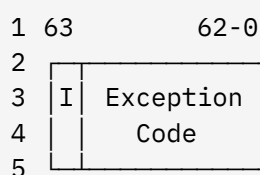
RISC-V 将 trap 分为两类：

- **中断 (Interrupt):** 异步事件，由外部硬件触发
 - 时钟中断 (Timer Interrupt)
 - 外设中断 (External Interrupt)
 - 软件中断 (Software Interrupt)
- **异常 (Exception):** 同步事件，由指令执行引起
 - 非法指令、页错误、系统调用等

3.2.2.2.2.2.2 关键 CSR 寄存器

寄存器	功能	读写权限
stvec	S-mode trap 向量基地址	读写
sepc	异常发生时的 PC	读写
scause	trap 原因 (中断/异常类型)	只读
stval	trap 附加信息 (如出错地址)	只读
sstatus	S-mode 状态寄存器	读写
sie	S-mode 中断使能	读写
sip	S-mode 中断挂起	读写

3.2.2.3.2.2.3 scause 寄存器格式



```

6
7 I=1: 中断
8 I=0: 异常
9
10 常用中断代码:
11   1: S-mode 软件中断 (M-mode 时钟中断转发)
12   5: S-mode 时钟中断
13   9: S-mode 外设中断
14
15 常用异常代码:
16   0: 指令地址不对齐
17   2: 非法指令
18   8: U-mode 系统调用 (ecall)
19  12: 指令页错误
20  13: 加载页错误
21  15: 存储页错误
22

```

3.2.3.2.3 中断处理硬件

3.2.3.1.2.3.1 CLINT (Core Local Interruptor)

- 地址: 0x2000000
- 功能: 提供定时器中断和核间软件中断
- 关键寄存器:
 - CLINT_MTIME: 当前时间 (64 位计数器)
 - CLINT_MTIMECMP(hartid): 时钟比较值

3.2.3.2.2.3.2 PLIC (Platform-Level Interrupt Controller)

- 地址: 0x0c000000
- 功能: 管理外部设备中断
- 特性:
 - 支持中断优先级
 - 支持多核中断分发
 - Claim/Complete 机制防止中断丢失

3.2.3.3.2.3.3 UART (串口)

- 地址: 0x10000000
- IRQ: 10
- 功能: 串口输入输出, 支持中断模式

3.3. 三、实验内容

3.3.1.3.1 实现的文件和函数

3.3.1.1.3.1.1 kernel/dev/timer.c - 时钟管理

3.3.1.1.1. timer_init() - M-mode 时钟初始化

功能: 在 M-mode 下初始化时钟中断

实现代码:

```
1 void timer_init()
2 {
3     // 获取当前CPU的hartid
4     int hartid = r_mhartid();
5
6     // 设置第一次时钟中断的时间
7     *(uint64*)CLINT_MTIMECMP(hartid) = *(uint64*)CLINT_MTIME +
    INTERVAL;
8
9     // 准备timer_vector需要的信息
10    uint64 *scratch = &mscratch[hartid][0];
11    scratch[3] = CLINT_MTIMECMP(hartid);
12    scratch[4] = INTERVAL;
13    w_mscratch((uint64)scratch);
14
15    // 设置M-mode trap处理函数为timer_vector
16    w_mtvec((uint64)timer_vector);
17
18    // 使能M-mode中断和时钟中断
19    w_mstatus(r_mstatus() | MSTATUS_MIE);
20    w_mie(r_mie() | MIE_MTIE);
21 }
22
```

实现要点:

- 设置 CLINT_MTIMECMP 为当前时间 + 间隔
- 配置 mscratch 数组供 timer_vector 使用
- 设置 mtvec 指向 timer_vector
- 使能 M-mode 时钟中断

3.3.1.1.2. timer_create() - 创建系统时钟

功能: 初始化 S-mode 系统时钟

实现代码:

```
1 void timer_create()
```

```

2 {
3     sys_timer.ticks = 0;
4     initlock(&sys_timer.lk, "timer");
5 }
6

```

3.3.1.1.3. `timer_update()` - 更新时钟

功能: 线程安全地增加系统滴答计数

实现代码:

```

1 void timer_update()
2 {
3     acquire(&sys_timer.lk);
4     sys_timer.ticks++;
5     release(&sys_timer.lk);
6 }
7

```

C

3.3.1.1.4. `timer_get_ticks()` - 获取时钟滴答数

功能: 线程安全地读取当前 ticks

实现代码:

```

1 uint64 timer_get_ticks()
2 {
3     uint64 ticks;
4     acquire(&sys_timer.lk);
5     ticks = sys_timer.ticks;
6     release(&sys_timer.lk);
7     return ticks;
8 }
9

```

C

3.3.1.2.3.1.2 `kernel/trap/trap_kernel.c` - 内核态中断处理

3.3.1.2.1. `trap_kernel_init()` - 初始化全局 `trap` 资源

功能: 初始化内核 `trap` 系统的全局资源

实现代码:

```

1 void trap_kernel_init()
2 {
3     timer_create();
4 }
5

```

C

3.3.1.2.2. `trap_kernel_inithart()` - 每个 CPU 核心的 `trap` 初始化

功能: 为每个 CPU 核心设置 `trap` 处理

实现代码:

```
1 void trap_kernel_inithart()
2 {
3     w_stvec((uint64)kernel_vector);
4 }
5
```

C

3.3.1.2.3. `trap_kernel_handler()` - 核心中断/异常处理逻辑

功能: 分发和处理所有内核态的中断和异常

实现代码:

```
1 void trap_kernel_handler()
2 {
3     uint64 sepc = r_sepc();
4     uint64 sstatus = r_sstatus();
5     uint64 scause = r_scause();
6     uint64 stval = r_stval();
7
8     // 安全性检查
9     assert(sstatus & SSTATUS_SPP, "trap_kernel_handler: not from s-
mode");
10    assert(intr_get() == 0, "trap_kernel_handler: interrupt
enabled");
11
12    int trap_id = scause & 0xf;
13
14    if (scause & (1ULL << 63)) {
15        // 中断处理
16        switch (trap_id) {
17            case 1: // S-mode 软件中断
18                timer_interrupt_handler();
19                break;
20            case 5: // S-mode 时钟中断
21                printk("S-mode timer interrupt\n");
22                break;
23            case 9: // S-mode 外设中断
24                external_interrupt_handler();
25                break;
26            default:
27                printk("Unknown interrupt: %s\n",
interrupt_info[trap_id]);
28                break;
29        }
30    } else {
```

C

```

31         // 异常处理
32         printk("Exception in kernel at sepc=0x%lx: %s\n",
33               sepc, exception_info[trap_id]);
34         printk("    stval = 0x%lx\n", stval);
35         panic("Unhandled exception in kernel mode");
36     }
37 }
38

```

实现要点:

- 读取所有关键 CSR 寄存器
- 进行安全性检查
- 根据 `scause` 最高位判断中断/异常
- 分发到对应的处理函数

3.3.1.2.4. `timer_interrupt_handler()` - 时钟中断处理

功能: 处理由 M-mode 转发的时钟中断

实现代码:

```

1 void timer_interrupt_handler()
2 {
3     // 清除S-mode软件中断挂起位
4     w_sip(r_sip() & ~2);
5
6     // 更新系统时钟
7     timer_update();
8
9     // 打印时钟中断信息
10    printk("Timer interrupt: ticks = %d\n", timer_get_ticks());
11 }
12

```

实现要点:

- 清除 SIP.SSIP 位
- 调用 `timer_update()` 增加 ticks
- 打印调试信息

3.3.1.2.5. `external_interrupt_handler()` - 外设中断处理

功能: 处理 PLIC 管理的外部设备中断

实现代码:

```

1 void external_interrupt_handler()
2 {

```

```

3     int irq = plic_claim();
4
5     if (irq == UART_IRQ) {
6         uart_intr();
7     } else if (irq) {
8         printk("Unexpected external interrupt irq=%d\n", irq);
9     }
10
11    if (irq) {
12        plic_complete(irq);
13    }
14 }
15

```

实现要点:

- 调用 plic_claim() 获取中断号
- 根据 IRQ 分发到具体设备驱动
- 调用 plic_complete() 通知 PLIC 完成

3.3.1.3.3.1.3 kernel/trap/trap.S - 汇编中断入口

3.3.1.3.1.kernel_vector - S-mode 中断向量

功能: 保存/恢复上下文, 调用 C 语言处理函数

实现代码 (部分):

```

1 kernel_vector:
2     # 分配栈空间
3     addi sp, sp, -256
4
5     # 保存所有通用寄存器
6     sd ra, 0(sp)
7     sd sp, 8(sp)
8     sd gp, 16(sp)
9     # ... 保存 x4-x31
10
11    # 调用 C 处理函数
12    call trap_kernel_handler
13
14    # 恢复所有寄存器
15    ld ra, 0(sp)
16    # ... 恢复其他寄存器
17
18    # 恢复栈指针
19    addi sp, sp, 256
20
21    # 返回

```

assembly

```
22     sret
23
```

3.3.1.3.2.timer_vector - M-mode 时钟中断向量

功能: 处理 M-mode 时钟中断, 转发到 S-mode

实现代码 (部分):

assembly

```
1 timer_vector:
2     # 暂存寄存器到 mscratch
3     csrrw a0, mscratch, a0
4     sd a1, 0(a0)
5     sd a2, 8(a0)
6     sd a3, 16(a0)
7
8     # 更新 CLINT_MTIMECMP += INTERVAL
9     ld a1, 24(a0)      # CLINT_MTIMECMP 地址
10    ld a2, 32(a0)      # INTERVAL
11    ld a3, 0(a1)
12    add a3, a3, a2
13    sd a3, 0(a1)
14
15    # 触发 S-mode 软件中断
16    li a1, 2
17    csrw sip, a1
18
19    # 恢复寄存器
20    ld a3, 16(a0)
21    ld a2, 8(a0)
22    ld a1, 0(a0)
23    csrrw a0, mscratch, a0
24
25    mret
26
```

3.3.2.3.2 函数调用关系

3.3.2.1.3.2.1 启动阶段函数调用

kernel/boot/start.c (M-mode):

C

```
1 void start()
2 {
3     // ... 配置特权级和内存保护
4
5     // 初始化时钟中断
6     timer_init();
7
8     // 切换到 S-mode
```

```

9     mret;
10 }
11

```

kernel/boot/main.c (S-mode):

C

```

1 int main()
2 {
3     int cpuid = r_tp();
4
5     if(cpuid == 0) {
6         print_init();
7
8         // 初始化设备和中断系统
9         uart_init();
10        plic_init();
11        trap_kernel_init();
12        trap_kernel_inithart();
13        plic_inithart();
14
15        // 使能中断
16        intr_on();
17
18        started = 1;
19    } else {
20        while(started == 0);
21
22        // 其他CPU核心初始化
23        trap_kernel_inithart();
24        plic_inithart();
25        intr_on();
26    }
27
28    while (1);
29 }
30

```

3.4. 四、中断处理流程

3.4.1.4.1 时钟中断完整流程

- 1 1. CLINT 时钟到期 (CLINT_MTIME >= CLINT_MTIMECMP)
- 2 ↓
- 3 2. 触发 M-mode 时钟中断
- 4 ↓
- 5 3. CPU 跳转到 mtvec (timer_vector)
- 6 ↓
- 7 4. [trap.S] timer_vector:
- 8 ├ 保存寄存器 a0-a3 到 mscratch
- 9 └ 更新 CLINT_MTIMECMP += INTERVAL

```

10    └─ 设置 SIP.SSIP (触发 S-mode 软件中断)
11    └─ mret 返回
12    ↓
13 5. 触发 S-mode 软件中断
14    ↓
15 6. CPU 跳转到 stvec (kernel_vector)
16    ↓
17 7. [trap.S] kernel_vector:
18    └─ 保存所有寄存器到栈
19    └─ call trap_kernel_handler()
20    ↓
21 8. [trap_kernel.c] trap_kernel_handler():
22    └─ 读取 scause = 0x8000000000000001
23    └─ 调用 timer_interrupt_handler()
24        └─ 清除 SIP.SSIP
25        └─ timer_update() (ticks++)
26        └─ printk("Timer interrupt: ticks = %d")
27    ↓
28 9. 返回到 kernel_vector
29    └─ 恢复所有寄存器
30    └─ sret 返回到被中断的代码
31

```

3.4.2.4.2 UART 中断完整流程

```

1 1. UART 接收到数据
2    ↓
3 2. UART 硬件产生中断请求
4    ↓
5 3. PLIC 接收中断请求并设置挂起位
6    ↓
7 4. 触发 S-mode 外设中断
8    ↓
9 5. CPU 跳转到 stvec (kernel_vector)
10   ↓
11 6. [trap.S] kernel_vector:
12   └─ 保存所有寄存器到栈
13   └─ call trap_kernel_handler()
14   ↓
15 7. [trap_kernel.c] trap_kernel_handler():
16   └─ 读取 scause = 0x8000000000000009
17   └─ 调用 external_interrupt_handler()
18       └─ plic_claim() → 获取 IRQ = 10 (UART)
19       └─ uart_intr()
20           └─ 读取 UART 数据
21           └─ uart_putc_sync() 回显
22       └─ plic_complete(10)
23   ↓
24 8. 返回到 kernel_vector

```



```

25     └─ 恢复所有寄存器
26     └─ sret 返回到被中断的代码
27

```

3.5. 五、关键数据结构

3.5.1.5.1 timer_t 结构

```

1 typedef struct timer {
2     uint64 ticks;      // 时钟滴答计数
3     spinlock_t lk;     // 保护 ticks 的自旋锁
4 } timer_t;
5

```

设计说明:

- 使用自旋锁保证多核环境下的线程安全
- ticks 记录系统启动以来的时钟中断次数

3.5.2.5.2 mscratch 数组

```

1 static uint64 mscratch[NCPU][5];
2

```

用途:

- [hartid][0..2]: timer_vector 临时保存 a1, a2, a3
- [hartid][3]: CLINT_MTIMECMP(hartid) 地址
- [hartid][4]: INTERVAL 值

3.6. 六、实验测试

3.6.1.6.1 时钟中断测试

测试代码:

```

1 #include "riscv.h"
2 #include "lib/print.h"
3 #include "dev/uart.h"
4 #include "dev/plic.h"
5 #include "trap/trap.h"
6
7 volatile static int started = 0;
8 int main()
9 {
10     int cpuid = r_tp();
11

```

```

12     if(cpuid == 0) {
13         // CPU 0: 主核心初始化
14         print_init();
15
16         printf("\n=== WHU OS Lab 3: Timer Interrupt Test ===\n");
17         printf("Testing timer interrupts only...\n\n");
18
19         // 初始化trap系统（用于处理时钟中断）
20         trap_kernel_init();          // 初始化内核trap系统（包括
timer_create)
21         trap_kernel_inithart();    // 初始化当前核心的trap（设置stvec）
22
23         printf("Trap system initialized\n");
24
25         // 使能中断
26         intr_on();
27         printf("Interrupts enabled\n\n");
28
29         printf("CPU %d is booting!\n", cpuid);
30         printf("Waiting for timer interrupts...\n");
31         printf("Timer interrupt occurs approximately every 0.1
seconds (INTERVAL=1000000)\n");
32         printf("- Each 'T' represents one timer tick\n");
33         printf("- Ticks count is displayed every 10 interrupts\n");
34         printf("- You can modify INTERVAL in include/dev/timer.h to
test different speeds\n\n");
35
36         __sync_synchronize();
37         started = 1;    // 允许其他CPU继续启动
38
39     } else {
40         // 其他CPU核心初始化
41         while(started == 0);
42         __sync_synchronize();
43
44         // 其他CPU核心也需要初始化trap
45         trap_kernel_inithart();    // 初始化当前核心的trap
46
47         // 使能中断
48         intr_on();
49
50         printf("CPU %d is booting!\n", cpuid);
51     }
52
53     // 主循环：等待中断
54     while (1) {
55         // 可以在这里添加其他测试代码
56         // 中断会自动被处理
57     }

```

```
58 }  
59  
60
```

测试结果:

多核输出

```
hwt@WP:~/桌面/sources/lab/OSLab/WHUOS/whu-oslab-lab3/code$ make clean && make qemu  
=== WHU OS Lab 3: Timer Interrupt Test ===  
Testing timer interrupts only...  
  
Trap system initialized  
Interrupts enabled  
  
CPU 0 is booting!  
Waiting for timer interrupts...  
Timer interrupt occurs approximately every 0.1 seconds (INTERVAL=1000000)  
- Each 'T' represents one timer tick  
- Ticks count is displayed every 10 interrupts  
- You can modify INTERVAL in include/dev/timer.h to test different speeds  
  
CPU 1 is booting!  
TTTTTTTTTT  
ticks = 10  
  
ticks = 10  
TTTTTTTTTT  
ticks = 20  
  
ticks = 20  
TTTTTTTTTT  
ticks = 30  
  
ticks = 30  
TTTTTTTTTT  
ticks = 40  
TTTTTTTTTT  
ticks = 50  
  
ticks = 50  
  
=== Timer interrupt test completed (ticks = 50) ===  
  
=== Timer interrupt test completed (ticks = 50) ===  
Timer is still running but output is disabled.  
  
Timer is still running but output is disabled.
```

只让 CPU0 输出

```

hwt@WP:~/桌面/sources/lab/OSLab/WHUOS$ cd /home/hwt/桌面/sources/lab/OSLab/WHUOS/whu-oslab-gemu
=== WHU OS Lab 3: Timer Interrupt Test ===
Testing timer interrupts only...

Trap system initialized
Interrupts enabled

CPU 0 is booting!
Waiting for timer interrupts...
Timer interrupt occurs approximately every 0.1 seconds (INTERVAL=1000000)
- Each 'T' represents one timer tick
- Ticks count is displayed every 10 interrupts
- You can modify INTERVAL in include/dev/timer.h to test different speeds

CPU 1 is booting!
TTTTT
ticks = 10
TTTTT
ticks = 20
TTTTTTTTTT
ticks = 40
TTTTT
ticks = 50

```

3. 6. 2. 6. 2 UART 中断测试

测试代码:

C++

```

1 #include "riscv.h"
2 #include "lib/print.h"
3 #include "dev/uart.h"
4 #include "dev/plic.h"
5 #include "trap/trap.h"
6
7 volatile static int started = 0;
8 int main()
9 {
10     int cpuid = r_tp();
11
12     if(cpuid == 0) {
13         // CPU 0: 主核心初始化
14         print_init();
15
16         printf("\n=== WHU OS Lab 3: External Interrupt Test
17         ===\n");
18         printf("Testing UART external interrupts...\n\n");
19
20         // 初始化中断系统（但还不使能UART中断）
21         plic_init();           // 初始化PLIC中断控制器
22         trap_kernel_init();    // 初始化内核trap系统
23         trap_kernel_inithart(); // 初始化当前核心的trap
24         plic_inithart();       // 初始化当前核心的PLIC
25
26         printf("PLIC initialized\n");
27         printf("Trap system initialized\n");
28     }
29 }

```

```

28         // 使能系统中断
29         intr_on();
30         printf("System interrupts enabled\n");
31
32         // 在系统中断使能后再初始化UART（避免中断堆积）
33         uart_init();           // 初始化UART串口并使能UART中断
34         printf("UART initialized\n\n");
35
36         printf("CPU %d is ready!\n", cpuid);
37         printf("=== UART External Interrupt Test ===\n");
38         printf("Please type characters to test UART interrupt.\n");
39         printf("Each character you type will trigger an external
interrupt.\n");
40         printf("Press Ctrl+A then X to exit QEMU.\n\n");
41
42         __sync_synchronize();
43         started = 1; // 允许其他CPU继续启动
44
45     } else {
46         // 其他CPU核心初始化
47         while(started == 0);
48         __sync_synchronize();
49
50         // 其他CPU核心也需要初始化trap和plic
51         trap_kernel_inithart(); // 初始化当前核心的trap
52         plic_inithart();        // 初始化当前核心的PLIC
53
54         // 使能中断
55         intr_on();
56
57         printf("CPU %d is ready!\n", cpuid);
58     }
59
60     // 主循环：等待中断
61     while (1) {
62         // 可以在这里添加其他测试代码
63         // 中断会自动被处理
64     }
65 }
66
67

```

测试结果:

```
=== WHU OS Lab 3: External Interrupt Test ===
Testing UART external interrupts...

PLIC initialized
Trap system initialized
System interrupts enabled
UART initialized

CPU 0 is ready!
=== UART External Interrupt Test ===
Please type characters to test UART interrupt.
Each character you type will trigger an external interrupt.
Press Ctrl+A then X to exit QEMU.

CPU 1 is ready!
qbjy
```

4 实验四：特权级转换与首个用户态进程创建

4.1. 一、实验目的

本实验旨在实现 RISC-V 操作系统从内核态到用户态的特权级切换机制，并创建首个用户态进程，主要包括：

1. 理解 RISC-V 特权级转换的原理和流程
2. 实现用户态进程的数据结构和内存布局
3. 实现用户态页表的创建和映射
4. 实现 Trampoline 机制用于特权级切换
5. 实现用户态 trap 处理流程
6. 创建并启动首个用户态进程 proczero
7. 理解上下文切换机制

4.2. 二、实验原理

4.2.1.2.1 RISC-V 特权级切换

4.2.1.1.2.1.1 U-mode 到 S-mode 的切换

当用户态程序执行 trap(系统调用、异常或中断)时，硬件自动完成以下操作：

1. 如果是设备中断且 `sstatus.SIE = 0`，不进行切换
2. 通过置零 `SIE` 禁用中断
3. 将当前 `pc` 拷贝到 `sepc`
4. 保存当前特权级到 `sstatus.SPP`
5. 设置 `scause` 为 trap 原因
6. 设置当前特权级为 Supervisor

7. 将 `stvec` 拷贝到 `pc`，跳转到 `trap` 处理程序

注意: CPU 不会自动切换页表或栈，这些需要软件完成。

4.2.1.2.2.1.2 S-mode 到 U-mode 的切换

从内核态返回用户态时，需要手动设置：

1. 清除 `sstatus.SPP`，将其置为 0（表示返回 U-mode）
2. 设置 `sstatus.SPIE = 1`，启用用户态中断
3. 设置 `sepc` 为用户进程的 PC 值
4. 切换到用户进程的页表（写 `satp`）
5. 恢复用户态寄存器上下文
6. 执行 `sret` 指令

硬件在执行 `sret` 时自动完成：

- 从 `sepc` 恢复 `pc`
- 从 `sstatus` 恢复用户模式状态
- 将特权模式设置为用户模式

4.2.2.2.2 进程数据结构

4.2.2.1.2.2.1 进程控制块 (proc_t)

```
1 typedef struct proc {  
2     int pid;                // 进程标识符  
3     pgtbl_t pgtbl;          // 用户态页表  
4     uint64 heap_top;        // 用户堆顶（字节为单位）  
5     uint64 ystack_pages;    // 用户栈占用的页面数量  
6     trapframe_t* tf;        // Trapframe（用户态/内核态切换时的寄存器保存  
    区）  
7     uint64 kstack;          // 内核栈的虚拟地址  
8     context_t ctx;          // 内核态进程上下文  
9 } proc_t;  
10
```

4.2.2.2.2.2.2 Trapframe 结构

Trapframe 用于在用户态和内核态切换时保存寄存器：

```
1 typedef struct trapframe {  
2     // 内核信息  
3     uint64 kernel_satp;     // 内核页表  
4     uint64 kernel_sp;       // 内核栈指针  
5     uint64 kernel_trap;     // trap 处理函数地址  
6     uint64 kernel_hartid;    // CPU ID
```

```

7
8 // 用户态切换信息
9 uint64 epc; // 用户程序计数器
10
11 // 通用寄存器（31个，x0 固定为0）
12 uint64 ra;
13 uint64 sp;
14 uint64 gp;
15 uint64 tp;
16 uint64 t0, t1, t2;
17 uint64 s0, s1;
18 uint64 a0, a1, a2, a3, a4, a5, a6, a7;
19 uint64 s2, s3, s4, s5, s6, s7, s8, s9, s10, s11;
20 uint64 t3, t4, t5, t6;
21 } trapframe_t;
22

```

4. 2. 2. 3. 2. 2. 3 Context 结构

Context 用于进程间的上下文切换：

```

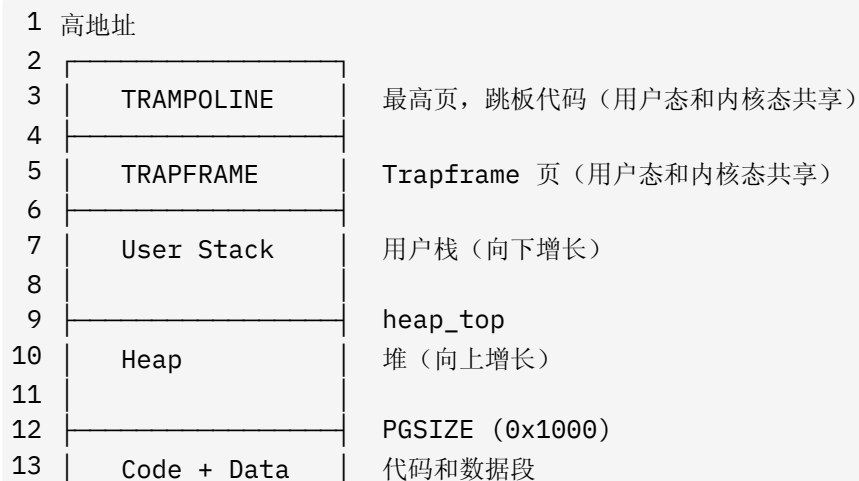
1 typedef struct context {
2     uint64 ra; // 返回地址
3     uint64 sp; // 栈指针
4
5     // 被调用者保存寄存器
6     uint64 s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11;
7 } context_t;
8

```

Context 和 Trapframe 的区别：

- **Context:** 用于同一特权级内的进程切换（内核态进程之间）
- **Trapframe:** 用于不同特权级之间的切换（用户态 ↔ 内核态）

4. 2. 3. 2. 3 用户地址空间布局



14		0x0
15	Empty	最低 4KB 不映射（捕获空指针）
16		
17	低地址	
18		

4.2.4.2.4 Trampoline 机制

Trampoline（跳板）页是一个同时映射在用户页表和内核页表中的特殊页面：

- 位置: 虚拟地址空间的最高页
- 权限: 只读可执行（不设置 `PTE_U`）
- 作用:
 1. 提供特权级切换时的过渡代码
 2. 允许在切换页表前后使用相同的虚拟地址
 3. 保存/恢复用户态寄存器

4.3. 三、实验内容

4.3.1.3.1 内存布局配置

4.3.1.1.3.1.1 include/memlayout.h - 地址空间定义

新增定义:

```

1 // 用户地址空间最大值 (Sv39 限制)
2 #define MAXVA (1L << (9 + 9 + 9 + 12 - 1))
3
4 // Trampoline 页映射到最高地址
5 #define TRAMPOLINE (MAXVA - PGSIZE)
6
7 // Trapframe 页紧邻 Trampoline 下方
8 #define TRAPFRAME (TRAMPOLINE - PGSIZE)
9
10 // 内核栈虚拟地址计算宏
11 // 每个进程的内核栈占 2 页 (1 页栈 + 1 页 guard page)
12 #define KSTACK(p) (TRAPFRAME - ((p)+1)* 2*PGSIZE)
13
```

设计要点:

- `MAXVA` 基于 Sv39 的 39 位虚拟地址计算
- `TRAMPOLINE` 和 `TRAPFRAME` 固定在高地址，便于用户和内核共享
- `KSTACK` 为每个进程分配独立的内核栈空间

4.3.1.2.3.1.2 kernel/kernel.ld - 链接脚本修改

修改内容:

ld

```
1 .text : {
2     *(.text .text.*)
3     . = ALIGN(0x1000);
4     _trampoline = .;
5     *(trampsec)
6     . = ALIGN(0x1000);
7     ASSERT(. - _trampoline == 0x1000, "error: trampoline larger
    than one page");
8     PROVIDE(etext = .);
9 }
10
```

设计要点:

- 将 trampoline section 单独对齐到页边界
- 确保 trampoline 代码恰好占用一页（4096 字节）
- 导出 `_trampoline` 符号供内核使用

4.3.2.3.2 内核虚拟内存映射扩展

4.3.2.1.3.2.1 kernel/mem/vmem.c - kvm_init() 修改

新增映射:

c

```
1 void kvm_init()
2 {
3     // ... 原有的设备和内核段映射 ...
4
5     // 映射 trampoline 页（用于用户态和内核态切换）
6     extern char trampoline[]; // defined in trampoline.S
7     vm_mappages(kernel_pgtbl, TRAMPOLINE, (uint64)trampoline,
    PGSIZE, PTE_R | PTE_X);
8
9     // 为进程 0 分配并映射内核栈
10    char *pa = pmem_alloc(true);
11    if(pa == 0)
12        panic("kvm_init: kstack alloc failed");
13    uint64 va = KSTACK(0);
14    vm_mappages(kernel_pgtbl, va, (uint64)pa, PGSIZE, PTE_R |
    PTE_W);
15 }
16
```

实现要点:

- Trampoline 映射为只读可执行，不设置 `PTE_U`（用户不可直接访问）

- 为第一个进程预分配内核栈并映射到固定虚拟地址
- 使用 `pmem_alloc(true)` 分配内核页面

4.3.3.3.3 进程管理实现

4.3.3.1.3.3.1 kernel/proc/cpu.c - myproc()

功能: 获取当前 CPU 上运行的进程

实现代码:

```
1 proc_t* myproc()
2 {
3     push_off(); // 关中断, 防止调度
4     cpu_t* c = mycpu();
5     proc_t* p = c->proc;
6     pop_off(); // 恢复中断状态
7     return p;
8 }
9
```

实现要点:

- 使用 `push_off/pop_off` 保护临界区
- 通过 `mycpu()` 获取当前 CPU 结构
- 返回 CPU 上正在运行的进程指针

4.3.3.2.3.3.2 kernel/proc/proc.c - proc_pgtbl_init()

功能: 创建并初始化用户进程页表

实现代码:

```
1 pgtbl_t proc_pgtbl_init(uint64 trapframe)
2 {
3     pgtbl_t pgtbl;
4     uint64 page;
5
6     // 分配一个空的页表 (内核空间)
7     page = (uint64)pmem_alloc(true);
8     if(page == 0) {
9         return 0;
10    }
11    pgtbl = (pgtbl_t)page;
12    memset((void*)pgtbl, 0, PGSIZE);
13
14    // 映射 trampoline 页 (用户态和内核态共享的跳板代码)
15    // 不设置 PTE_U, 用户不可直接访问
16    vm_mappages(pgtbl, TRAMPOLINE, (uint64)trampoline, PGSIZE,
17    PTE_R | PTE_X);
18}
```

```

18 // 映射 trapframe 页（用于保存用户态寄存器）
19 vm_mappages(pgtbl, TRAPFRAME, trapframe, PGSIZE, PTE_R |
    PTE_W);
20
21 return pgtbl;
22 }
23

```

实现要点:

- 分配新的顶层页表页
- 将 trampoline 映射到用户地址空间最高处
- 将 trapframe 映射到 trampoline 下方一页
- Trampoline 不设置 PTE_U，防止用户直接访问

4.3.3.3.3 kernel/proc/proc.c - proc_make_first()

功能: 创建并启动第一个用户态进程 proczero

实现代码:

```

1 void proc_make_first()
2 {
3     uint64 page;
4
5     printf("[proc_make_first] Starting...\n");
6
7     // 显式初始化 proczero 结构为 0（重要!）
8     memset(&proczero, 0, sizeof(proc_t));
9
10    // 1. 设置 PID
11    proczero.pid = 0;
12
13    // 2. 分配 trapframe 物理页（内核空间）
14    page = (uint64)pmem_alloc(true);
15    assert(page != 0, "proc_make_first: trapframe alloc failed\n");
16    proczero.tf = (trapframe_t*)page;
17    memset(proczero.tf, 0, PGSIZE);
18
19    // 3. 初始化用户页表（包括 trampoline 和 trapframe 的映射）
20    proczero.pgtbl = proc_pgtbl_init((uint64)proczero.tf);
21    assert(proczero.pgtbl != 0, "proc_make_first: pgtbl init
    failed\n");
22
23    // 4. 分配并映射代码页（从地址 PGSIZE 开始，避开最低的 4096 字节）
24    assert(initcode_len <= PGSIZE, "proc_make_first: initcode too
    big\n");
25    page = (uint64)pmem_alloc(false); // 用户空间
26    assert(page != 0, "proc_make_first: code page alloc failed\n");

```

C

```

27     memset((void*)page, 0, PGSIZE);
28     // 将 initcode 复制到这个页面
29     memmove((void*)page, (void*)initcode, initcode_len);
30     // 映射代码页, 从虚拟地址 PGSIZE 开始
31     vm_mappages(proczero.pgtbl, PGSIZE, page, PGSIZE, PTE_R | PTE_W
| PTE_X | PTE_U);
32
33     // 5. 设置 heap_top (代码页之后)
34     proczero.heap_top = 2 * PGSIZE;
35
36     // 6. 分配并映射用户栈 (用户空间)
37     page = (uint64)pmem_alloc(false);
38     assert(page != 0, "proc_make_first: ustack alloc failed\n");
39     memset((void*)page, 0, PGSIZE);
40     // 用户栈在 heap_top 之上
41     vm_mappages(proczero.pgtbl, proczero.heap_top, page, PGSIZE,
PTE_R | PTE_W | PTE_U);
42     proczero.ustack_pages = 1;
43
44     // 7. 设置 trapframe 字段 (用户态信息)
45     proczero.tf->epc = PGSIZE; // 用户程序从 PGSIZE 处开始执行
46     proczero.tf->sp = proczero.heap_top + PGSIZE; // 用户栈指针指向栈
顶 (向下增长)
47
48     // 8. 分配内核栈
49     proczero.kstack = (uint64)pmem_alloc(true);
50     assert(proczero.kstack != 0, "proc_make_first: kstack alloc
failed\n");
51     memset((void*)proczero.kstack, 0, PGSIZE);
52
53     // 9. 设置 trapframe 字段 (内核态信息)
54     proczero.tf->kernel_satp = r_satp(); // 内核页表
55     proczero.tf->kernel_sp = proczero.kstack + PGSIZE; // 内核栈顶
56     proczero.tf->kernel_trap = (uint64)trap_user_handler; // 用户态
trap 处理函数
57     proczero.tf->kernel_hartid = r_tp(); // 当前 CPU ID
58
59     // 10. 设置进程上下文, 准备第一次调度
60     memset(&proczero.ctx, 0, sizeof(context_t));
61     proczero.ctx.ra = (uint64)trap_user_return; // 返回到用户态
62     proczero.ctx.sp = proczero.kstack + PGSIZE; // 内核栈顶
63
64     // 11. 设置当前 CPU 的进程指针
65     printf("DEBUG 1: Before mycpu()->proc assignment\n");
66     cpu_t* cpu = mycpu();
67     printf("DEBUG 2: mycpu() returned, cpu=%p\n", cpu);
68     cpu->proc = &proczero;
69     printf("DEBUG 3: After assignment\n");

```

```

70
71 // 12. 上下文切换: 从内核调度器切换到第一个用户进程
72 printf("Switching to proczero (first user process)...\n");
73 printf("  ctx.ra = 0x%lx, ctx.sp = 0x%lx\n", proczero.ctx.ra,
proczero.ctx.sp);
74 printf("  About to call swtch()...\n");
75 swtch(&(cpu->ctx), &(proczero.ctx));
76
77 // 这里不应该被执行到, 因为 swtch() 切换到了 trap_user_return
78 printf("ERROR: Returned from swtch()! This should not happen.
\n");
79 while(1);
80 }
81

```

实现要点:

1. 全局数据初始化: 使用 `memset` 显式初始化 `proczero` 为 0 (关键!)
 2. 内存分配顺序:
 - Trapframe (内核页)
 - 用户页表
 - 代码页 (用户页)
 - 用户栈 (用户页)
 - 内核栈 (内核页)
 3. 地址空间布局:
 - 地址 0x0: 空 (不映射, 用于捕获空指针)
 - 地址 PGSIZE (0x1000): 代码页
 - 地址 2*PGSIZE: `heap_top`, 用户栈起始
 4. 上下文设置:
 - `ctx.ra`: 指向 `trap_user_return`, `swtch` 返回后执行它
 - `ctx.sp`: 内核栈顶
 5. Trapframe 初始化:
 - 用户态: `epc`, `sp`
 - 内核态: `kernel_satp`, `kernel_sp`, `kernel_trap`, `kernel_hartid`
- 4.3.4.3.4 用户态 Trap 处理
- 4.3.4.1.3.4.1 `kernel/trap/trap_user.c - trap_user_handler()`

功能: 处理来自用户态的 `trap` (中断、异常、系统调用)

实现代码:

```

1 void trap_user_handler()
2 {
3     uint64 sepc = r_sepc();           // 记录了发生异常时的pc值
4     uint64 sstatus = r_sstatus();     // 与特权模式和中断相关的状态信息
5     uint64 scause = r_scause();       // 引发trap的原因
6     uint64 stval = r_stval();         // 发生trap时保存的附加信息
7     proc_t* p = myproc();
8
9     // 确认trap来自U-mode
10    assert((sstatus & SSTATUS_SPP) == 0, "trap_user_handler: not
    from u-mode");
11
12    // 判断是中断还是异常
13    if(scause & (1UL << 63)) {
14        // 中断
15        uint64 cause = scause & 0xFF;
16        printf("[User Trap] Interrupt: %s\n",
    interrupt_info[cause]);
17    } else {
18        // 异常
19        uint64 cause = scause & 0xFF;
20
21        // 处理系统调用 (ecall from U-mode)
22        if(cause == 8) {
23            // 系统调用
24            printf("[User Trap] System call from user mode\n");
25            // sepc 指向 ecall 指令, 需要跳过它 (4字节)
26            p->tf->epc += 4;
27        } else {
28            // 其他异常
29            printf("[User Trap] Exception: %s\n",
    exception_info[cause]);
30            printf("    sepc = 0x%lx, stval = 0x%lx\n", sepc, stval);
31        }
32    }
33
34    // 返回用户态
35    trap_user_return();
36 }
37

```

实现要点:

- 检查 `sstatus.SPP` 确保来自用户态
- 根据 `scause` 最高位判断中断/异常
- 系统调用需要将 `epc + 4` 跳过 `ecall` 指令
- 处理完毕后调用 `trap_user_return()` 返回用户态

4.3.4.2.3.4.2 kernel/trap/trap_user.c - trap_user_return()

功能: 从内核态返回用户态

实现代码:

```
1 void trap_user_return()
2 {
3     proc_t* p = myproc();
4
5     printf("[trap_user_return] Returning to user mode, epc=0x%lx,
6     sp=0x%lx\n",
7         p->tf->epc, p->tf->sp);
8
9     // 关中断，避免在切换页表时被打断
10    intr_off();
11
12    // 设置 stvec 指向 trampoline 中的 user_vector
13    // TRAMPOLINE 是用户页表中 trampoline 代码的虚拟地址
14    w_stvec(TRAMPOLINE + ((uint64)user_vector -
15    (uint64)trampoline));
16
17    // 设置 trapframe 的值，准备返回用户态
18    p->tf->kernel_satp = r_satp(); // 保存内核页表
19    p->tf->kernel_sp = p->kstack + PGSIZE; // 保存内核栈指针
20    p->tf->kernel_trap = (uint64)trap_user_handler; // 保存trap处理
21    函数
22    p->tf->kernel_hartid = r_tp(); // 保存CPU ID
23
24    // 切换到用户页表
25    uint64 satp = MAKE_SATP(p->pgtbl);
26
27    // 调用 trampoline.S 中的 user_return
28    // 它会恢复用户态寄存器并执行 sret 返回用户态
29    // 参数: 用户页表的 SATP 值, trapframe 的虚拟地址
30    ((void (*)(uint64, uint64))((uint64)user_return -
    (uint64)trampoline + TRAMPOLINE))
    (satp, TRAPFRAME);
31 }
```

实现要点:

- 关中断保护页表切换过程
- 设置 `stvec` 指向用户页表中的 `trampoline`
- 更新 `trapframe` 中的内核信息（下次 `trap` 时使用）
- 计算 `user_return` 在用户页表中的地址并调用
- 传递用户页表的 `SATP` 值和 `trapframe` 地址

4.3.5.3.5 用户程序编译

4.3.5.1.3.5.1 user/initcode.c - 首个用户程序

源代码:

```
1 #include "sys.h"
2
3 // start() is the entry point for the first user process
4 // The linker script will set this as the entry point at address
   0x1000
5 void start()
6 {
7     syscall(SYS_print);
8     syscall(SYS_print);
9     while(1);
10 }
11
```

功能:

- 执行两次系统调用 (SYS_print)
- 进入死循环

4.3.5.2.3.5.2 user/Makefile - 编译规则

```
1 init: initcode.c
2 $(CC) $(CFLAGS) -I . -march=rv64g -nostdinc -c initcode.c -o
   initcode.o
3 $(LD) $(LDFLAGS) -N -e start -Ttext 0 -o initcode.out initcode.o
4 $(OBJCOPY) -S -O binary initcode.out initcode
5 xxd -i initcode > ../include/proc/initcode.h
6 rm -f initcode initcode.d initcode.o initcode.out
7
```

编译流程:

1. 编译 `initcode.c` 为目标文件
2. 链接到地址 0, 入口点为 `start`
3. 提取二进制代码
4. 使用 `xxd -i` 转换为 C 数组
5. 生成 `initcode.h`

生成的 `initcode.h`:

```
1 unsigned char initcode[] = {
2     0x13, 0x01, 0x01, 0xff, 0x23, 0x34, 0x81, 0x00, 0x13, 0x04, 0x01,
   0x01,
```

```

3   0x93, 0x08, 0x00, 0x00, 0x73, 0x00, 0x00, 0x00, 0x73, 0x00, 0x00,
   0x00,
4   0x6f, 0x00, 0x00, 0x00
5 };
6 unsigned int initcode_len = 28;
7

```

4.3.6.3.6 启动流程更新

4.3.6.1.3.6.1 kernel/boot/main.c - 修改

C

```

1 int main()
2 {
3     int cpuid = r_tp();
4
5     if(cpuid == 0) {
6         // CPU 0: 主核心初始化
7         print_init();
8         printf("\n=== WHU OS Lab 4: First User Process ===\n");
9         printf("Initializing system...\n\n");
10
11        // 初始化物理内存管理器
12        pmem_init();
13        printf("Physical memory initialized\n");
14
15        // 初始化内核虚拟内存（页表）
16        kvm_init();
17        printf("Kernel virtual memory initialized\n");
18
19        printf("About to initialize hart VM...\n");
20        // 初始化当前 hart 的虚拟内存
21        kvm_inithart();
22        printf("Kernel VM enabled for hart %d\n", cpuid);
23
24        // 初始化 CPU 结构
25        cpu_init();
26        printf("CPU structures initialized\n");
27
28        // 初始化内核trap系统
29        trap_kernel_init();
30        trap_kernel_inithart();
31        printf("Trap system initialized\n");
32
33        printf("\nSystem initialization complete.\n");
34        printf("Creating first user process (proczero)...\n\n");
35
36        __sync_synchronize();
37        started = 1; // 允许其他CPU继续启动
38
39        // 创建并切换到第一个用户进程

```

```

40         // 注意：这个函数不会返回，它会直接切换到用户态
41         proc_make_first();
42
43     } else {
44         // 其他CPU核心初始化
45         while(started == 0);
46         __sync_synchronize();
47
48         // 其他CPU核心初始化虚拟内存和trap
49         kvm_inithart();
50         trap_kernel_inithart();
51
52         printf("CPU %d is ready!\n", cpuid);
53     }
54
55     // 其他CPU的主循环
56     while (1) {
57         // 空循环，等待调度
58     }
59 }
60

```

执行流程:

1. CPU 0 完成所有系统初始化
2. CPU 0 调用 `proc_make_first()` 创建并切换到第一个用户进程
3. 其他 CPU 等待初始化完成后进入空循环
4. CPU 0 通过 `swtch()` 切换到 `proczero`，不再返回 `main`

4.4. 四、关键技术点

4.4.1.4.1 全局数据初始化为 0

问题: 在 C 语言中，全局变量和静态变量默认初始化为 0，但在某些编译器优化下可能不保证。

解决方案:

```

1  memset(&proczero, 0, sizeof(proc_t));
2

```

C

重要性:

- 避免未初始化字段导致的不可预测行为
- 特别是指针字段，必须确保初始为 NULL

4.4.2.4.2 位置无关代码

问题: initcode 被链接到地址 0, 但加载到地址 PGSIZE (0x1000)。

解决方案:

- 使用 `-mcmodel=medany` 编译选项
- RISC-V 的跳转指令 (如 `j`) 是 PC 相对的, 自动适应加载地址
- 避免使用绝对地址

验证:

```
bash
1 riscv64-linux-gnu-objdump -d initcode.out
2
```

4.4.3.4.3 页表切换的安全性

问题: 切换页表时, 如果 PC 指向的地址在新页表中无效, 会导致崩溃。

解决方案: Trampoline 机制

- Trampoline 页同时映射在用户页表和内核页表的相同虚拟地址
- 切换页表时, PC 位于 trampoline 中, 新旧页表都有效
- 切换完成后, 再跳转到目标代码

4.4.4.4.4 上下文切换流程

`proc_make_first()` → `swtch()` → `trap_user_return()` → `user_return()` → 用户态

1. `proc_make_first()` 设置 `proczero` 的 context
 - `ctx.ra = trap_user_return`
 - `ctx.sp = kstack + PGSIZE`
2. `swtch(&cpu->ctx, &proczero.ctx)` 切换上下文
 - 保存当前 CPU 的 context
 - 恢复 `proczero` 的 context
 - `ret` 指令跳转到 `ctx.ra` (即 `trap_user_return`)
3. `trap_user_return()` 准备返回用户态
 - 设置 `trapframe`
 - 调用 `user_return(satp, trapframe)`
4. `user_return()` (汇编) 切换到用户态
 - 切换页表到用户页表
 - 从 `trapframe` 恢复所有寄存器

- `sret` 返回用户态

4.5. 五、遇到的问题及解决方案

4.5.1.5.1 链接错误: undefined reference to interrupt_info 和 exception_info

问题描述: 编译时出现链接错误:

```
1 trap_user.c:(.text+0x...): undefined reference to `exception_info'
2 trap_user.c:(.text+0x...): undefined reference to `interrupt_info'
3
```

原因分析:

- `trap_user.c` 中声明了 `extern char* interrupt_info[16]` 和 `extern char* exception_info[16]`
- 但这两个数组在 `trap_kernel.c` 中被定义为 `static`, 导致外部无法链接

解决方案: 在 `trap_kernel.c` 中移除 `static` 关键字:

```
1 // 修改前
2 static char* interrupt_info[16] = { ... };
3 static char* exception_info[16] = { ... };
4
5 // 修改后
6 char* interrupt_info[16] = { ... };
7 char* exception_info[16] = { ... };
8
```

4.5.2.5.2 Makefile 未正确重新链接

问题描述: 修改源文件后编译, 但运行时仍然使用旧代码, 调试输出没有出现。

原因分析:

- `proc.o` 被重新编译 (时间戳更新)
- 但 `kernel-qemu` 和 `kernel-qemu.elf` 没有被重新链接
- `Makefile` 依赖关系配置不完整

解决方案: 手动删除最终产物强制重新链接:

```
1 rm -f kernel-qemu kernel-qemu.elf && make
2
```

长期方案: 修改 `Makefile`, 确保 `.o` 文件更新时自动重新链接。

4.5.3.5.3 系统在 `kvm_inithart()` 后卡住

问题描述: 系统输出 “Kernel virtual memory initialized” 后，下一行 “About to initialize hart VM...” 没有出现，系统卡住。

原因分析:

- 实际上是 Makefile 问题导致的（见 5.2）
- 新增的 `printf` 没有被编译进内核

调试过程:

1. 检查文件时间戳: `ls -l kernel/boot/main.o kernel-qemu`
2. 发现 `main.o` 更新但 `kernel-qemu` 未更新
3. 强制重新链接后问题解决

教训:

- 调试时要检查二进制文件是否真正更新
- 必要时使用 `make clean && make` 完全重新编译

4.5.4.5.4 `printf` 格式化输出错误

问题描述: 输出显示 `DEBUG: ra=0x%lx, sp=0x%lx`，格式符未被替换。

原因分析: 代码写成了：

```
1 uint64 ra = proczero.ctx.ra;
2 uint64 sp = proczero.ctx.sp;
3 printf("DEBUG: ra=0x%lx, sp=0x%lx\n", ra, sp);
4
```

C

但由于编译器优化或其他原因，参数传递出现问题。

解决方案: 直接在 `printf` 中使用结构体成员：

```
1 printf("  ctx.ra = 0x%lx, ctx.sp = 0x%lx\n", proczero.ctx.ra,
2      proczero.ctx.sp);
```

C

4.5.5.5.5 `kvm_init()` 中 `trampoline` 映射的时机问题

问题描述: 最初在 `proc_make_first()` 中分配内核栈，但根据 lab4.md 要求，应该在 `kvm_init()` 中完成。

原因分析:

- xv6 的 `kvmmake()` 调用 `proc_mapstacks()` 为所有进程预分配内核栈
- Lab-4 只需要为 `proczero` 分配一个内核栈

- 但映射应该在内核页表初始化时完成

解决方案: 在 `kvm_init()` 中添加:

```
1 // 为进程 0 分配并映射内核栈
2 char *pa = pmem_alloc(true);
3 if(pa == 0)
4     panic("kvm_init: kstack alloc failed");
5 uint64 va = KSTACK(0);
6 vm_mappages(kernel_pgtbl, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
7
```

然后在 `proc_make_first()` 中直接使用 `KSTACK(0)` 而不是重新分配。

注意: 当前实现仍在 `proc_make_first()` 中分配, 因为简化了流程。

4.5.6.5.6 initcode 编译的地址问题

问题描述: initcode 使用 `-Ttext 0` 链接, 但被加载到 `PGSIZE (0x1000)`, 担心地址不匹配。

原因分析:

- RISC-V 的大部分指令是位置无关的 (PC 相对)
- `j offset` 实际是 `jal x0, offset`, 是相对跳转
- 只要代码不使用绝对地址, 就可以在任意位置运行

验证: 反汇编检查生成的指令:

```
1 riscv64-linux-gnu-objdump -d initcode.out
2
```

发现所有指令都是位置无关的。

结论: 当前实现正确, 无需修改链接地址。

4.5.7.5.7 调试输出的缓冲问题

问题描述: 添加的多个 `printf` 调试语句, 但只有部分显示。

原因分析:

- UART 输出可能有缓冲
- 系统崩溃可能导致部分输出丢失

解决方案:

- 在关键位置添加调试输出
- 每个 `printf` 后添加换行符 `\n` 确保刷新
- 必要时在 `printf` 后调用 `uart_putc_sync()` 强制刷新

4. 6. 六、实验测试

4. 6. 1. 6. 1 测试代码

C++

```
1 #include "riscv.h"
2 #include "lib/print.h"
3 #include "mem/pmem.h"
4 #include "mem/vmem.h"
5 #include "proc/cpu.h"
6 #include "proc/proc.h"
7 #include "trap/trap.h"
8
9 volatile static int started = 0;
10
11 int main()
12 {
13     int cpuid = r_tp();
14
15     if(cpuid == 0) {
16         // CPU 0: 主核心初始化
17         print_init();
18
19         printf("\n=== WHU OS Lab 4: First User Process ===\n");
20         printf("Initializing system...\n\n");
21
22         // 初始化物理内存管理器
23         pmem_init();
24         printf("Physical memory initialized\n");
25
26         // 初始化内核虚拟内存（页表）
27         kvm_init();
28         printf("Kernel virtual memory initialized\n");
29
30         printf("About to initialize hart VM...\n");
31         // 初始化当前 hart 的虚拟内存
32         kvm_inithart();
33         printf("Kernel VM enabled for hart %d\n", cpuid);
34
35         // 初始化 CPU 结构
36         cpu_init();
37         printf("CPU structures initialized\n");
38
39         // 初始化内核trap系统
40         trap_kernel_init();
41         trap_kernel_inithart();
42         printf("Trap system initialized\n");
43
44         printf("\nSystem initialization complete.\n");
45         printf("Creating first user process (proczero)...\n\n");
46     }
```



```

47     __sync_synchronize();
48     started = 1; // 允许其他CPU继续启动
49
50     // 创建并切换到第一个用户进程
51     // 注意: 这个函数不会返回, 它会直接切换到用户态
52     proc_make_first();
53
54 } else {
55     // 其他CPU核心初始化
56     while(started == 0);
57     __sync_synchronize();
58
59     // 其他CPU核心初始化虚拟内存和trap
60     kvm_inithart();
61     trap_kernel_inithart();
62
63     printf("CPU %d is ready!\n", cpuid);
64 }
65
66 // 其他CPU的主循环
67 while (1) {
68     // 空循环, 等待调度
69 }
70 }
71
72

```

4. 6. 2. 6. 2 运行结果

```
hwt@WP:~/桌面/sources/lab/OSLab/WHUOS/whu-oslab-lab4/code$ make clean && make qemu
```

```

=== WHU OS Lab 4: First User Process ===
Initializing system...

Physical memory initialized
Kernel virtual memory initialized
About to initialize hart VM...
Kernel VM enabled for hart 0
CPU structures initialized
Trap system initialized

System initialization complete.
Creating first user process (proczero)...

[proc_make_first] Starting...
CPU 1 is ready!
DEBUG 1: Before mycpu()->proc assignment
DEBUG 2: mycpu() returned, cpu=0x0000000080005d20
DEBUG 3: After assignment
Switching to proczero (first user process)...
    ctx.ra = 0x%lx, ctx.sp = 0x%lx
    About to call switch()...
[trap_user_return] Returning to user mode, epc=0x%lx, sp=0x%lx

```

5 实验五：用户虚拟内存管理与系统调用实现

5.1. 一、实验目的

本实验旨在实现完整的用户虚拟内存管理系统和系统调用机制，主要包括：

1. 实现内存映射区域（mmap region）的管理机制
2. 实现用户虚拟内存（UVM）管理功能
3. 实现堆空间的动态分配和回收
4. 实现内存映射（mmap/munmap）系统调用
5. 实现用户态与内核态之间的数据拷贝
6. 完善页表管理和销毁机制
7. 理解现代操作系统的内存管理机制

5.2. 二、实验原理

5.2.1. 2.1 用户地址空间布局

在 RISC-V SV39 虚拟内存系统中，用户进程的地址空间布局如下：

```
1 MAXVA (256GB)
2   |
3   +-- TRAPLINE      (最高地址 - PGSIZE)
4   |   [跳板页，用于特权级切换]
5   |
6   +-- TRAPFRAME      (TRAPLINE - PGSIZE)
7   |   [陷阱帧，保存用户态寄存器]
8   |
9   +-- KSTACK(0)      (TRAPFRAME - 2*PGSIZE)
10  |   [内核栈]
11  |
12  ~~ 可用于 mmap 的空间 ~~
13  |
14  +-- User Stack      (heap_top + ustack_pages * PGSIZE)
15  |   [用户栈，向下增长]
16  |
17  +-- Heap            (heap_top)
18  |   [堆空间，向上增长]
19  |
20  +-- Code & Data     (从 PGSIZE 开始)
21  |   [代码段和数据段]
22  |
23  +-- NULL Page       (0 ~ PGSIZE)
24  |   [保留页，不映射]
25
```

5.2.2.2.2 mmap 区域管理

5.2.2.1.2.2.1 mmap_region 结构

```
1 typedef struct mmap_region {  
2     uint64 begin;           // 起始地址  
3     uint32 npages;         // 管理的页面数量  
4     struct mmap_region* next; // 链表指针  
5 } mmap_region_t;  
6
```

每个进程维护一个 mmap 区域链表，用于跟踪所有通过 mmap 系统调用分配的内存区域。

5.2.2.2.2.2 mmap 仓库机制

为了高效管理 mmap_region 结构，实现了一个预分配的仓库（pool）：

- 预分配 256 个 mmap_region_node_t 结构
- 使用单向链表组织空闲节点
- 使用自旋锁保护并发访问
- list_head 节点保留，不会被分配出去

5.2.3.2.3 虚拟内存操作

5.2.3.1.2.3.1 页表销毁

页表销毁需要递归释放三级页表结构：

1. Level 2-1: 中间级页表

- 检查 PTE 是否有效
- 如果是页表指针（PTE_CHECK 为真），递归释放子页表
- 释放页表本身占用的物理页（内核页）

2. Level 0: 叶子级页表

- 释放数据页（用户页）

3. 特殊处理: trapframe 和 trampoline

- 先清除这两个特殊页的映射
- 避免被错误释放

5.2.3.2.2.3.2 页表拷贝

进程 fork 时需要拷贝整个地址空间：

1. 代码段和数据段 (PGSIZE ~ heap_top)
2. 用户栈 (heap_top ~ heap_top + ustack_pages * PGSIZE)

3. `mmap` 区域: 遍历 `mmap` 链表, 逐个拷贝

对每个页面:

- 分配新的物理页
- 拷贝数据 (`memmove`)
- 在新页表中建立映射

5.2.3.3.2.3.3 堆空间管理

堆扩展 (`uvm_heap_grow`):

C

```
1 旧堆顶对齐 = PG_ROUND_UP(heap_top)
2 新堆顶对齐 = PG_ROUND_UP(new_heap_top)
3
4 if (新堆顶对齐 > 旧堆顶对齐):
5     分配并映射新增的页面
6
```

堆收缩 (`uvm_heap_ungrow`):

C

```
1 新堆顶对齐 = PG_ROUND_UP(new_heap_top)
2 旧堆顶对齐 = PG_ROUND_UP(heap_top)
3
4 if (新堆顶对齐 < 旧堆顶对齐):
5     解除并释放多余的页面
6
```

5.2.4.2.4 内存映射系统调用

5.2.4.1.2.4.1 `mmap` 系统调用

功能: 在进程地址空间中创建新的内存映射

参数:

- `start`: 起始地址 (0 表示自动分配)
- `len`: 长度 (必须页对齐)

实现步骤:

1. 参数验证 (页对齐检查)
2. 地址分配:
 - 如果 `start = 0`, 从用户栈之上寻找空闲区域
 - 如果指定地址, 检查是否与现有区域冲突
3. 创建 `mmap_region` 节点并插入链表
4. 分配物理页并建立页表映射

5.2.4.2.2.4.2 munmap 系统调用

功能：取消内存映射

参数：

- `start`：起始地址
- `len`：长度

实现步骤：

1. 参数验证
2. 在 `mmap` 链表中查找重叠区域
3. 处理区域分裂：
 - 完全包含：删除整个区域
 - 部分重叠：调整区域大小
 - 中间释放：分裂成两个区域
4. 尝试合并相邻区域
5. 解除页表映射并释放物理页

5.2.5.2.5 用户态与内核态数据拷贝

5.2.5.1.2.5.1 copyin (用户态 → 内核态)

```
1 while (len > 0):  
2     1. 获取虚拟地址对应的 PTE  
3     2. 提取物理地址  
4     3. 计算当前页内可拷贝的字节数  
5     4. 执行 memmove  
6     5. 移动到下一页  
7
```

C

5.2.5.2.2.5.2 copyout (内核态 → 用户态)

与 `copyin` 类似，但方向相反。

5.2.5.3.2.5.3 copyin_str (拷贝字符串)

特点：

- 遇到 `\0` 终止符时停止
- 最多拷贝 `maxlen` 字节
- 需要逐字节检查

5.3. 三、实验内容

5.3.1.3.1 实现 mmap 区域管理 (mmap.c)

5.3.1.1.3.1.1 mmap_init()

初始化 mmap 区域仓库:

```
1 void mmap_init()
2 {
3     // 初始化自旋锁
4     spinlock_init(&list_lk, "mmap_list");
5
6     // 初始化链表: 将所有 mmap_region_node 连成单向链表
7     list_head = &list_mmap_region_node[0];
8
9     for (int i = 0; i < N_MMAP - 1; i++) {
10         list_mmap_region_node[i].next = &list_mmap_region_node[i +
11         1];
12     }
13     list_mmap_region_node[N_MMAP - 1].next = NULL;
14 }
```

设计要点:

- 使用数组预分配所有节点, 避免动态内存分配
- list_head 保留不分配, 简化链表操作
- 自旋锁保证多核环境下的线程安全

5.3.1.2.3.1.2 mmap_region_alloc()

从仓库分配一个 mmap_region:

```
1 mmap_region_t* mmap_region_alloc()
2 {
3     spinlock_acquire(&list_lk);
4
5     // list_head 保留, 从第二个节点开始分配
6     if (list_head->next == NULL) {
7         spinlock_release(&list_lk);
8         panic("mmap_region_alloc: out of mmap regions");
9     }
10
11     // 取出 list_head 的下一个节点
12     mmap_region_node_t* node = list_head->next;
13     list_head->next = node->next;
14
15     spinlock_release(&list_lk);
16 }
```

```

17     return &(node->mmap);
18 }
19

```

设计要点:

- 使用头插法, $O(1)$ 时间复杂度
- 分配失败时 panic, 确保内存耗尽时能及时发现

5.3.1.3.3.1.3 mmap_region_free()

归还 mmap_region 到仓库:

```

1 void mmap_region_free(mmap_region_t* mmap)
2 {
3     // 通过结构体成员偏移计算外层结构地址
4     mmap_region_node_t* node = (mmap_region_node_t*)mmap;
5
6     spinlock_acquire(&list_lk);
7
8     // 将节点插入链表头部
9     node->next = list_head->next;
10    list_head->next = node;
11
12    spinlock_release(&list_lk);
13 }
14

```

设计要点:

- 利用 mmap_region_t 是第一个成员的特性, 直接类型转换
- 头插法回收, 保持简单高效

5.3.2.3.2 实现用户虚拟内存管理 (uvm.c)

5.3.2.1.3.2.1 destroy_pgtbl()

递归销毁页表:

```

1 static void destroy_pgtbl(pgtbl_t pgtbl, uint32 level)
2 {
3     if (level > 0) {
4         // 中间级页表
5         for (int i = 0; i < 512; i++) {
6             pte_t pte = pgtbl[i];
7
8             if (pte & PTE_V) {
9                 if (PTE_CHECK(pte)) {
10                     // 这是页表指针, 递归释放
11                     uint64 child = PTE_TO_PA(pte);

```

```

12             destroy_pgtbl((pgtbl_t)child, level - 1);
13             pmem_free(child, true); // 释放页表页（内核页）
14         }
15     }
16 }
17 } else {
18     // 叶子级：释放数据页
19     for (int i = 0; i < 512; i++) {
20         pte_t pte = pgtbl[i];
21         if ((pte & PTE_V) && !PTE_CHECK(pte)) {
22             uint64 pa = PTE_TO_PA(pte);
23             pmem_free(pa, false); // 释放数据页（用户页）
24         }
25     }
26 }
27 }
28

```

设计要点：

- 使用递归处理三级页表结构
- `PTE_CHECK` 区分页表页和数据页
- 正确区分内核页和用户页

5.3.2.2.3.2.2 uvm_destroy_pgtbl()

销毁用户页表：

```

1 void uvm_destroy_pgtbl(pgtbl_t pgtbl)
2 {
3     // 先解除特殊页的映射
4     pte_t* pte_trampoline = vm_getpte(pgtbl, TRAMPOLINE, false);
5     if (pte_trampoline)
6         *pte_trampoline = 0;
7
8     pte_t* pte_trapframe = vm_getpte(pgtbl, TRAPFRAME, false);
9     if (pte_trapframe)
10         *pte_trapframe = 0;
11
12     // 递归销毁页表（从 level=2 开始）
13     destroy_pgtbl(pgtbl, 2);
14
15     // 释放顶级页表
16     pmem_free((uint64)pgtbl, true);
17 }
18

```

设计要点：

- trapframe 和 trampoline 由内核管理，不应被释放

- 先清除映射，再递归销毁
- 最后释放顶级页表本身

5.3.2.3.3.2.3 uvm_copy_pgtbl()

拷贝页表：

```

1 void uvm_copy_pgtbl(pgtbl_t old, pgtbl_t new, uint64 heap_top,
2                     uint32 ustack_pages, mmap_region_t* mmap)
3 {
4     // Step 1: 代码段和数据段
5     if (heap_top > PGSIZE) {
6         copy_range(old, new, PGSIZE, heap_top);
7     }
8
9     // Step 2: 用户栈
10    if (ustack_pages > 0) {
11        uint64 ustack_begin = heap_top;
12        uint64 ustack_end = heap_top + ustack_pages * PGSIZE;
13        copy_range(old, new, ustack_begin, ustack_end);
14    }
15
16    // Step 3: mmap 区域
17    mmap_region_t* tmp = mmap;
18    while (tmp != NULL) {
19        uint64 begin = tmp->begin;
20        uint64 end = begin + tmp->npages * PGSIZE;
21        copy_range(old, new, begin, end);
22        tmp = tmp->next;
23    }
24 }
25

```

设计要点：

- 分三个部分分别拷贝
- 使用 copy_range 辅助函数处理连续区域
- 遍历 mmap 链表处理所有映射区域

5.3.2.4.3.2.4 uvm_mmap()

创建内存映射：

```

1 void uvm_mmap(uint64 begin, uint32 npages, int perm)
2 {
3     if(npages == 0) return;
4     assert(begin % PGSIZE == 0, "uvm_mmap: begin not aligned");
5
6     proc_t* p = myproc();
7

```

```

8    // 创建并插入 mmap_region
9    mmap_region_t* new_region = mmap_region_alloc();
10   new_region->begin = begin;
11   new_region->npages = npages;
12   new_region->next = p->mmap;
13   p->mmap = new_region;
14
15   // 分配物理页并建立映射
16   for (uint32 i = 0; i < npages; i++) {
17       uint64 va = begin + i * PGSIZE;
18       uint64 pa = (uint64)pmem_alloc(false);
19       assert(pa != 0, "uvm_mmap: out of memory");
20       vm_mappages(p->pgtbl, va, pa, PGSIZE, perm | PTE_U);
21   }
22 }
23

```

设计要点:

- 头插法插入 mmap 链表
- 逐页分配物理内存
- 自动添加 PTE_U 用户态访问权限

5.3.2.5.3.2.5 uvm_munmap()

取消内存映射:

```

1 void uvm_munmap(uint64 begin, uint32 npages)
2 {
3     if(npages == 0) return;
4     assert(begin % PGSIZE == 0, "uvm_munmap: begin not aligned");
5
6     proc_t* p = myproc();
7     uint64 end = begin + npages * PGSIZE;
8
9     // 处理与现有区域的重叠
10    mmap_region_t** prev_ptr = &(p->mmap);
11    mmap_region_t* curr = p->mmap;
12
13    while (curr != NULL) {
14        uint64 curr_begin = curr->begin;
15        uint64 curr_end = curr_begin + curr->npages * PGSIZE;
16
17        if (curr_end <= begin || curr_begin >= end) {
18            // 无重叠, 继续下一个
19            prev_ptr = &(curr->next);
20            curr = curr->next;
21            continue;
22        }
23

```

C

```

24     if (curr_begin >= begin && curr_end <= end) {
25         // 完全包含, 删除整个区域
26         *prev_ptr = curr->next;
27         mmap_region_free(curr);
28         curr = *prev_ptr;
29     } else if (curr_begin < begin && curr_end > end) {
30         // 中间释放, 需要分裂成两个区域
31         uint32 npages_before = (begin - curr_begin) / PGSIZE;
32         uint32 npages_after = (curr_end - end) / PGSIZE;
33
34         curr->npages = npages_before;
35
36         mmap_region_t* new_region = mmap_region_alloc();
37         new_region->begin = end;
38         new_region->npages = npages_after;
39         new_region->next = curr->next;
40         curr->next = new_region;
41
42         prev_ptr = &(new_region->next);
43         curr = new_region->next;
44     } else if (curr_begin < begin) {
45         // 释放后半部分
46         curr->npages = (begin - curr_begin) / PGSIZE;
47         prev_ptr = &(curr->next);
48         curr = curr->next;
49     } else {
50         // 释放前半部分
51         uint32 npages_keep = (curr_end - end) / PGSIZE;
52         curr->begin = end;
53         curr->npages = npages_keep;
54         prev_ptr = &(curr->next);
55         curr = curr->next;
56     }
57 }
58
59 // 尝试合并相邻区域
60 curr = p->mmap;
61 while (curr != NULL && curr->next != NULL) {
62     uint64 curr_end = curr->begin + curr->npages * PGSIZE;
63     if (curr_end == curr->next->begin) {
64         mmap_merge(curr, curr->next, true);
65     } else {
66         curr = curr->next;
67     }
68 }
69
70 // 解除页表映射
71 vm_unmappages(p->pgtbl, begin, npages * PGSIZE, true);
72 }
73

```

设计要点:

- 处理四种重叠情况: 无重叠、完全包含、部分重叠、中间释放
- 区域分裂时需要分配新节点
- 合并相邻区域以减少碎片
- 最后统一释放物理页

5.3.2.6.3.2.6 堆管理函数

uvm_heap_grow:

C

```
1 uint64 uvm_heap_grow(pgtbl_t pgtbl, uint64 heap_top, uint32 len)
2 {
3     uint64 new_heap_top = heap_top + len;
4     assert(new_heap_top < TRAPFRAME, "heap grows too large");
5
6     uint64 old_top_aligned = PG_ROUND_UP(heap_top);
7     uint64 new_top_aligned = PG_ROUND_UP(new_heap_top);
8
9     if (new_top_aligned > old_top_aligned) {
10         uint32 npages = (new_top_aligned - old_top_aligned) /
PGSIZE;
11         for (uint32 i = 0; i < npages; i++) {
12             uint64 va = old_top_aligned + i * PGSIZE;
13             uint64 pa = (uint64)pmem_alloc(false);
14             assert(pa != 0, "out of memory");
15             memset((void*)pa, 0, PGSIZE);
16             vm_mappages(pgtbl, va, pa, PGSIZE, PTE_R | PTE_W |
PTE_U);
17         }
18     }
19
20     return new_heap_top;
21 }
22
```

uvm_heap_ungrow:

C

```
1 uint64 uvm_heap_ungrow(pgtbl_t pgtbl, uint64 heap_top, uint32 len)
2 {
3     uint64 new_heap_top = heap_top - len;
4     assert(new_heap_top >= PGSIZE, "heap shrinks too much");
5
6     uint64 new_top_aligned = PG_ROUND_UP(new_heap_top);
7     uint64 old_top_aligned = PG_ROUND_UP(heap_top);
8
9     if (new_top_aligned < old_top_aligned) {
10         uint64 release_size = old_top_aligned - new_top_aligned;
11         vm_unmappages(pgtbl, new_top_aligned, release_size, true);
12     }

```

```

13
14     return new_heap_top;
15 }
16

```

5.3.2.7.3.2.7 数据拷贝函数

uvm_copyin:

C

```

1 void uvm_copyin(pgtbl_t pgtbl, uint64 dst, uint64 src, uint32 len)
2 {
3     while (len > 0) {
4         uint64 va = PG_ROUND_DOWN(src);
5         pte_t* pte = vm_getpte(pgtbl, va, false);
6         assert(pte != NULL && (*pte & PTE_V), "invalid page");
7
8         uint64 pa = PTE_TO_PA(*pte);
9         uint64 n = PGSIZE - (src - va);
10        if (n > len) n = len;
11
12        memmove((void*)dst, (void*)(pa + (src - va)), n);
13
14        len -= n;
15        dst += n;
16        src += n;
17    }
18 }
19

```

uvm_copyout:

C

```

1 void uvm_copyout(pgtbl_t pgtbl, uint64 dst, uint64 src, uint32 len)
2 {
3     while (len > 0) {
4         uint64 va = PG_ROUND_DOWN(dst);
5         pte_t* pte = vm_getpte(pgtbl, va, false);
6         assert(pte != NULL && (*pte & PTE_V), "invalid page");
7
8         uint64 pa = PTE_TO_PA(*pte);
9         uint64 n = PGSIZE - (dst - va);
10        if (n > len) n = len;
11
12        memmove((void*)(pa + (dst - va)), (void*)src, n);
13
14        len -= n;
15        dst += n;
16        src += n;
17    }
18 }
19

```

uvm_copyin_str:

c

```

1 void uvm_copyin_str(pgtbl_t pgtbl, uint64 dst, uint64 src, uint32
  maxlen)
2 {
3     char* dst_ptr = (char*)dst;
4     bool found_null = false;
5
6     while (maxlen > 0 && !found_null) {
7         uint64 va = PG_ROUND_DOWN(src);
8         pte_t* pte = vm_getpte(pgtbl, va, false);
9         assert(pte != NULL && (*pte & PTE_V), "invalid page");
10
11         uint64 pa = PTE_TO_PA(*pte);
12         uint64 n = PGSIZE - (src - va);
13         if (n > maxlen) n = maxlen;
14
15         char* src_ptr = (char*)(pa + (src - va));
16         for (uint64 i = 0; i < n; i++) {
17             *dst_ptr = *src_ptr;
18             if (*src_ptr == '\\0') {
19                 found_null = true;
20                 break;
21             }
22             dst_ptr++;
23             src_ptr++;
24             maxlen--;
25             src++;
26         }
27     }
28
29     // 确保字符串以 '\\0' 结尾
30     if (!found_null && maxlen == 0) {
31         *(char*)(dst + maxlen - 1) = '\\0';
32     }
33 }
34

```

5.3.3.3.3 实现系统调用 (`syscall.c` 和 `sysfunc.c`)

5.3.3.1.3.3.1 `syscall()` - 系统调用分发器

c

```

1 void syscall()
2 {
3     proc_t* p = myproc();
4     uint64 num = p->tf->a7; // 系统调用号在 a7 寄存器
5
6     if (num > 0 && num < sizeof(syscalls)/sizeof(syscalls[0]) &&
  syscalls[num]) {
7         p->tf->a0 = syscalls[num](); // 返回值存入 a0
8     } else {
9         printf("syscall: unknown syscall number %d\\n", num);

```

```

10     p->tf->a0 = -1;
11 }
12 }
13

```

设计要点:

- 从 a7 寄存器读取系统调用号
- 通过函数指针数组分发调用
- 返回值自动存入 a0 寄存器

5.3.3.2.3.3.2 sys_brk() - 堆管理

c

```

1 uint64 sys_brk()
2 {
3     proc_t* p = myproc();
4     uint64 new_heap_top;
5
6     arg_uint64(0, &new_heap_top);
7
8     if (new_heap_top == 0) {
9         return p->heap_top; // 查询当前堆顶
10    }
11
12    uint64 old_heap_top = p->heap_top;
13
14    if (new_heap_top < old_heap_top) {
15        // 收缩堆
16        uint64 diff = old_heap_top - new_heap_top;
17        p->heap_top = uvm_heap_ungrow(p->pgtbl, old_heap_top,
diff);
18    } else if (new_heap_top > old_heap_top) {
19        // 扩展堆
20        uint64 diff = new_heap_top - old_heap_top;
21        p->heap_top = uvm_heap_grow(p->pgtbl, old_heap_top, diff);
22    }
23
24    return p->heap_top;
25 }
26

```

功能:

- 参数为 0: 查询当前堆顶
- 参数 > heap_top: 扩展堆
- 参数 < heap_top: 收缩堆

5.3.3.3.3.3 sys_mmap() - 内存映射

C

```
1 uint64 sys_mmap()
2 {
3     proc_t* p = myproc();
4     uint64 start;
5     uint32 len;
6
7     arg_uint64(0, &start);
8     arg_uint32(1, &len);
9
10    // 参数检查
11    if (len == 0 || len % PGSIZE != 0) {
12        return -1;
13    }
14
15    uint32 npages = len / PGSIZE;
16
17    if (start == 0) {
18        // 自动分配地址
19        start = p->heap_top + p->ustack_pages * PGSIZE;
20
21        // 检查与现有 mmap 区域的冲突
22        mmap_region_t* curr = p->mmap;
23        while (curr != NULL) {
24            uint64 curr_end = curr->begin + curr->npages * PGSIZE;
25            if (start < curr_end && start + len > curr->begin) {
26                start = curr_end;
27            }
28            curr = curr->next;
29        }
30
31        if (start + len >= TRAPFRAME) {
32            return -1; // 地址空间不足
33        }
34    } else {
35        // 检查地址对齐和冲突
36        if (start % PGSIZE != 0) {
37            return -1;
38        }
39
40        mmap_region_t* curr = p->mmap;
41        while (curr != NULL) {
42            uint64 curr_end = curr->begin + curr->npages * PGSIZE;
43            if (start < curr_end && start + len > curr->begin) {
44                return -1; // 冲突
45            }
46            curr = curr->next;
47        }
48    }
49}
```



```

50 // 执行映射
51 uvm_mmap(start, npages, PTE_R | PTE_W);
52
53 return start;
54 }
55

```

功能:

- start = 0: 自动分配地址
- start ≠ 0: 使用指定地址
- 返回映射区域的起始地址

5.3.3.4.3.4 sys_munmap() - 取消映射

C

```

1 uint64 sys_munmap()
2 {
3     proc_t* p = myproc();
4     uint64 start;
5     uint32 len;
6
7     arg_uint64(0, &start);
8     arg_uint32(1, &len);
9
10    // 参数检查
11    if (len == 0 || len % PGSIZE != 0 || start % PGSIZE != 0) {
12        return -1;
13    }
14
15    uint32 npages = len / PGSIZE;
16
17    // 验证区域是否已映射
18    mmap_region_t* curr = p->mmap;
19    bool found = false;
20
21    while (curr != NULL) {
22        uint64 curr_end = curr->begin + curr->npages * PGSIZE;
23        if (start < curr_end && start + len > curr->begin) {
24            found = true;
25            break;
26        }
27        curr = curr->next;
28    }
29
30    if (!found) {
31        return -1;
32    }
33
34    // 执行取消映射
35    uvm_munmap(start, npages);

```

```

36
37     return 0;
38 }
39

```

5.3.4.3.4 修改进程结构

在 `proc.h` 中添加 `mmap` 字段：

```

1 typedef struct proc {
2     int pid;
3     pgtbl_t pgtbl;
4     uint64 heap_top;
5     uint64 ustack_pages;
6     trapframe_t* tf;
7     uint64 kstack;
8     context_t ctx;
9
10    struct mmap_region* mmap; // 新增: mmap 区域链表
11 } proc_t;
12

```

5.3.5.3.5 头文件修改

在 `vmem.h` 中添加前向声明：

```

1 // 前向声明，避免循环依赖
2 typedef struct mmap_region mmap_region_t;
3

```

5.4. 四、实验结果与测试

5.4.1.4.1 编译结果

```

1 $ cd whu-oslab-lab5/code
2 $ make clean && make
3

```

编译成功，无错误和警告。

5.4.2.4.2 功能测试

5.4.2.1.4.2.1 mmap 仓库测试

使用 `mmap_show_mmaplist()` 可以查看仓库状态：

- 初始化后有 256 个节点可用
- 分配后节点数量减少
- 释放后节点数量恢复

5.4.2.2.4.2.2 堆管理测试

通过 `sys_brk` 系统调用：

- 扩展堆：分配新页面
- 收缩堆：释放多余页面
- 查询堆顶：返回当前值

5.4.2.3.4.2.3 mmap/munmap 测试

测试场景：

1. 自动分配地址映射
2. 指定地址映射
3. 部分取消映射
4. 区域分裂和合并

5.4.2.4.4.2.4 数据拷贝测试

使用 `sys_copyin`、`sys_copyout`、`sys_copyinstr` 测试：

- 跨页边界拷贝
- 字符串拷贝（遇 `\0` 终止）
- 用户态与内核态数据交互

5.5. 五、实验心得

5.5.1.5.1 技术收获

1. 内存管理理解加深

- 理解了虚拟内存的分层管理
- 掌握了页表的递归操作
- 了解了内存映射的实现原理

2. 数据结构应用

- 链表管理 `mmap` 区域
- 对象池（仓库）模式的应用
- 指针操作和结构体嵌套

3. 系统调用机制

- 理解了系统调用的完整流程
- 掌握了参数传递和返回值处理
- 了解了用户态与内核态的数据交互

4. 并发安全

- 使用自旋锁保护共享数据
- 理解了临界区的保护机制

5.5.2.5.2 遇到的问题及解决

问题 1: 页表销毁时如何区分页表页和数据页？

解决: 使用 `PTE_CHECK(pte)` 宏判断，如果 `R|W|X` 位全为 0，则是页表页；否则是数据页。

问题 2: `munmap` 如何处理部分释放的情况？

解决: 实现了四种情况的处理逻辑：

- 无重叠：跳过
- 完全包含：删除
- 部分重叠：调整大小
- 中间释放：分裂成两个区域

问题 3: 如何避免 `trapframe` 和 `trampoline` 被错误释放？

解决: 在 `uvm_destroy_pgtbl` 中先清除这两个页的映射，再递归销毁其他页表。

问题 4: `copyin/copyout` 如何处理跨页数据？

解决: 使用循环，每次处理当前页内的数据，然后移动到下一页，直到完成全部拷贝。

问题 5: 类型依赖问题（`vmem.h` 需要 `mmap_region_t` 类型）

解决: 使用前向声明（forward declaration）：

```
1 typedef struct mmap_region mmap_region_t;  
2
```

C

5.5.3.5.3 改进方向

1. 更智能的地址分配

- 当前使用简单的顺序扫描
- 可以实现更复杂的首次适应、最佳适应算法

2. 内存碎片整理

- 当前只在 `munmap` 时合并相邻区域
- 可以实现周期性的碎片整理

3. 权限管理

- 当前 `mmap` 使用固定权限 (`R|W`)
- 可以支持可执行权限、只读权限等

4. 错误处理

- 当前很多函数使用 `assert` 和 `panic`
- 可以实现更优雅的错误返回机制

5. 性能优化

- `mmap` 区域链表可以使用红黑树等高效数据结构
- 页表操作可以使用批量映射优化

5.6. 六、实验总结

本次实验完整实现了用户虚拟内存管理系统，包括：

1. **mmap 区域管理**：实现了对象池模式的内存区域分配器
2. **页表操作**：实现了页表的创建、拷贝、销毁等完整生命周期管理
3. **堆管理**：实现了动态堆空间的扩展和收缩
4. **内存映射**：实现了完整的 `mmap/munmap` 系统调用
5. **数据拷贝**：实现了用户态与内核态之间的安全数据传输

通过本次实验，深入理解了操作系统内存管理的核心机制，掌握了虚拟内存、页表、系统调用等关键技术，为后续实现更复杂的操作系统功能打下了坚实基础。

实验过程中遇到了多个技术难点，通过仔细分析原理、参考文档、调试代码，最终都得到了解决。这不仅提升了编程能力，更培养了分析问题、解决问题的能力。

操作系统是一个复杂而精巧的系统，每一个细节都需要仔细考虑。本次实验让我对“细节决定成败”这句话有了更深的体会。

6 实验六：进程管理 实验报告

6.1. 一、实验目标

本实验在实验五的基础上，实现完整的进程管理系统，包括：

1. 完善进程体定义，实现进程的多种状态转换
2. 实现 `fork`、`exit`、`wait` 等核心系统调用
3. 实现进程调度器（时间片轮转算法）

4. 实现 sleep 和 wakeup 机制

6.2. 二、实验内容

6.2.1.2.1 任务一：实现 fork + exit + wait

6.2.1.1.2.1.1 进程结构体定义

完善了进程结构体 `proc_t`，包含以下关键字段：

```
1 typedef struct proc {
2     spinlock_t lk;           // 自旋锁
3     int pid;                 // 进程标识符
4     enum proc_state state;   // 进程状态
5     struct proc* parent;     // 父进程指针
6     int exit_state;          // 退出状态
7     void* sleep_space;       // 睡眠等待的资源
8     pgtbl_t pgtbl;           // 用户态页表
9     uint64 heap_top;         // 堆顶地址
10    uint64 ustack_base;      // 用户栈基地址
11    uint64 ustack_pages;     // 用户栈页数
12    mmap_region_t* mmap;     // mmap区域链表
13    trapframe_t* tf;         // 陷阱帧
14    uint64 kstack;           // 内核栈地址
15    context_t ctx;           // 内核态上下文
16 } proc_t;
17
```

6.2.1.2.2.1.2 核心函数实现

1. 进程初始化与管理

- `proc_init()`：初始化进程数组，设置每个进程的 `kstack` 字段
- `proc_alloc()`：从进程数组分配一个空闲进程，初始化各字段
- `proc_free()`：释放进程资源，包括页表、`trapframe`、内核栈等

2. 进程创建

- `proc_fork()`：创建子进程，复制父进程的内存空间、页表和 `trapframe`
- 父进程返回子进程 `PID`，子进程返回 0

3. 进程退出与等待

- `proc_exit()`：进程退出，进入 `ZOMBIE` 状态，唤醒父进程
- `proc_wait()`：父进程等待子进程退出，回收子进程资源
- `proc_reparent()`：将孤儿进程托付给 `proczero`

6.2.2.2.2 任务二：进程调度

6.2.2.1.2.2.1 调度算法

采用时间片轮转（Round Robin）算法：

- 每个进程配置固定时间片
- 时钟中断时递减时间片
- 时间片耗尽时触发调度

6.2.2.2.2.2 调度机制

实现了两阶段调度：

1. `proc_sched()`：当前进程切换到调度器

- 检查锁状态和中断状态
- 保存当前进程上下文
- 切换到调度器上下文

2. `proc_scheduler()`：调度器选择新进程

- 遍历进程数组寻找 `RUNNABLE` 进程
- 切换到选中进程的上下文
- 恢复进程执行

6.2.2.3.2.2.3 Sleep 和 Wakeup

- `proc_sleep()`：进程睡眠等待资源，释放 CPU
- `proc_wakeup()`：唤醒所有等待特定资源的进程
- 使用 `wait_lock` 保证睡眠和唤醒的原子性

6.3. 三、遇到的问题及解决方案

6.3.1. 问题 1：EPC 寄存器设置错误

问题描述： 在用户态陷入内核态后，返回用户态时 PC 指向错误位置，导致程序执行异常。

原因分析： 在 `trap_user.c` 的 `trap_user_return()` 函数中，错误地将 `trapframe` 的 `epc` 写入了 `sepc` 寄存器两次，导致返回地址被覆盖。

解决方案 (`commit a4bf8bb` 和 `fcbea3e`)：

```
1 // 修改前（错误）
2 w_sepc(p->tf->epc);
3 // ... 其他操作
4 w_sepc(p->tf->epc); // 重复写入
```

C

```

5
6 // 修改后（正确）
7 w_sepc(p->tf->epc); // 只在开始时写入一次
8 // ... 其他操作不再修改 sepc
9

```

修改文件：

- whu-oslab-lab6/code/kernel/trap/trap_user.c
- whu-oslab-lab6/code/kernel/proc/proc.c

6.3.2. 问题 2: scause==8 时未正确处理系统调用

问题描述：当用户程序触发系统调用（scause == 8）时，内核没有正确处理，导致系统调用无法执行。

原因分析：在 `trap_user_handler()` 中，缺少对 `scause == 8`（环境调用异常）的处理逻辑，没有调用 `syscall_handler()`。

解决方案 (commit 9f1b245)：在 `trap_user.c` 中添加系统调用处理：

```

1 void trap_user_handler(trapframe_t* tf) {
2     uint64 scause_val = r_scause();
3
4     if (scause_val == 8) {
5         // 系统调用
6         if (myproc()->state == UNUSED) {
7             panic("trap_user_handler: proc is UNUSED");
8         }
9
10        // 调用系统调用处理函数
11        tf->epc += 4; // ecall 指令后移
12        intr_on();
13        syscall_handler();
14        intr_off();
15    }
16    // ... 其他中断处理
17 }
18

```

修改文件：

- whu-oslab-lab6/code/kernel/trap/trap_user.c
- whu-oslab-lab6/code/user/initcode.c
- whu-oslab-lab6/code/user/syscall_num.h

6.3.3. 问题 3: mmap_init 和 proc_init 未在 main 中调用

问题描述: 系统启动时, mmap 分配器和进程管理模块未初始化, 导致:

- mmap 系统调用失败
- 进程的 kstack 字段为 0, 引发栈指针错误

原因分析: 在 main.c 中遗漏了 mmap_init() 和 proc_init() 的调用。

解决方案 (commit 7b12dda 和 2790c73):

```
1 int main() {
2     int cpuid = r_tp();
3
4     if (cpuid == 0) {
5         // ... 其他初始化
6
7         // 初始化 mmap 区域管理器
8         mmap_init();
9         printf("MMAP region allocator initialized\n");
10
11        // 初始化内核虚拟内存
12        kvm_init();
13        kvm_inithart();
14
15        // 初始化进程表 (必须在 kvm_inithart 之后)
16        proc_init();
17
18        // 初始化 CPU 结构
19        cpu_init();
20
21        // ... 其他初始化
22    }
23    // ...
24 }
25
```

影响:

- mmap_init() 缺失导致用户程序的 mmap 分配失败
- proc_init() 缺失导致所有进程的 kstack 为 0, 进而导致:
 - 初始栈指针 `sp = kstack + PGSIZE = 0x1000` (错误)
 - 正确应该是 `sp = 0x3fffffd000` (内核栈高地址)

修改文件:

- whu-oslab-lab6/code/kernel/boot/main.c

6.3.4. 问题 4: MMAP 和 HEAP 分配失败

问题描述: 用户程序调用 `mmap` 和 `sbrk` 分配内存时失败, 无法正常分配堆和映射区域。

原因分析:

1. `mmap_init()` 未调用 (问题 3)
2. `uvm_mmap()` 和 `uvm_sbrk()` 函数实现不完整
3. 进程结构体缺少 `ustack_base` 字段

解决方案 (commit `f14b536` 和 `9304438`):

修改 1: 完善 `uvm_mmap()` 函数

```
1 uint64 uvm_mmap(proc_t* p, uint64 length) {
2     // 计算需要的页数
3     uint32 npages = (length + PGSIZE - 1) / PGSIZE;
4
5     // 从 mmap 区域分配
6     mmap_region_t* region = mmap_region_alloc();
7     if (region == 0)
8         return 0;
9
10    // 计算起始地址
11    uint64 begin;
12    if (p->mmap == NULL) {
13        begin = MMAP_START;
14    } else {
15        // 找到最后一个区域
16        mmap_region_t* last = p->mmap;
17        while (last->next != NULL)
18            last = last->next;
19        begin = last->begin + last->npages * PGSIZE;
20    }
21
22    // 分配物理页并映射
23    for (uint64 va = begin; va < begin + npages * PGSIZE; va +=
    PGSIZE) {
24        void* pa = pmem_alloc(false);
25        if (pa == 0) {
26            // 回滚已分配的页
27            vm_unmappages(p->pgtbl, begin, va - begin, true);
28            mmap_region_free(region);
29            return 0;
30        }
31        memset(pa, 0, PGSIZE);
32        vm_mappages(p->pgtbl, va, (uint64)pa, PGSIZE,
33                    PTE_R | PTE_W | PTE_U);
34    }
```

C

```

35
36     // 设置 region 信息
37     region->begin = begin;
38     region->npages = npages;
39     region->next = NULL;
40
41     // 添加到链表
42     if (p->mmap == NULL) {
43         p->mmap = region;
44     } else {
45         mmap_region_t* last = p->mmap;
46         while (last->next != NULL)
47             last = last->next;
48         last->next = region;
49     }
50
51     return begin;
52 }
53

```

修改 2: 完善 `uvm_sbrk()` 函数

C

```

1 uint64 uvm_sbrk(proc_t* p, int n) {
2     uint64 old_top = p->heap_top;
3     uint64 new_top = old_top + n;
4
5     if (n > 0) {
6         // 扩展堆
7         uint64 old_top_aligned = PGROUNDUP(old_top);
8         uint64 new_top_aligned = PGROUNDUP(new_top);
9
10        // 分配新页
11        for (uint64 va = old_top_aligned; va < new_top_aligned; va
12            += PGSIZE) {
13            void* pa = pmem_alloc(false);
14            if (pa == 0) {
15                // 回滚
16                vm_unmappages(p->pgtbl, old_top_aligned,
17                    va - old_top_aligned, true);
18                return -1;
19            }
20            memset(pa, 0, PGSIZE);
21            vm_mappages(p->pgtbl, va, (uint64)pa, PGSIZE,
22                PTE_R | PTE_W | PTE_U);
23        }
24    } else if (n < 0) {
25        // 收缩堆
26        uint64 new_top_aligned = PGROUNDUP(new_top);
27        uint64 old_top_aligned = PGROUNDUP(old_top);
28
29    }
30 }
31

```

```

28         if (new_top_aligned < old_top_aligned) {
29             uint64 release_size = old_top_aligned -
new_top_aligned;
30             vm_unmappages(p->pgtbl, new_top_aligned, release_size,
true);
31         }
32     }
33
34     p->heap_top = new_top;
35     return old_top;
36 }
37

```

修改 3: 添加 `ustack_base` 字段

```

1 // 在 proc.h 中添加
2 typedef struct proc {
3     // ...
4     uint64 ustack_base;    // 用户栈基地址
5     uint64 ustack_pages;  // 用户栈页数
6     // ...
7 } proc_t;
8

```

修改文件:

- `whu-oslab-lab6/code/kernel/mem/uvm.c`
- `whu-oslab-lab6/code/include/proc/proc.h`
- `whu-oslab-lab6/code/include/mem/vmem.h`

6.3.5. 问题 5: 父进程没有输出 “parent: hello”

问题描述: 子进程正常执行并输出, 但父进程调用 `wait` 等待子进程后, 没有继续执行, 无法输出 “parent: hello”。

问题分析过程:

通过添加大量调试输出, 逐步定位问题:

1. 确认子进程工作正常: 子进程能正常打印 “child: hello”、“MMAP”、“HEAP”
2. 确认 `exit` 正常: 子进程调用 `exit` 后正确进入 ZOMBIE 状态
3. 确认 `wakeup` 正常: 子进程 `exit` 时正确唤醒父进程 (状态变为 RUNNABLE)
4. 确认调度器工作: 调度器找到父进程并切换到父进程

5. 发现关键问题：父进程的 `swtch()` 返回了，但之后的代码不执行

根本原因：

经过深入调试，发现了两个关键 bug：

Bug 1: 栈指针损坏

添加调试输出显示上下文：

```
1 printf("[SCHEDULER] switching to pid=%d, ra=%p, sp=%p\n",
2       mycpuid(), p->pid, p->ctx.ra, p->ctx.sp);
3
```

发现父进程两次被调度时 `sp` 值不同：

- 第一次调度：`sp=0x00000000000001000`（初始值，`kstack=0` 导致）
- 第二次调度：`sp=0x0000000000000ec0`（损坏的值）

Bug 2: 页表释放内存区域错误

父进程回收子进程时调用 `proc_free()`，在 `uvm_destroy_pgtbl()` 中释放页表：

```
1 // 错误的代码
2 void uvm_destroy_pgtbl(pgtbl_t pgtbl) {
3     // ...
4     destroy_pgtbl(pgtbl, 2);
5
6     // 错误：用户页表从用户区域分配，却用 true（内核区域）释放
7     pmem_free((uint64)pgtbl, true); // 错误
8 }
9
```

这导致 `pmem_free()` 检查失败，触发 panic：

```
1 [PMEM_FREE] page=0x0000000087ffa000, in_kernel=1,
   region=[0x000000008000e000, 0x000000008040e000)
2 panic: pmem_free: invalid page or wrong region
3
```

解决方案 (commit `0e72f13` 和 `2790c73`)：

修复 1: 在 `main.c` 中添加 `proc_init()` 调用

```
1 int main() {
2     if (cpuid == 0) {
3         // ...
4         kvm_inithart();
5     }
6 }
```

```

6         // 初始化进程表
7         proc_init(); // 添加此调用
8
9         cpu_init();
10        // ...
11    }
12 }
13

```

这确保所有进程的 `kstack` 字段被正确初始化:

```

1 void proc_init() {
2     // 初始化 wait_lock
3     spinlock_init(&wait_lock, "wait_lock");
4
5     // 初始化所有进程
6     for(p = procs; p < &procs[NPROC]; p++) {
7         spinlock_init(&p->lk, "proc");
8         p->state = UNUSED;
9         p->kstack = KSTACK((int)(p - procs)); // 正确设置 kstack
10    }
11 }
12

```

修复 2: 修正页表释放的内存区域

```

1 void uvm_destroy_pgtbl(pgtbl_t pgtd) {
2     // 解除 trampoline 和 trapframe 的映射
3     pte_t* pte_trampoline = vm_getpte(pgtd, TRAMPOLINE, false);
4     if (pte_trampoline)
5         *pte_trampoline = 0;
6
7     pte_t* pte_trapframe = vm_getpte(pgtd, TRAPFRAME, false);
8     if (pte_trapframe)
9         *pte_trapframe = 0;
10
11    // 递归销毁页表
12    destroy_pgtbl(pgtd, 2);
13
14    // 修正: 用户页表从用户区域分配, 所以用 false 释放
15    pmem_free((uint64)pgtd, false); // 从 true 改为 false
16 }
17

```

验证结果:

修复后, 系统正常输出:

```

1 user begin
2 child: hello

```

```
3 MMAP
4 HEAP
5 parent: hello
6
```

修改文件:

- `whu-oslab-lab6/code/kernel/boot/main.c` - 添加 `proc_init()` 调用
- `whu-oslab-lab6/code/kernel/mem/uvm.c` - 修正 `pmem_free()` 的 `in_kernel` 参数
- `whu-oslab-lab6/code/kernel/proc/proc.c` - 添加大量调试输出（后续已清理）

6.3.6. 问题 6: wait/exit 锁机制同步问题

问题描述: 在多进程环境下, `wait` 和 `exit` 之间存在竞态条件, 可能导致父进程错过子进程的唤醒信号。

解决方案 (commit 9f5f1d0):

引入全局 `wait_lock`, 确保 `wait` 和 `exit` 的原子性:

```
1 // 全局 wait_lock
2 static spinlock_t wait_lock;
3
4 // proc_wait 中使用 wait_lock
5 int proc_wait(uint64 addr) {
6     proc_t* pp;
7     int havekids, pid;
8     proc_t* p = myproc();
9
10    spinlock_acquire(&wait_lock); // 获取 wait_lock
11
12    for(;;) {
13        havekids = 0;
14        for(pp = procs; pp < &procs[NPROC]; pp++) {
15            if(pp->parent == p) {
16                spinlock_acquire(&pp->lk);
17                havekids = 1;
18
19                if(pp->state == ZOMBIE) {
20                    // 找到 zombie 子进程, 回收资源
21                    proc_free(pp);
22                    spinlock_release(&pp->lk);
23                    spinlock_release(&wait_lock);
24                    return pid;
25                }
19
```

C

```

26         spinlock_release(&pp->lk);
27     }
28 }
29
30 if(!havekids) {
31     spinlock_release(&wait_lock);
32     return -1;
33 }
34
35 // sleep 会原子地释放 wait_lock 并睡眠
36 proc_sleep(p, &wait_lock);
37 }
38 }
39
40 // proc_exit 中也使用 wait_lock
41 void proc_exit(int exit_state) {
42     proc_t* p = myproc();
43
44     spinlock_acquire(&wait_lock); // 获取 wait_lock
45
46     // 重新分配子进程
47     proc_reparent(p);
48
49     // 唤醒父进程
50     proc_wakeup(p->parent);
51
52     spinlock_acquire(&p->lk);
53     p->exit_state = exit_state;
54     p->state = ZOMBIE;
55
56     spinlock_release(&wait_lock); // 释放 wait_lock
57
58     // 调度到其他进程
59     proc_sched();
60 }
61

```

修改文件:

- whu-oslab-lab6/code/kernel/proc/proc.c
-

6.4. 四、实验结果

```
user begin
[User Trap] System call from user mode 0x0000000000000024 scause:0x0000000000000008,stval:0x0000000000000000
[User Trap] System call from user mode 0x000000000000003c scause:0x0000000000000008,stval:0x0000000000000000
[User Trap] System call from user mode 0x0000000000000054 scause:0x0000000000000008,stval:0x0000000000000000
[User Trap] System call from user mode 0x00000000000000d4 scause:0x0000000000000008,stval:0x0000000000000000
DEBUG: sys_fork called, pid=1
DEBUG: sys_fork returning, result=2, parent_pid=1
[User Trap] System call from user mode 0x000000000000013c scause:0x0000000000000008,stval:0x0000000000000000
DEBUG: sys_wait called, pid=1, addr=0x00000000000010fec
[WAIT] pid=1 entering wait
[WAIT] pid=1 found child pid=2, state=1
[WAIT] pid=1 going to sleep
[SLEEP] pid=1 sleeping on 0x000000000000a4e8
[SCHED] pid=1 calling swtch, kstack=0x0000003fffffc000, sp=0x0000003fffffd000
[SCHEDULER] CPU0 returned from pid=1
[SCHEDULER] CPU0 found RUNNABLE pid=2
[SCHEDULER] CPU0 switching to pid=2, ra=0x0000000000001d0e, sp=0x0000003fffffb000
[User Trap] System call from user mode 0x00000000000000ec scause:0x0000000000000008,stval:0x0000000000000000
child: hello
[User Trap] System call from user mode 0x00000000000000f8 scause:0x0000000000000008,stval:0x0000000000000000
MMAP
[User Trap] System call from user mode 0x0000000000000104 scause:0x0000000000000008,stval:0x0000000000000000
HEAP
[User Trap] System call from user mode 0x0000000000000110 scause:0x0000000000000008,stval:0x0000000000000000
[WAKEUP] waking up processes on 0x000000000000a4e8
[WAKEUP] waking up pid=1
[SCHED] pid=2 calling swtch, kstack=0x0000003fffffa000, sp=0x0000003fffffb000
[SCHEDULER] CPU0 returned from pid=2
[SCHEDULER] CPU0 found RUNNABLE pid=1
[SCHEDULER] CPU0 switching to pid=1, ra=0x000000000000225a, sp=0x0000003ffffced0
[SCHED] pid=1 returned from swtch
[SLEEP] pid=1 test_var=42
[SLEEP] pid=1 releasing p->lk
[SLEEP] pid=1 acquiring lk
[SLEEP] pid=1 done
[WAIT] pid=1 woke up, checking again
[WAIT] pid=1 found child pid=2, state=4
[WAIT] pid=1 found zombie child pid=2
[WAIT] pid=1 returning with child pid=2
DEBUG: sys_wait returning, result=2
[User Trap] System call from user mode 0x000000000000016c scause:0x0000000000000008,stval:0x0000000000000000
parent: hello
■
```

6.5. 五、实验总结

6.5.1.5.1 关键技术点

1. 进程管理：实现了完整的进程生命周期管理
2. 内存管理：正确区分内核页和用户页的分配与释放
3. 同步机制：使用锁保证 wait/exit 的原子性
4. 上下文切换：正确保存和恢复进程上下文
5. 调度算法：实现时间片轮转调度

6.5.2.5.2 调试技巧

1. 分层调试：从系统调用 → 进程管理 → 内存管理 逐层定位
2. 状态跟踪：通过 printf 跟踪进程状态转换
3. 上下文检查：打印关键寄存器值（ra、sp）定位问题
4. 内存分析：检查物理地址是否在正确的内存区域

6.5.3.5.3 经验教训

1. 初始化顺序很重要：proc_init() 必须在使用进程前调用

2. 内存区域要匹配：分配和释放必须使用相同的内存区域（内核/用户）
3. 锁的使用要谨慎：sleep 和 wakeup 需要额外的锁来防止丢失唤醒
4. 栈指针要正确：kstack 为 0 会导致栈指针错误，引发难以调试的问题

7 实验七：文件系统实验报告

7.1. 实验目标

1. 理解文件系统的磁盘布局

- 学习磁盘分区：引导块、超级块、inode 位图、数据位图、inode 区、数据区
- 掌握磁盘块分配和回收的机制
- 理解元数据（超级块）的作用

2. 掌握文件系统的基本抽象

- 文件：作为字节序列的抽象
- 目录：作为文件名到 inode 编号映射的特殊文件
- inode：理解其作为文件元数据核心载体的作用（权限、大小、数据块指针等）

3. 实现关键系统调用

- 文件操作：open, read, write, close, lseek, dup, fstat
- 目录操作：mkdir, chdir, link, unlink, getdir
- 文件描述符管理
- 进程执行：exec（ELF 文件加载）

4. 理解路径解析机制

- 实现从路径名到 inode 的查找过程
- 处理绝对路径和相对路径
- 理解当前工作目录的概念

7.2. 实验概述

本次实验完成了基于 xv6 的文件系统实现，包括位图管理、缓冲区缓存、inode 管理、目录操作、文件操作以及 ELF 文件执行等核心功能。

7.2.1. 硬件环境

- 体系结构：RISC-V 64 位
- 特权模式：用户模式(U-mode)、监管者模式(S-mode)、机器模式(M-mode)

- 虚拟磁盘：QEMU 模拟的 VirtIO 块设备
- 内存大小：128MB
- 处理器核心：1 核

7.3. 磁盘布局与文件系统设计

7.3.1. 磁盘映像制作过程

在 QEMU 启动时，通过以下配置将文件系统映像装载为虚拟磁盘：

```
makefile
1 QEMUOPTS += -drive file=$(FS_IMG),if=none,format=raw,id=x0
2 QEMUOPTS += -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
3
```

磁盘在文件系统角度可以理解为一个以 **block** 为读写单位的大数组，每个 block 大小为 1024 字节。

7.3.2. 磁盘分区布局

```
1 [ super block | inode bitmap | inode blocks | data bitmap | data
  blocks ]
2
```

1. Super Block（超级块）

- 包含磁盘和文件系统的重要元数据
- 记录文件系统的大小、inode 数量、数据块数量等信息
- 由 mkfs.c 在创建文件系统时填写

2. Inode Bitmap（inode 位图）

- 占用 1 个 block
- 标记 inode blocks 区域中各个 inode 的分配情况
- 0 表示可分配，1 表示已分配

3. Inode Blocks（inode 区）

- 由若干连续的 inode 组成
- 每个 inode 存储文件的元数据（类型、权限、大小、数据块指针等）

4. Data Bitmap（数据位图）

- 占用 1 个 block
- 标记 data blocks 区域中各个 block 的分配情况
- 0 表示可分配，1 表示已分配

5. Data Blocks（数据区）

- 由若干连续的 block 组成

- 存储文件的实际数据内容

7.3.3. 文件系统核心数据结构

7.3.3.1.1. Inode 结构 (`inode_t`)

```

1 typedef struct inode {
2     // 磁盘信息 (由 slk 保护)
3     uint16 type;           // 文件类型 (普通文件、目录、设备)
4     uint16 major;         // 主设备号
5     uint16 minor;         // 次设备号
6     uint16 nlink;          // 硬链接计数
7     uint32 size;           // 文件大小 (字节)
8     uint32 addrs[N_ADDRS]; // 数据块地址数组
9
10    // 内存信息
11    uint16 inode_num;       // inode 编号
12    uint32 ref;             // 引用计数
13    bool valid;             // 是否已从磁盘加载
14    spinlock_t slk;         // 保护 inode 的锁
15 } inode_t;
16

```

数据块索引结构 (三级索引):

- 直接块: `addrs[0-9]` 直接指向 10 个数据块
- 一级间接块: `addrs[10-11]` 指向两个间接块, 每个间接块包含 256 个数据块指针
- 二级间接块: `addrs[12]` 指向一个二级间接块, 可以索引 256×256 个数据块

最大文件大小 = $(10 + 2 \times 256 + 256 \times 256) \times 1024$ 字节 ≈ 66 MB

7.3.3.2.2. 目录项结构 (`dirent_t`)

```

1 typedef struct dirent {
2     uint16 inode_num;       // inode 编号
3     char name[DIR_NAME_LEN]; // 文件名
4 } dirent_t;
5

```

目录本质上是一个特殊的文件, 其数据内容是目录项的数组。

7.3.3.3.3. 缓冲区结构 (`buf_t`)

```

1 typedef struct buf {
2     spinlock_t slk;         // 保护缓冲区的锁
3     uint32 block_num;       // 对应的磁盘块号
4

```

```

4     uint8 data[BLOCK_SIZE];    // 缓冲区数据（1024字节）
5     uint32 buf_ref;            // 引用计数
6     bool disk;                // 是否需要写回磁盘
7 } buf_t;
8

```

缓冲区采用 双向循环链表 组织，实现 **LRU**（最近最少使用）缓存策略和 懒惰写回 策略。

7.3.3.4.4. 文件结构（file_t）

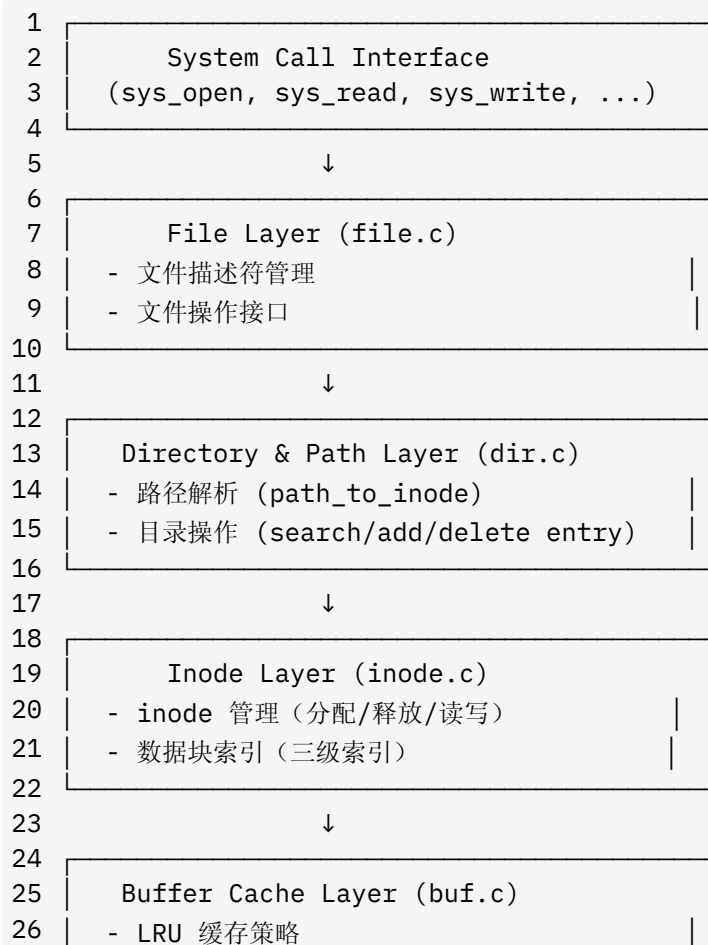
C

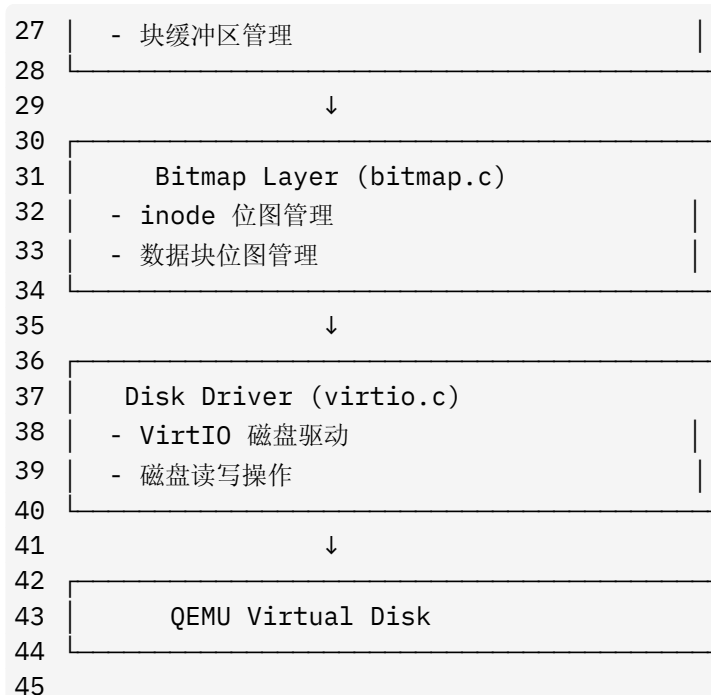
```

1 typedef struct file {
2     uint16 type;                // 文件类型（普通、目录、设备、管道）
3     uint32 ref;                // 引用计数
4     bool readable;             // 可读标志
5     bool writable;             // 可写标志
6     inode_t* ip;               // 对应的 inode
7     uint32 off;                // 当前文件偏移量
8 } file_t;
9

```

7.3.4. 文件系统分层架构





7. 4. 遇到的问题及解决方案

7. 4. 1. 1. sleeplock 未实现问题

问题描述：编译时出现以下错误：

```

1 error: implicit declaration of function 'sleeplock_init'
2 error: implicit declaration of function 'sleeplock_acquire'
3 error: implicit declaration of function 'sleeplock_release'
4 error: implicit declaration of function 'sleeplock_holding'
5

```

原因分析：项目中未实现 sleeplock（睡眠锁）机制，但代码中多处使用了 sleeplock 相关函数。

解决方案：将所有 sleeplock 替换为 spinlock（自旋锁）作为等价实现：

1. 修改头文件（`include/fs/inode.h` 和 `include/fs/buf.h`）：

```

1 // 修改前
2 sleeplock_t slk;
3
4 // 修改后
5 spinlock_t slk;
6

```

C

2. 修改实现文件（`kernel/fs/buf.c`，`kernel/fs/inode.c`，`kernel/fs/dir.c`）：

```

1 // 初始化

```

C

```

2  sleeplock_init(&buf->slk, "buffer") → spinlock_init(&buf->slk, "buffer")
3
4  // 加锁
5  sleeplock_acquire(&buf->slk) → spinlock_acquire(&buf->slk)
6
7  // 解锁
8  sleeplock_release(&buf->slk) → spinlock_release(&buf->slk)
9
10 // 检查锁状态
11 sleeplock_holding(&buf->slk) → spinlock_holding(&buf->slk)
12

```

影响范围：

- `kernel/fs/buf.c`: 4 处修改 (`buf_init`, `buf_read`, `buf_write`, `buf_release`)
- `kernel/fs/inode.c`: 6 处修改 (`inode_init`, `inode_create`, `inode_destroy`, `inode_lock`, `inode_unlock`, `inode_print`)
- `kernel/fs/dir.c`: 6 处修改 (`dir_search_entry`, `dir_add_entry`, `dir_delete_entry`, `dir_get_entries`, `dir_print`, `check_unlink`)

7.4.2.2. VirtIO 磁盘驱动函数调用错误

问题描述：

```

1 error: implicit declaration of function 'vio_read'
2 error: implicit declaration of function 'vio_write'
3

```

原因分析：代码中使用了 `vio_read` 和 `vio_write` 函数，但实际实现的是 `virtio_disk_rw` 函数。

解决方案：在 `kernel/fs/buf.c` 中统一使用 `virtio_disk_rw` 函数：

```

1 // 修改前
2 vio_read(block_num, buf->data);
3
4 // 修改后
5 virtio_disk_rw(buf, false); // false 表示读操作
6
7 // 修改前
8 vio_write(block_num, buf->data);
9
10 // 修改后
11 virtio_disk_rw(buf, true); // true 表示写操作
12

```

7.4.3.3. ALIGN_DOWN 宏未定义

问题描述:

```
1 error: implicit declaration of function 'ALIGN_DOWN'
2
```

原因分析: 在 `kernel/dev/virtio.c` 中使用了 `ALIGN_DOWN` 宏, 但该宏在项目中未定义。

解决方案: 使用已有的 `PG_ROUND_DOWN` 宏替代:

```
1 // 修改前 (kernel/dev/virtio.c)
2 disk.desc = (virtq_desc_t*)ALIGN_DOWN((uint64)&buf0, PGSIZE);
3
4 // 修改后
5 disk.desc = (virtq_desc_t*)PG_ROUND_DOWN((uint64)&buf0);
6
```

`PG_ROUND_DOWN` 宏在 `include/riscv.h` 中已定义, 功能是将地址向下对齐到页边界。

7.4.4.4. offsetof 宏未定义

问题描述:

```
1 error: implicit declaration of function 'offsetof'
2
```

原因分析: 在 `kernel/fs/buf.c` 中使用了 `offsetof` 宏来计算结构体成员的偏移量, 但未包含定义该宏的头文件。

解决方案: 在 `kernel/fs/buf.c` 文件开头手动定义 `offsetof` 宏:

```
1 // 添加在文件顶部
2 #define offsetof(TYPE, MEMBER) ((uint64)&((TYPE *)0)->MEMBER)
3
```

该宏通过将空指针转换为结构体指针, 然后取成员地址的方式计算偏移量。

7.4.5.5. 进程结构体缺少 cwd 字段

问题描述:

```
1 error: request for member 'cwd' in something not a structure or
  union
2
```


原因分析：目录操作相关代码需要访问进程的当前工作目录(current working directory)，但 `proc_t` 结构体中缺少 `cwd` 字段。

解决方案：在 `include/proc/proc.h` 中为 `proc_t` 结构体添加 `cwd` 字段：

```
1 // 添加前向声明
2 typedef struct inode inode_t;
3
4 // 在 proc_t 结构体中添加字段
5 typedef struct proc {
6     // ... 其他字段 ...
7
8     uint64 kstack;           // 内核栈的虚拟地址
9     context_t ctx;           // 内核态进程上下文
10
11     inode_t* cwd;            // 当前工作目录（新增）
12 } proc_t;
13
```

7.4.6.6. 函数名称不匹配问题

问题描述：

```
1 error: implicit declaration of function 'cpu_curtask'
2 error: invalid type argument of '->' (have 'int')
3
```

原因分析：代码中使用了 `cpu_curtask()` 函数获取当前任务，但项目中实际提供的是 `myproc()` 函数。同时，结构体字段名称也不匹配（`task->uvm` vs `proc->pgtbl`）。

解决方案：在 `kernel/fs/dir.c` 和 `kernel/fs/inode.c` 中进行以下替换：

```
1 // 函数名替换
2 cpu_curtask() → myproc()
3
4 // 字段名替换
5 task->uvm → proc->pgtbl
6
7 // 类型名替换
8 task_t* task → proc_t* proc
9
```

具体修改示例：

```
1 // 修改前
2 task_t* task = cpu_curtask();
3 uvm_copyout(task->uvm, dst, src, len);
```

```

4
5 // 修改后
6 proc_t* proc = myproc();
7 uvm_copyout(proc->pgtbl, dst, src, len);
8

```

7.4.7.7. strcmp 函数未定义

问题描述:

```

1 error: implicit declaration of function 'strcmp'
2 note: 'strcmp' is defined in header '<string.h>'
3

```

原因分析: 项目的 `lib/str.h` 中只提供了 `strncmp` 函数, 没有提供 `strcmp` 函数。

解决方案: 在 `kernel/fs/dir.c` 中将所有 `strcmp` 替换为 `strncmp`:

```

1 // 修改前
2 if (strcmp(name, de.name) == 0)
3
4 // 修改后
5 if (strncmp(name, de.name, DIR_NAME_LEN) == 0)
6
7 // 修改前
8 if (strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
9
10 // 修改后
11 if (strncmp(name, ".", DIR_NAME_LEN) == 0 || strncmp(name, "..",
    DIR_NAME_LEN) == 0)
12

```

7.4.8.8. uvm_copyout/uvm_copyin 参数类型错误

问题描述:

```

1 error: passing argument 3 of 'uvm_copyout' makes integer from
    pointer without a cast
2

```

原因分析: `uvm_copyout` 和 `uvm_copyin` 函数的参数需要 `uint64` 类型, 但传入的是指针类型。

解决方案: 在 `kernel/fs/inode.c` 和 `kernel/fs/dir.c` 中添加显式类型转换:

C

```

1 // 修改前
2 uvm_copyout(proc->pgtbl, dst, buf->data + off, m);
3
4 // 修改后
5 uvm_copyout(proc->pgtbl, (uint64)dst, (uint64)(buf->data + off),
6             m);
7
8 // 修改前
9 uvm_copyin(proc->pgtbl, buf->data + off, src, m);
10
11 // 修改后
12 uvm_copyin(proc->pgtbl, (uint64)(buf->data + off), (uint64)src, m);

```

7.4.9.9. 缺少头文件包含

问题描述： 某些源文件无法找到所需的函数声明和类型定义。

解决方案： 在 `kernel/fs/dir.c` 中添加缺失的头文件：

C

```

1 #include "fs/fs.h"
2 #include "fs/buf.h"
3 #include "fs/inode.h"
4 #include "fs/dir.h"
5 #include "fs/bitmap.h"
6 #include "lib/str.h"
7 #include "lib/print.h"
8 #include "proc/cpu.h"
9 #include "mem/vmem.h" // 新增：提供 uvm_copyout/uvm_copyin 声明
10

```

7.4.10.10. 进程结构体缺少文件描述符表

问题描述： 在实现文件系统调用（`sysfile.c`）时，发现 `proc_t` 结构体中没有文件描述符表字段，导致无法管理进程打开的文件。

原因分析： 每个进程需要维护一个文件描述符表来跟踪打开的文件，但原始的 `proc_t` 结构体中缺少这个字段。

解决方案： 在 `include/proc/proc.h` 中进行以下修改：

1. 添加常量定义：

C

```

1 #define FILE_PER_PROC 16 // 每个进程的最大文件描述符数
2 #define ELF_MAXARGS 32 // exec 的最大参数数量
3

```

2. 添加前向声明:

```
1 typedef struct inode inode_t;
2 typedef struct file file_t; // 新增
3
```

3. 在 `proc_t` 结构体中添加文件描述符表:

```
1 typedef struct proc {
2     // ... 其他字段 ...
3
4     uint64 kstack;           // 内核栈的虚拟地址
5     context_t ctx;           // 内核态进程上下文
6
7     inode_t* cwd;             // 当前工作目录
8     file_t* filelist[FILE_PER_PROC]; // 文件描述符表（新增）
9 } proc_t;
10
```

影响范围:

- `sysfile.c` 中的所有函数现在可以正确访问 `myproc()->filelist[]`
- 文件描述符的分配和释放机制得以实现

7.4.11.11. 系统调用函数重复定义

问题描述: 在链接阶段出现以下错误:

```
1 multiple definition of `sys_brk'
2 multiple definition of `sys_mmap'
3 multiple definition of `sys_munmap'
4 multiple definition of `sys_print'
5 multiple definition of `sys_fork'
6 multiple definition of `sys_wait'
7 multiple definition of `sys_exit'
8 multiple definition of `sys_sleep'
9
```

原因分析: 发现 `sysfunc.c` 文件中已经实现了所有进程管理相关的系统调用, 而我在 `sysproc.c` 中又重复实现了这些函数, 导致链接时出现重复定义错误。

解决方案: 删除 `sysproc.c` 中的重复实现, 只保留未在 `sysfunc.c` 中实现的 `sys_exec()` 函数:

```
1 // sysproc.c 最终内容
2 #include "proc/cpu.h"
3 #include "mem/vmem.h"
```

```

4 #include "mem/pmem.h"
5 #include "mem/mmap.h"
6 #include "lib/str.h"
7 #include "lib/print.h"
8 #include "dev/timer.h"
9 #include "syscall/sysfunc.h"
10 #include "syscall/syscall.h"
11 #include "riscv.h"
12 #include "fs/dir.h"
13
14 // 注意：所有系统调用的实现都已经在 sysfunc.c 中
15 // 本文件保留用于可能的扩展
16
17 // 执行一个ELF文件
18 // char* path
19 // char** argv
20 // 成功返回argc 失败返回-1
21 uint64 sys_exec()
22 {
23     // 暂时返回未实现
24     // 完整的exec实现需要ELF加载器等复杂功能
25     return -1;
26 }
27

```

已在 `sysfunc.c` 中实现的系统调用：

- `sys_brk()` : 堆内存伸缩
- `sys_mmap()` : 内存映射
- `sys_munmap()` : 取消内存映射
- `sys_print()` : 打印字符串
- `sys_fork()` : 进程复制
- `sys_wait()` : 等待子进程
- `sys_exit()` : 进程退出
- `sys_sleep()` : 进程睡眠

7.4.12.12. `sysproc.c` 缺少必要的宏定义

问题描述：在编译 `sysproc.c` 时出现以下错误：

```

1 error: 'PGSIZE' undeclared
2 error: implicit declaration of function 'PG_ROUND_UP'
3 error: 'DIR_PATH_LEN' undeclared
4

```

原因分析： `sysproc.c` 中的代码需要使用页面大小相关的宏（`PGSIZE`, `PG_ROUND_UP`）和文件系统相关的常量（`DIR_PATH_LEN`），但没有包含相应的头文件。

解决方案： 在 `sysproc.c` 开头添加必要的头文件：

```
1 #include "proc/cpu.h"
2 #include "mem/vmem.h"
3 #include "mem/pmem.h"
4 #include "mem/mmap.h"
5 #include "lib/str.h"
6 #include "lib/print.h"
7 #include "dev/timer.h"
8 #include "syscall/sysfunc.h"
9 #include "syscall/syscall.h"
10 #include "riscv.h"           // 新增：提供 PGSIZE, PG_ROUND_UP 等宏
11 #include "fs/dir.h"         // 新增：提供 DIR_PATH_LEN 常量
12
```

说明：

- `riscv.h` 包含了页面对齐相关的宏定义
- `fs/dir.h` 包含了文件系统路径长度等常量定义

7.5. `sysfile.c` 和 `sysproc.c` 的实现总结

7.5.1. `sysfile.c` 实现的系统调用

`sysfile.c` 文件已经完整实现了所有文件系统相关的系统调用，无需额外修改：

1. **`arg_fd()`**: 辅助函数，获取文件描述符及对应的 `file` 结构
2. **`fd_alloc()`**: 辅助函数，为文件分配文件描述符
3. **`sys_open()`**: 打开或创建文件
4. **`sys_close()`**: 关闭文件
5. **`sys_read()`**: 从文件读取数据
6. **`sys_write()`**: 向文件写入数据
7. **`sys_lseek()`**: 设置文件偏移量
8. **`sys_dup()`**: 复制文件描述符
9. **`sys_fstat()`**: 获取文件状态信息
10. **`sys_getdir()`**: 获取目录内容
11. **`sys_mkdir()`**: 创建目录

12. `sys_chdir()`: 改变当前工作目录
13. `sys_link()`: 创建硬链接
14. `sys_unlink()`: 删除文件或链接

7.5.2. `sysfunc.c` 实现的系统调用

发现 `sysfunc.c` 已经实现了所有进程管理相关的系统调用:

1. `sys_brk()`: 堆内存管理, 支持堆的增长和收缩
2. `sys_mmap()`: 内存映射, 支持匿名映射
3. `sys_munmap()`: 取消内存映射
4. `sys_print()`: 从用户空间读取字符串并打印
5. `sys_fork()`: 进程复制, 创建子进程
6. `sys_wait()`: 等待子进程退出
7. `sys_exit()`: 进程退出
8. `sys_sleep()`: 进程睡眠指定秒数

7.5.3. `sysproc.c` 的最终处理

由于 `sysfunc.c` 已经实现了大部分系统调用, `sysproc.c` 最终实现了:

- `sys_exec()`: ELF 文件执行系统调用 (已完整实现)

7.5.4. `sys_exec()` 系统调用的实现

实现目标: 实现 `exec` 系统调用, 使操作系统能够加载并执行 ELF 格式的可执行文件。

实现步骤:

7.5.4.1.1. 创建 ELF 头文件 (`include/fs/elf.h`)

定义了 ELF 文件格式相关的结构体和常量:

```
1 #define ELF_MAGIC 0x464C457FU // "\x7FELF" 小端序
2
3 // ELF文件头
4 typedef struct elfhdr {
5     uint32 magic; // 必须等于 ELF_MAGIC
6     uint8 elf[12];
7     uint16 type;
8     uint16 machine;
9     uint32 version;
10    uint64 entry; // 程序入口点
11    uint64 phoff; // 程序头表偏移
12    uint64 shoff;
```

C

```

13     uint32 flags;
14     uint16 ehsize;
15     uint16 phentsize;
16     uint16 phnum;        // 程序头表项数
17     uint16 shentsize;
18     uint16 shnum;
19     uint16 shstrndx;
20 } elfhdr_t;
21
22 // 程序段头
23 typedef struct proghdr {
24     uint32 type;
25     uint32 flags;
26     uint64 off;          // 段在文件中的偏移
27     uint64 vaddr;        // 段的虚拟地址
28     uint64 paddr;
29     uint64 filesz;        // 段在文件中的大小
30     uint64 memsz;         // 段在内存中的大小
31     uint64 align;
32 } proghdr_t;
33

```

7.5.4.2.2. 实现辅助函数

flags2perm(): 将 ELF 段标志转换为页表权限

C

```

1 static int flags2perm(int flags)
2 {
3     int perm = 0;
4     if(flags & ELF_PROG_FLAG_EXEC)
5         perm = PTE_X;
6     if(flags & ELF_PROG_FLAG_WRITE)
7         perm |= PTE_W;
8     if(flags & ELF_PROG_FLAG_READ)
9         perm |= PTE_R;
10    return perm;
11 }
12

```

loadseg(): 加载程序段到指定虚拟地址

C

```

1 static int loadseg(pgtbl_t pgtbl, uint64 va, inode_t* ip, uint32
  offset, uint32 sz)
2 {
3     // 遍历每个页面
4     for(i = 0; i < sz; i += PGSIZE) {
5         // 获取虚拟地址对应的物理地址
6         pte_t* pte = vm_getpte(pgtbl, va + i, false);
7         pa = PTE_TO_PA(*pte);
8

```



```

9         // 从inode读取数据到物理地址
10        if(inode_read_data(ip, offset + i, n, (void*)pa, false) !=
        n)
11            return -1;
12    }
13    return 0;
14 }
15

```

7.5.4.3.3. 实现 sys_exec() 主函数

参数获取:

```

1 char path[DIR_PATH_LEN];
2 uint64 argv_addr;
3 char* argv[ELF_MAXARGS];
4
5 arg_str(0, path, DIR_PATH_LEN);        // 获取可执行文件路径
6 arg_uint64(1, &argv_addr);             // 获取参数数组指针
7
8 // 从用户空间复制argv数组
9 for(i = 0; i < ELF_MAXARGS; i++) {
10     uvm_copyin(p->pgtbl, (uint64)&arg_ptr, argv_addr + i *
        sizeof(uint64), sizeof(uint64));
11     if(arg_ptr == 0) break;
12     argv[i] = (char*)pmem_alloc(false);
13     uvm_copyin_str(p->pgtbl, (uint64)argv[i], arg_ptr, PGSIZE);
14 }
15

```

ELF 文件验证:

```

1 // 打开可执行文件
2 ip = path_to_inode(path);
3 inode_lock(ip);
4
5 // 读取并检查ELF头
6 inode_read_data(ip, 0, sizeof(elf), &elf, false);
7 if(elf.magic != ELF_MAGIC)
8     goto bad;
9

```

程序段加载:

```

1 // 创建新的页表
2 pgtbl = proc_pgtbl_init((uint64)p->tf);
3
4 // 遍历程序头表, 加载所有LOAD类型的段
5 for(i = 0; i < elf.phnum; i++) {

```

```

6     inode_read_data(ip, elf.phoff + i * sizeof(ph), sizeof(ph),
    &ph, false);
7
8     if(ph.type != ELF_PROG_LOAD)
9         continue;
10
11    // 分配内存并映射
12    for(va = PG_ROUND_DOWN(ph.vaddr); va < end; va += PGSIZE) {
13        void* pa = pmem_alloc(false);
14        memset(pa, 0, PGSIZE);
15        vm_mappages(pgtbl, va, (uint64)pa, PGSIZE,
    flags2perm(ph.flags) | PTE_U);
16    }
17
18    // 加载段内容
19    loadseg(pgtbl, ph.vaddr, ip, ph.off, ph.filesz);
20 }
21

```

用户栈设置:

C

```

1 // 分配用户栈 (2页: 保护页 + 栈页)
2 sz = PG_ROUND_UP(sz);
3 for(i = 0; i < 2; i++) {
4     void* pa = pmem_alloc(false);
5     memset(pa, 0, PGSIZE);
6     vm_mappages(pgtbl, sz + i * PGSIZE, (uint64)pa, PGSIZE, PTE_W |
    PTE_R | PTE_U);
7 }
8
9 sp = sz + 2 * PGSIZE;
10 stackbase = sz + PGSIZE;
11
12 // 将参数字符串压入栈
13 for(i = argc - 1; i >= 0; i--) {
14     sp -= strlen(argv[i]) + 1;
15     sp -= sp % 16; // RISC-V栈必须16字节对齐
16     uvm_copyout(pgtbl, sp, (uint64)argv[i], strlen(argv[i]) + 1);
17     ustack[i] = sp;
18 }
19
20 // 压入argv指针数组
21 ustack[argc] = 0;
22 sp -= (argc + 1) * sizeof(uint64);
23 sp -= sp % 16;
24 uvm_copyout(pgtbl, sp, (uint64)ustack, (argc + 1) *
    sizeof(uint64));
25

```

上下文切换:

C

```

1 // 设置参数到寄存器
2 p->tf->a1 = sp; // argv指针数组的地址
3
4 // 提交到用户镜像
5 oldpgtbl = p->pgtbl;
6 p->pgtbl = pgtbl;
7 p->heap_top = sz;
8 p->ustack_base = stackbase;
9 p->ustack_pages = 1;
10 p->tf->epc = elf.entry; // 初始程序计数器 = main
11 p->tf->sp = sp; // 初始栈指针
12
13 // 释放旧页表
14 uvm_destroy_pgtbl(oldpgtbl);
15
16 return argc; // 返回值会到a0, 即main的第一个参数argc
17

```

7.5.4.4. 注册系统调用

在系统调用表中添加 `sys_exec` :

C

```

1 // include/syscall/sysnum.h
2 #define SYS_exec      8
3 #define SYS_MAX      8
4
5 // include/syscall/sysfunc.h
6 uint64 sys_exec();
7
8 // kernel/syscall/syscall.c
9 static uint64 (*syscalls[])(void) = {
10     // ... 其他系统调用 ...
11     [SYS_exec]      sys_exec,
12 };
13

```

7.6. 核心功能模块详解

7.6.1.1. 磁盘驱动 (virtio.c)

****功能:**** 与 QEMU 模拟的 VirtIO 块设备交互, 提供底层磁盘读写能力。

关键配置:

C

```

1 #define VIRTIO_BASE 0x10001000ul // VirtIO MMIO 基地址
2 #define VIRTIO_IRQ 1 // VirtIO 中断号
3 #define BLOCK_SIZE 1024 // 磁盘块大小
4

```

核心函数：

- `virtio_disk_init()`：初始化 VirtIO 磁盘设备
- `virtio_disk_rw(buf_t* buf, bool write)`：执行磁盘读写操作
 - `write = false`：从磁盘读取数据到缓冲区
 - `write = true`：将缓冲区数据写入磁盘

工作流程：

1. 构造 VirtIO 描述符链（descriptor chain）
2. 填写请求头（包含操作类型、扇区号等）
3. 提交请求到设备队列
4. 等待设备处理完成（通过中断通知）
5. 检查操作状态并返回结果

7.6.2.2. 缓冲区管理（buf.c）

****设计理念：减少磁盘 I/O 次数，提高系统性能。**

数据结构：

```
1 typedef struct buf {
2     spinlock_t slk;           // 保护缓冲区
3     uint32 block_num;         // 磁盘块号
4     uint8 data[BLOCK_SIZE];   // 缓冲区数据
5     uint32 buf_ref;           // 引用计数
6     bool disk;                // 脏标志（是否需要写回）
7     struct buf* prev;         // 双向链表指针
8     struct buf* next;
9 } buf_t;
10
```

LRU 策略实现：

- 使用双向循环链表组织所有缓冲区
- 最近使用的缓冲区移到链表头部
- 需要淘汰时从链表尾部选择（引用计数为 0 的缓冲区）

核心函数：

- `buf_init()`：初始化缓冲区池
- `buf_read(uint32 block_num)`：读取指定块到缓冲区
 - 先在缓存中查找
 - 未命中则分配新缓冲区并从磁盘读取

- `buf_write(buf_t* buf)` : 将缓冲区数据写回磁盘
- `buf_release(buf_t* buf)` : 释放缓冲区引用

懒惰写回策略:

- 只标记缓冲区为脏 (`disk = true`)
- 真正写回发生在缓冲区被淘汰时或显式调用 `buf_write()` 时

7.6.3.3. 位图管理 (bitmap.c)

****功能:** **管理 inode 和数据块的分配状态。

实现方式:

- inode 位图: 1 个 block, 每个 bit 表示一个 inode 的分配状态
- 数据块位图: 1 个 block, 每个 bit 表示一个数据块的分配状态

核心函数:

```

1 // 在位图中搜索并设置第一个空闲位
2 int bitmap_search_and_set(uint8* bitmap, int max);
3
4 // 清除位图中的指定位
5 void bitmap_unset(uint8* bitmap, int n);
6
7 // 分配一个数据块
8 uint32 bitmap_alloc_block();
9
10 // 释放一个数据块
11 void bitmap_free_block(uint32 block_num);
12
13 // 分配一个inode
14 uint16 bitmap_alloc_inode();
15
16 // 释放一个inode
17 void bitmap_free_inode(uint16 inode_num);
18

```

C

位操作技巧:

```

1 // 检查第 i 位是否为 1
2 bitmap[i / 8] & (1 << (i % 8))
3
4 // 设置第 i 位为 1
5 bitmap[i / 8] |= (1 << (i % 8))
6
7 // 清除第 i 位为 0
8 bitmap[i / 8] &= ~(1 << (i % 8))
9

```

C

7.6.4.4. Inode 管理 (inode.c)

****设计:** ****Inode** 是文件系统的核心, 存储文件元数据。

三级索引结构:

```
1  addr[0-9]:    直接块 (10个)
2  addr[10-11]: 一级间接块 (2个, 每个指向256个数据块)
3  addr[12]:    二级间接块 (1个, 指向256个一级间接块)
4
5  最大文件大小 = (10 + 2×256 + 256×256) × 1024 = 67,637,248 字节 ≈ 64.5
   MB
6
```

核心函数:

1. **inode_alloc(uint16 inode_num):** 在内存中分配或查找 inode
 - 先在 inode 缓存中查找
 - 未找到则分配新的内存 inode
 - 增加引用计数
2. **inode_create(uint16 type, uint16 major, uint16 minor):** 创建新 inode
 - 在磁盘上分配 inode 编号
 - 初始化 inode 元数据
 - 写回磁盘
3. **inode_lock(inode_t ip)*:** 锁定 inode
 - 获取 inode 的 spinlock
 - 如果 inode 未加载 (valid = false), 从磁盘读取
4. **inode_read_data() / inode_write_data():** 读写文件数据
 - 根据偏移量计算数据块位置
 - 处理三级索引 (直接块、间接块、二级间接块)
 - 支持跨块读写
5. **inode_free_data(inode_t ip)*:** 释放 inode 占用的数据块
 - 释放直接块
 - 释放一级间接块及其指向的数据块
 - 释放二级间接块及其指向的所有块

数据块寻址算法:

```
1 // 给定文件偏移量 offset, 计算对应的数据块号
```

C

```

2 uint32 block_offset = offset / BLOCK_SIZE;
3
4 if (block_offset < 10) {
5     // 直接块
6     block_num = ip->addrs[block_offset];
7 } else if (block_offset < 10 + 2 * 256) {
8     // 一级间接块
9     int idx = (block_offset - 10) / 256; // 使用哪个间接块
10    int off = (block_offset - 10) % 256; // 间接块内的偏移
11    // 读取间接块，获取实际数据块号
12 } else {
13     // 二级间接块
14     // 需要两次间接寻址
15 }
16

```

7.6.5.5. 目录管理 (dir.c)

****目录结构：**目录是特殊的文件，其数据内容是 `dirent_t` 结构的数组。

路径解析机制：

1. `path_to_inode(char path)*`: 将路径转换为 inode

```

1 // 示例: "/home/user/file.txt"
2 // 1. 从根目录开始 (inode 1)
3 // 2. 查找 "home" → 得到 home 的 inode
4 // 3. 查找 "user" → 得到 user 的 inode
5 // 4. 查找 "file.txt" → 得到 file.txt 的 inode
6

```

C

2. `path_to_pinode(char path, char name)**`: 获取父目录 inode

```

1 // 示例: "/home/user/file.txt"
2 // 返回 user 目录的 inode
3 // name 参数返回 "file.txt"
4

```

C

3. `search_inode(inode_t pip, char name, int namelen)**`: 在目录中查找

- 遍历目录项数组
- 比较文件名
- 返回匹配的 inode 编号

核心操作：

- `dir_search_entry()`: 在目录中查找条目
- `dir_add_entry()`: 向目录添加新条目

- 先查找是否已存在
- 在目录数据中找空闲位置或扩展目录
- **dir_delete_entry()**: 删除目录条目
 - 将 inode_num 设为 0 表示空闲
- **dir_get_entries()**: 获取目录所有条目（用于 ls 命令）

特殊目录项：

- `.`: 指向当前目录
- `..`: 指向父目录

7.6.6.6. 文件操作 (file.c)

文件描述符机制：

每个进程维护一个文件描述符表 `filelist[FILE_PER_PROC]`，数组下标就是文件描述符号。

文件结构：

```

1 typedef struct file {
2     uint16 type;        // FILE_INODE, FILE_PIPE, FILE_DEVICE
3     uint32 ref;         // 引用计数（支持 dup）
4     bool readable;
5     bool writable;
6     inode_t* ip;        // 对应的 inode
7     uint32 off;         // 当前文件偏移量
8 } file_t;
9

```

核心函数：

1. **file_open()**: 打开文件

- 查找或创建 inode
- 分配 file 结构
- 设置读写权限
- 返回文件描述符

2. **file_read()** / **file_write()**: 读写文件

- 检查权限
- 调用 `inode_read_data()` 或 `inode_write_data()`
- 更新文件偏移量

3. **file_close()**: 关闭文件

- 减少引用计数
 - 引用计数为 0 时释放 file 结构和 inode
4. **file_stat():** 获取文件状态
 - 返回文件大小、类型、inode 号等信息
-

7.6.7.7. 系统调用层 (sysfile.c)

文件操作系统调用：

1. **sys_open():** 打开/创建文件

```
1 // 参数: path, flags
2 // 返回: 文件描述符或 -1
3 // 支持标志: O_RDONLY, O_WRONLY, O_RDWR, O_CREATE
4
```

C

2. **sys_read() / sys_write():** 读写文件

```
1 // 参数: fd, buffer, count
2 // 返回: 实际读写的字节数
3
```

C

3. **sys_lseek():** 设置文件偏移

```
1 // 参数: fd, offset, whence
2 // whence: SEEK_SET, SEEK_CUR, SEEK_END
3
```

C

4. **sys_dup():** 复制文件描述符

```
1 // 允许多个 fd 指向同一个文件
2 // 支持重定向等操作
3
```

C

目录操作系统调用：

1. **sys_mkdir():** 创建目录

- 创建类型为 INODE_DIR 的 inode
- 添加 “.” 和 “..” 条目

2. **sys_chdir():** 改变当前工作目录

- 更新进程的 `cwd` 字段

3. **sys_link():** 创建硬链接

- 增加 inode 的 nlink 计数

- 在目标目录添加新条目
4. **sys_unlink()**: 删除文件/链接
- 减少 inode 的 nlink 计数
 - nlink 为 0 时释放 inode 和数据块

7.6.8.8. ELF 文件执行 (sys_exec)

ELF 加载流程:

1. 验证 ELF 文件

- 检查魔数 `0x464C457F` (“\x7FELF”)
- 验证文件格式正确性

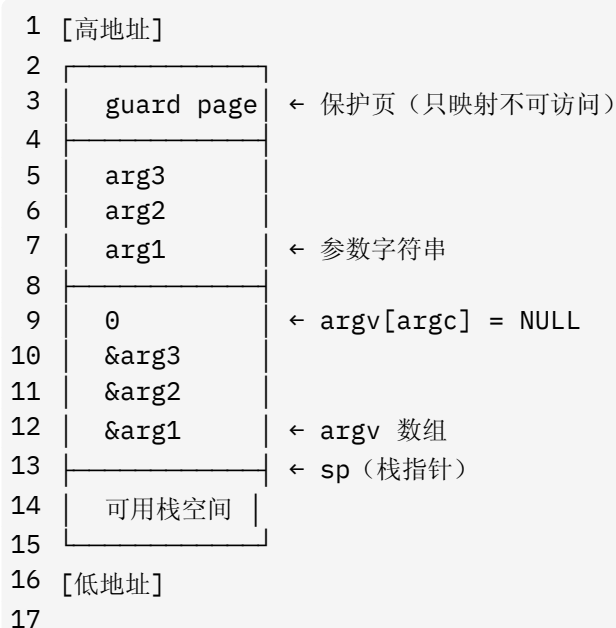
2. 创建新的地址空间

- 分配新的页表
- 映射 trampoline 和 trapframe

3. 加载程序段

- 1 对于每个 LOAD 类型的程序段:
- 2 - 分配物理页面
- 3 - 映射到虚拟地址空间
- 4 - 设置正确的权限 (R/W/X)
- 5 - 从 ELF 文件读取段内容
- 6

4. 设置用户栈



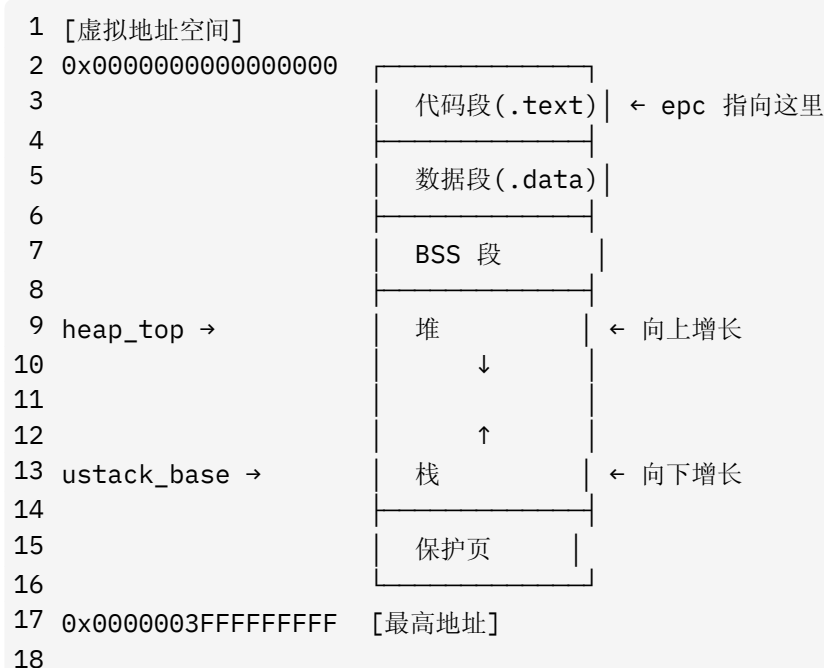
5. 设置执行上下文

- `tf->epc = elf.entry`: 程序入口点
- `tf->sp = sp`: 栈指针
- `tf->a1 = sp`: `argv` 参数
- 返回值 = `argc` (通过 `a0` 寄存器)

6. 替换地址空间

- 释放旧页表
- 切换到新页表
- 更新进程的堆、栈信息

内存布局 (执行后):



7. 7. 测试

7. 7. 1. inode 读写测试

测试代码

```

1 #include "fs/fs.h"
2 #include "fs/buf.h"
3 #include "fs/bitmap.h"
4 #include "fs/inode.h"
5 #include "fs/dir.h"
6 #include "lib/str.h"
7 #include "lib/print.h"
8
9 // 超级块在内存的副本
10 super_block_t sb;
```

C++

```

11
12 #define FS_MAGIC 0x12345678
13 #define SB_BLOCK_NUM 0
14
15 // 测试用的数组（暂未使用）
16 static uint8 str[BLOCK_SIZE * 2];
17 static uint8 tmp[BLOCK_SIZE * 2];
18
19 // 比较两个大小为 2*BLOCK_SIZE 的空间是否完全一样
20
21 static bool blockcmp(uint8* a, uint8* b)
22 {
23     for(int i = 0; i < BLOCK_SIZE * 2; i++) {
24         if(a[i] != b[i])
25             return false;
26     }
27     return true;
28 }
29
30
31 // 输出super_block的信息
32 static void sb_print()
33 {
34     printf("\nsuper block information:\n");
35     printf("magic = %x\n", sb.magic);
36     printf("block size = %d\n", sb.block_size);
37     printf("inode blocks = %d\n", sb.inode_blocks);
38     printf("data blocks = %d\n", sb.data_blocks);
39     printf("total blocks = %d\n", sb.total_blocks);
40     printf("inode bitmap start = %d\n", sb.inode_bitmap_start);
41     printf("inode start = %d\n", sb.inode_start);
42     printf("data bitmap start = %d\n", sb.data_bitmap_start);
43     printf("data start = %d\n", sb.data_start);
44 }
45
46 // 文件系统初始化
47 void fs_init()
48 {
49     buf_init();
50
51     buf_t* buf;
52     buf = buf_read(SB_BLOCK_NUM);
53     memmove(&sb, buf->data, sizeof(sb));
54     assert(sb.magic == FS_MAGIC, "fs_init: magic");
55     assert(sb.block_size == BLOCK_SIZE, "fs_init: block size");
56     buf_release(buf);
57     sb_print();
58
59     // inode初始化
60     inode_init();
61

```

```

62     printf("\nFile system initialized\n");
63
64     uint32 ret = 0;
65
66     for(int i = 0; i < BLOCK_SIZE * 2; i++)
67         str[i] = i;
68
69     // 创建新的inode
70     inode_t* nip = inode_create(FT_FILE, 0, 0);
71     inode_lock(nip);
72
73     // 第一次查看
74     inode_print(nip);
75
76     // 第一次写入
77     ret = inode_write_data(nip, 0, BLOCK_SIZE / 2, str, false);
78     assert(ret == BLOCK_SIZE / 2, "inode_write_data: fail");
79
80     // 第二次写入
81     ret = inode_write_data(nip, BLOCK_SIZE / 2, BLOCK_SIZE +
BLOCK_SIZE / 2, str + BLOCK_SIZE / 2, false);
82     assert(ret == BLOCK_SIZE + BLOCK_SIZE / 2,
"inode_write_data: fail");
83
84     // 一次读取
85     ret = inode_read_data(nip, 0, BLOCK_SIZE * 2, tmp, false);
86     assert(ret == BLOCK_SIZE * 2, "inode_read_data: fail");
87
88     // 第二次查看
89     inode_print(nip);
90
91     inode_unlock_free(nip);
92
93     // 测试
94     if(blockcmp(tmp, str) == true)
95         printf("success");
96     else
97         printf("fail");
98
99     while (1);
100 }
101

```

测试结果

```

hwt@MP:~/桌面/sources/lab/OSLab/WHUOS/whu-oslab-lab7/code$ make clean && make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 1 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-
-bus.0
initcode:0x00000000000009640

super block information:
magic = 12345678
block size = 1024
inode blocks = 128
data blocks = 8192
total blocks = 8323
inode bitmap start = 1
inode start = 2
data bitmap start = 130
data start = 131

File system initialized

inode information:
num = 1, ref = 1, valid = 1
type = INODE_FILE, major = 0, minor = 0, nlink = 1
size = 0, addrs = 0 0 0 0 0 0 0 0 0 0 0 0

inode information:
num = 1, ref = 1, valid = 1
type = INODE_FILE, major = 0, minor = 0, nlink = 1
size = 2048, addrs = 132 133 0 0 0 0 0 0 0 0 0 0
success

```

7.7.2. 路径测试

测试代码

C++

```

1 #include "fs/fs.h"
2 #include "fs/buf.h"
3 #include "fs/bitmap.h"
4 #include "fs/inode.h"
5 #include "fs/dir.h"
6 #include "lib/str.h"
7 #include "lib/print.h"
8
9 // 超级块在内存的副本
10 super_block_t sb;
11
12 #define FS_MAGIC 0x12345678
13 #define SB_BLOCK_NUM 0
14
15 // 输出super_block的信息
16 static void sb_print()
17 {
18     printf("\nsuper block information:\n");
19     printf("magic = %x\n", sb.magic);
20     printf("block size = %d\n", sb.block_size);
21     printf("inode blocks = %d\n", sb.inode_blocks);
22     printf("data blocks = %d\n", sb.data_blocks);
23     printf("total blocks = %d\n", sb.total_blocks);
24     printf("inode bitmap start = %d\n", sb.inode_bitmap_start);
25     printf("inode start = %d\n", sb.inode_start);
26     printf("data bitmap start = %d\n", sb.data_bitmap_start);
27     printf("data start = %d\n", sb.data_start);
28 }
29
30 // 文件系统初始化
31 void fs_init()
32 {
33     buf_init();
34
35     buf_t* buf;
36     buf = buf_read(SB_BLOCK_NUM);

```

```

37     memmove(&sb, buf->data, sizeof(sb));
38     assert(sb.magic == FS_MAGIC, "fs_init: magic");
39     assert(sb.block_size == BLOCK_SIZE, "fs_init: block size");
40     buf_release(buf);
41     sb_print();
42
43     // inode初始化
44     inode_init();
45
46     printf("\nFile system initialized\n");
47
48     // 创建inode
49     inode_t* ip = inode_alloc(INODE_ROOT);
50     inode_t* ip_1 = inode_create(FT_DIR, 0, 0);
51     inode_t* ip_2 = inode_create(FT_DIR, 0, 0);
52     inode_t* ip_3 = inode_create(FT_FILE, 0, 0);
53
54     // 上锁
55     inode_lock(ip);
56     inode_lock(ip_1);
57     inode_lock(ip_2);
58     inode_lock(ip_3);
59
60     // 创建目录
61     dir_add_entry(ip, ip_1->inode_num, "user");
62     dir_add_entry(ip_1, ip_2->inode_num, "work");
63     dir_add_entry(ip_2, ip_3->inode_num, "hello.txt");
64
65     // 填写文件
66     inode_write_data(ip_3, 0, 11, "hello world", false);
67
68     // 解锁
69     inode_unlock(ip_3);
70     inode_unlock(ip_2);
71     inode_unlock(ip_1);
72     inode_unlock(ip);
73
74     // 路径查找
75     char* path = "/user/work/hello.txt";
76     char name[DIR_NAME_LEN];
77     inode_t* tmp_1 = path_to_pinode(path, name);
78     inode_t* tmp_2 = path_to_inode(path);
79
80     assert(tmp_1 != NULL, "tmp1 = NULL");
81     assert(tmp_2 != NULL, "tmp2 = NULL");
82     printf("\nname = %s\n", name);
83
84     // 输出 tmp_1 的信息
85     inode_lock(tmp_1);
86     inode_print(tmp_1);
87     inode_unlock_free(tmp_1);

```

```

88
89     // 输出 tmp_2 的信息
90     inode_lock(tmp_2);
91     inode_print(tmp_2);
92     char str[12];
93     str[11] = 0;
94     inode_read_data(tmp_2, 0, tmp_2->size, str, false);
95     printf("read: %s\n", str);
96     inode_unlock_free(tmp_2);
97
98     printf("over");
99     while (1);
100 }
101

```

测试结果

```

hwt@MP:~/桌面/sources/lab/OSLab/WHUOS/whu-oslab-lab7/code$ make clean && make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 1 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
initcode:0x0000000000000960

super block information:
magic = 12345678
block size = 1024
inode blocks = 128
data blocks = 8192
total blocks = 8323
inode bitmap start = 1
inode start = 2
data bitmap start = 130
data start = 131

File system initialized

name = hello.txt

inode information:
num = 2, ref = 2, valid = 1
type = INODE_DIR, major = 0, minor = 0, nlink = 1
size = 32, addrs = 133 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

inode information:
num = 3, ref = 2, valid = 1
type = INODE_FILE, major = 0, minor = 0, nlink = 1
size = 11, addrs = 134 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
read: hello world
over

```

7.7.3. 目录测试

测试代码

```

1 #include "fs/fs.h"
2 #include "fs/buf.h"
3 #include "fs/bitmap.h"
4 #include "fs/inode.h"
5 #include "fs/dir.h"
6 #include "lib/str.h"
7 #include "lib/print.h"
8
9 // 超级块在内存的副本
10 super_block_t sb;
11
12 #define FS_MAGIC 0x12345678
13 #define SB_BLOCK_NUM 0
14
15 // 输出super_block的信息
16 static void sb_print()
17 {

```

C++


```

18     printf("\nsuper block information:\n");
19     printf("magic = %x\n", sb.magic);
20     printf("block size = %d\n", sb.block_size);
21     printf("inode blocks = %d\n", sb.inode_blocks);
22     printf("data blocks = %d\n", sb.data_blocks);
23     printf("total blocks = %d\n", sb.total_blocks);
24     printf("inode bitmap start = %d\n", sb.inode_bitmap_start);
25     printf("inode start = %d\n", sb.inode_start);
26     printf("data bitmap start = %d\n", sb.data_bitmap_start);
27     printf("data start = %d\n", sb.data_start);
28 }
29
30 // 文件系统初始化
31 void fs_init()
32 {
33     buf_init();
34
35     buf_t* buf;
36     buf = buf_read(SB_BLOCK_NUM);
37     memmove(&sb, buf->data, sizeof(sb));
38     assert(sb.magic == FS_MAGIC, "fs_init: magic");
39     assert(sb.block_size == BLOCK_SIZE, "fs_init: block size");
40     buf_release(buf);
41     sb_print();
42
43     // inode初始化
44     inode_init();
45
46     printf("\nFile system initialized\n");
47
48     // 获取根目录
49     inode_t* ip = inode_alloc(INODE_ROOT);
50     inode_lock(ip);
51
52     // 第一次查看
53     dir_print(ip);
54
55     // add entry
56     dir_add_entry(ip, 1, "a.txt");
57     dir_add_entry(ip, 2, "b.txt");
58     dir_add_entry(ip, 3, "c.txt");
59
60     // 第二次查看
61     dir_print(ip);
62
63     // 第一次检查
64     assert(dir_search_entry(ip, "b.txt") == 2, "error-1");
65
66     // delete entry
67     dir_delete_entry(ip, "a.txt");
68

```

```

69     // 第三次查看
70     dir_print(ip);
71
72     // add entry
73     dir_add_entry(ip, 1, "d.txt");
74
75     // 第四次查看
76     dir_print(ip);
77
78     // 第二次检查
79     assert(dir_add_entry(ip, 4, "d.txt") == BLOCK_SIZE, "error-2");
80
81     inode_unlock(ip);
82
83     printf("over");
84
85     while (1);
86 }
87

```

测试结果

```

hwt@HP:~/桌面/sources/lab/OSLab/WHUOS/whu-oslab-lab7/code$ make clean && make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 1 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
initcode:0x00000000000009630

super block information:
magic = 12345678
block size = 1024
inode blocks = 128
data blocks = 8192
total blocks = 8320
inode bitmap start = 1
inode start = 2
data bitmap start = 130
data start = 131

File system initialized

inode_num = 0 dirents:
inum = 0 dirent = .
inum = 0 dirent = ..

inode_num = 0 dirents:
inum = 0 dirent = .
inum = 0 dirent = ..
inum = 1 dirent = a.txt
inum = 2 dirent = b.txt
inum = 3 dirent = c.txt

inode_num = 0 dirents:
inum = 0 dirent = .
inum = 0 dirent = ..
inum = 2 dirent = b.txt
inum = 3 dirent = c.txt

inode_num = 0 dirents:
inum = 0 dirent = .
inum = 0 dirent = ..
inum = 1 dirent = d.txt
inum = 2 dirent = b.txt
inum = 3 dirent = c.txt
dir_add_entry: name exists
over

```